CATHOLIC UNIVERSITY OF EASTERN AFRICA

PROJECT PROGRESS FOR FINAL YEAR STUDY IN BACHELOR IN COMPUTER SCIENCE

BY

JULIUS MAINGI NJUGUNA ,1062811

PROJECT TITLE

INTERNATIONAL BUS BOOKING SYSTEM

DATE: [MARCH, 2026]

BACKGROUND STUDY

1. Introduction

In recent years, Kenya and neighboring East African countries have embraced digital transformation in transport management. Many transport companies have adopted **online booking systems** that allow passengers to **reserve seats**, **select travel dates**, and **pay using M-Pesa or cards** without visiting physical offices.

However, while these systems have improved convenience for **domestic travel**, challenges remain when it comes to **international routes** (e.g., Kenya–Uganda, Kenya–Tanzania). Systems are often **not interconnected**, passengers must **print tickets physically**, and **border clearance coordination** is largely manual.

1. Passengers Must Print Tickets Physically

Even though passengers **book and pay online**, many transport companies (especially across East Africa) still **require printed tickets** for verification at the bus station.

What this means

- After booking through the company's website or app (e.g., Modern Coast, Mash Poa, or Easy Coach), the passenger receives:
    - A **confirmation message** (SMS or email) with a **booking reference,** booking ID or **ticket number**.
- However, when arriving at the bus station, **they must visit a physical counter** to:
    - Confirm their booking manually from the company's system.
    - Print the actual ticket slip before boarding.

Why this happens

1. **Verification issues** – Some systems at bus stations are not fully integrated with online databases in real time, so staff rely on printed tickets as proof of payment.
2. **Limited infrastructure** – Conductors or drivers might not have handheld devices or scanners to verify digital tickets directly.
3. **Connectivity problems** – Network outages or slow systems can make digital verification unreliable.
4. **Regulatory requirements** – In some cases, transport authorities still require printed tickets for record-keeping, insurance, or audit purposes.

Implications

- Increased **waiting time** for passengers (queues to print).
- Extra **operational costs** (paper, printers, staffing).
- Reduced **environmental sustainability** (paper waste).
- Poor **customer experience**, especially for cross-border travelers who expect fully digital boarding.

  The gap my project will address based on the problems above is:

- To Implement a system that **enables digital ticket verification** (QR codes or e-tickets).

Overview of the Domain Area

The **bus transport sector in Kenya** plays a crucial role in both domestic and regional mobility. With the integration of **digital payments (M-Pesa)** and **online booking**, passengers can now book tickets from anywhere.

However, in **international travel**, such as routes connecting **Nairobi–Kampala** or **Nairobi–Arusha**, operations face unique challenges:

- **Manual check-in** at stations or border points.
- **Physical ticket printing** required for verification.
- Lack of **system integration** across borders.

Thus, the domain area includes:

- **Online ticketing**
- **Seat management**
- **Bus scheduling**
- **Passenger verification**

1. Online Ticketing

This is the core function of the system.
It allows passengers to:

- Browse available routes, dates, and bus operators.

- Select a preferred bus, seat, and travel date.
- Generate an **electronic ticket (e-ticket)** or **QR code** once payment is completed.
- Access their ticket from any device from the website.

Why is this important:
It removes the need to visit a bus station physically and minimizes errors caused by manual booking.

---

2. **Seat Management**

This feature ensures accurate and real-time seat reservation.
It allows:

- Passengers to **view seat availability** in real time.
- Automatic seat locking once a passenger begins a booking.
- Prevention of **double-booking** or **overbooking**.

Why important:
It improves user experience and ensures that bus operators can efficiently plan their trips and capacities.

---

3. **Bus Scheduling**

This manages **bus departures, arrivals, and route planning**.
The system records:

- Departure and arrival times.
- Stops along the route.
- Assigned buses and drivers.

Why important:
It allows managers to monitor fleet usage, optimize time, and ensure punctuality across routes.

---

4. **Passenger Verification**

This component ensures that passengers boarding the bus are **genuine ticket holders**.
It may involve:

- Scanning **QR codes or confirmation codes** at the station.
- Validating tickets using mobile or handheld devices.
- Matching ticket details to passenger identification.

Why important:
It enhances security, prevents ticket fraud, and speeds up the check-in process.

Problem Statement

The "Manual Verification Bottleneck"

Despite the growth of online booking and digital payments, the international bus industry remains slowed down by a **"Last-Mile Manual Bottleneck."** Passengers are still forced to print physical tickets or present confirmation codes that terminal staff must check by hand. This reliance on manual verification causes long queues at boarding, increases the risk of fraud through counterfeit records, and makes it difficult for operators to maintain accurate passenger manifests for cross-border compliance.

Note: While full hardware integration for QR-scanning requires specialized mobile APIs, this project addresses the core challenge by providing a secure digital token framework for automated validation.

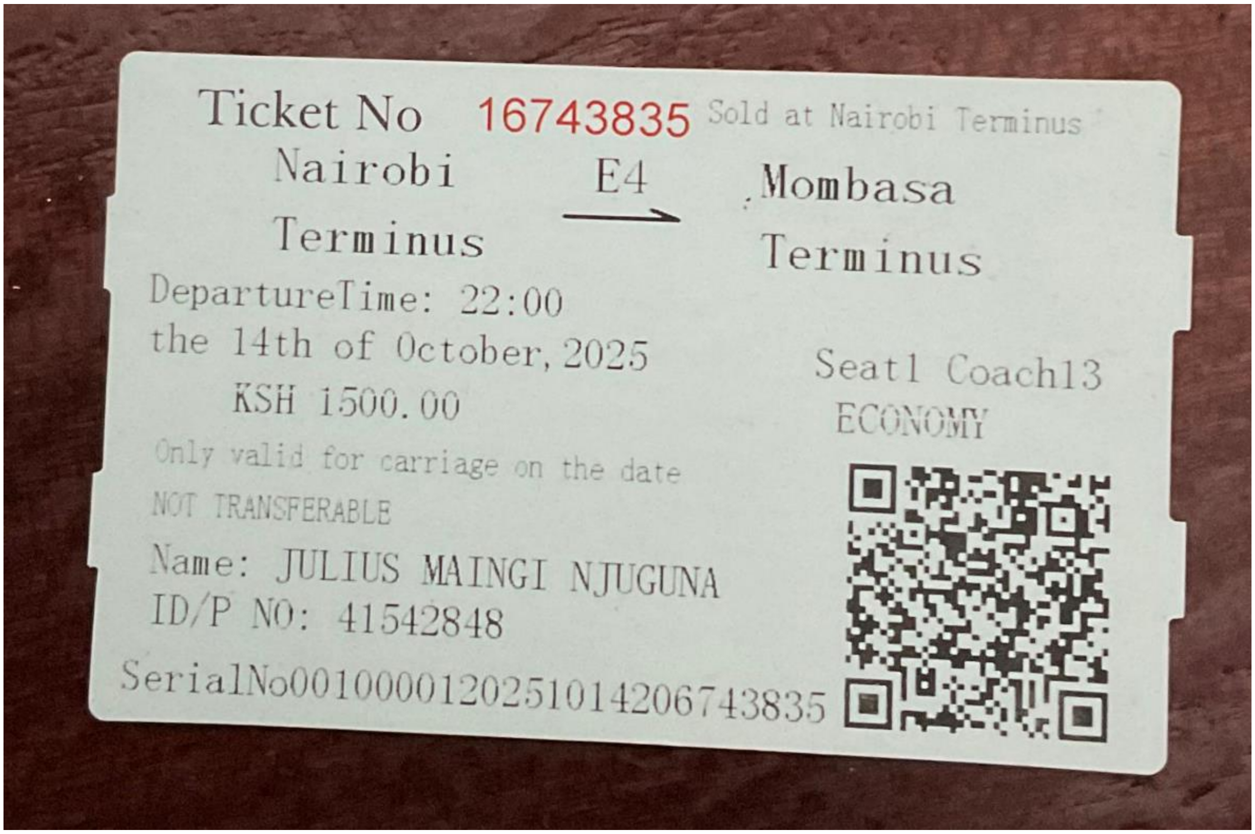
Methods I Used to Understand the Domain:

To fully understand the problem area, I applied the following methods:

Method	Description
Observation	I booked an SGR train and intercity bus online (Tahmeed Express) to experience the real system workflow.

Method	Description
Interviews	I spoke to bus staff and passengers (my relatives who have used online booking) about ticketing and check-in experiences.
Document Review	I studied booking receipts, SMS confirmations, and station ticket printouts.
Online Research	I reviewed bus company websites and apps (i.e, Madaraka Express, Tahmeed Express, Modern Coast, Easy-Coach, ENA Coach, Dreamline).
Comparative Study	I compared Kenya's systems with other regional or global online bus booking services (e.g. FlixBus, Greyhound).

Document reviews/Observations



Ticket No 16743835 Sold at Nairobi Terminus

Nairobi Terminus → E4 Mombasa Terminus

DepartureTime: 22:00

the 14th of October, 2025

KSH 1500.00

Seat1 Coach13

ECONOMY

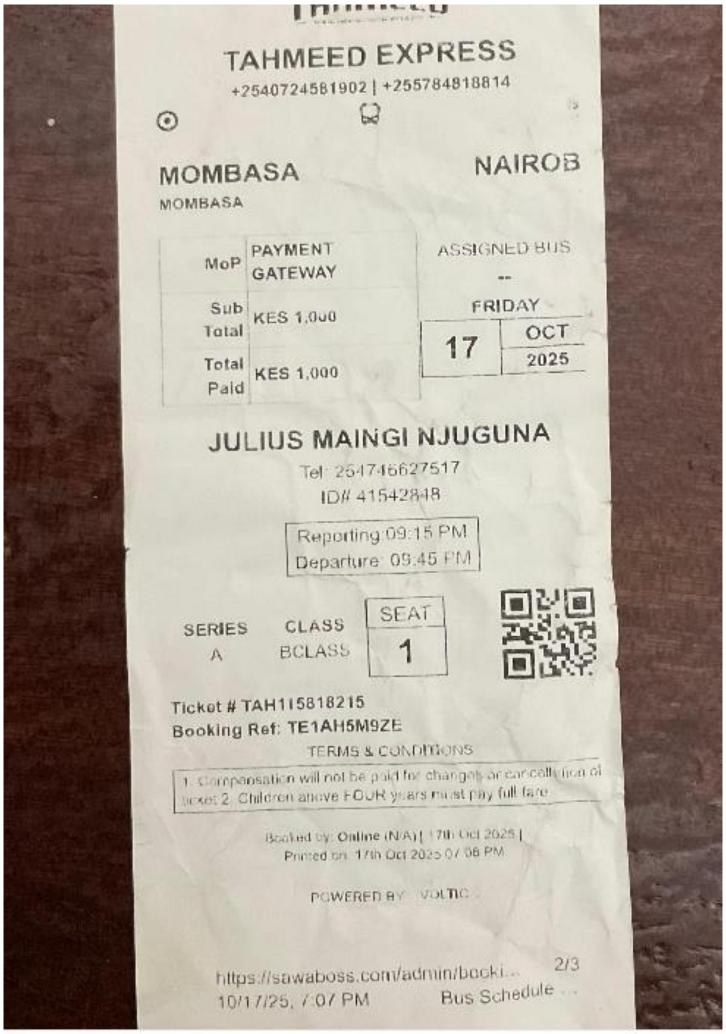
Only valid for carriage on the date

NOT TRANSFERABLE

Name: JULIUS MAINGI NJUGUNA

ID/P NO: 41542848

SerialNo0010000120251014206743835



System Scope

The IBBS is designed to be a global, multi-user platform for managing international bus transport. The system's scope includes the following capabilities:

- **Global User Registration**: Enables customers from anywhere in the world to create accounts and manage their travel profiles.
- **Multi-Role Access Control**: Specifically tailored portals for Passengers, Agents, and Administrators with restricted access based on roles.
- **Virtual Route Management**: Allows administrators to define international routes, set departure times, and adjust dynamic pricing.
- **Fleet & Crew Integration**: Management of physical bus assets and assignment of licensed drivers to specific vehicles.
- **Real-Time Booking Engine**:
  - ✓ Self-Service: For registered passengers using their mobile/web accounts.
  - ✓ Walk-in Office: For Agents to register tickets for customers without accounts.
- **Automated Payment Simulation**: Processes bookings automatically, simulating a seamless transaction flow.
- **Seat Allocation System**: Visualizes bus layouts and ensures unique seat numbers per trip to prevent overbooking.
- **Digital Ticket Retrieval**: Generates unique hashes and digital receipts (QR tokens) for passenger verification.
- **Feedback & Quality Control**: A system for passengers to rate their trips and for administrators to monitor service standards.
- **Revenue & Operational Insights**: Advanced reporting modules for daily, weekly, monthly, and yearly business performance analysis.
- **Occupancy Monitoring**: Real-time visibility into bus capacity for better logistical planning.

Literature Review: International Bus Booking Systems (IBBS)

Review 1: Modern Coast Express Online Reservation System

Project Title: Modern Coast Express Online Reservation System
Owner: Modern Coast Express Ltd
Region of Use: Kenya, Uganda, Tanzania, and Rwanda

1. Overview of the System

Modern Coast Express operates a centralized online booking system designed for long-distance and cross-border travel within East Africa. The system is accessible through a web portal and a mobile application, allowing passengers to search for routes, compare departure times, and select seats using a digital seat map. Customers can view available seats in real time and choose their preferred location (window, aisle, front, or back).

Payment is integrated through secure digital channels such as M-Pesa and major credit/debit cards. Once payment is completed, the system generates an electronic ticket containing a QR code. This QR code is scanned at the terminal before boarding, reducing the need for printed tickets and manual verification.

2. System Functionality

The platform is connected to a fleet management module. Each bus is equipped with GPS tracking devices that send location data to the central system. This allows:

- Real-time tracking of buses
- Estimated arrival time notifications
- Monitoring of route deviations
- Improved dispatch coordination

The system also manages booking records, payment confirmations, and passenger manifests. This ensures accurate passenger counts and improves operational efficiency at border checkpoints.

3. Challenges Encountered

- **Network Latency:** Because the system operates across multiple countries, syncing seat availability between regional offices can sometimes delay updates, leading to rare cases of double booking.
- **Multi-Currency Management:** Cross-border travel requires handling multiple currencies (KES, UGX, TZS). Exchange rate fluctuations and payment gateway conversions add technical complexity.
- **Cross-Border Regulations:** Different immigration and customs policies require accurate passenger data collection and storage.

4. Conclusion

Modern Coast positions itself as a premium service provider, offering WiFi-enabled coaches and comfortable seating. The integration of GPS tracking enhances safety monitoring and route optimization. The system also collects data analytics such as route demand trends and peak travel periods, helping management make strategic decisions regarding fleet allocation and pricing.

From a software architecture perspective, the system likely operates on a centralized database with distributed access from regional offices. This architecture improves coordination but requires strong database synchronization mechanisms.

References

Modern Coast Coaches App Description.

https://modern-coast-coaches.en.softonic.com/android

Review 2: EasyCoach Passenger Management System

Project Title: EasyCoach Passenger Management System
Owner: EasyCoach Kenya Limited
Region of Use: Kenya

1. Overview of the System

EasyCoach operates a hybrid booking and dispatch management system designed for national routes across Kenya. Unlike fully online platforms, EasyCoach combines digital booking with in-person ticket sales at terminals.

Passengers may book tickets online via the company website or mobile platforms. However, physical ticket agents at bus stations use a desktop-based application to register walk-in customers. The system synchronizes bookings between online and offline channels to prevent seat conflicts.

2. System Functionality

Key features include:

- Interactive seat selection
- M-Pesa and card payments
- Driver allocation and scheduling
- Route optimization
- Parcel/courier tracking integration

The parcel tracking module is particularly significant. EasyCoach transports both passengers and cargo. The system records parcel details, sender and receiver information, and delivery status in the same centralized database as passenger bookings. This integration increases operational efficiency.

Additionally, the system manages driver schedules to ensure compliance with labor laws and reduce fatigue-related risks.

3. Challenges Encountered

- **Offline Operations:** Some upcountry stations experience unstable internet connections. The system must allow offline ticket entry and later synchronization.
- **Cash Reconciliation:** Cash transactions at terminals must be matched with digital records, requiring accurate accounting controls.

- **Operational Coordination:** Coordinating passenger and cargo operations within one system increases complexity.

4. Additional Relevant Details

EasyCoach has built a reputation for punctuality and professionalism. Its management system supports performance monitoring, including tracking departure times, bus occupancy rates, and route profitability.

From a systems design perspective, the hybrid model requires strong database consistency mechanisms to avoid duplication or inconsistencies between online and offline records. The system demonstrates how transport companies in developing regions must design solutions that function effectively even with limited connectivity.

References

Easy Coach Online Booking Portal and Tickets via Mobile.

https://kenyapeaks.com/travel-booking/easy-coach

Review 3: Greyhound TRIPS Reservation System

Project Title: Greyhound TRIPS (Transportation Real-time Inventory and Passenger System)
Owner: Greyhound Lines (Acquired by FlixMobility in 2021)
Region of Use: United States, Canada, Mexico

1. Overview of the System

Greyhound operates one of the largest bus reservation systems in North America. The TRIPS system is designed to handle thousands of daily routes and millions of annual passengers. It functions similarly to airline reservation systems.

2. System Functionality

Key features include:

- Dynamic pricing (yield management)

- Real-time seat inventory management
- Online and third-party booking integration
- Passenger manifest generation
- Border compliance data handling

The system adjusts ticket prices based on demand, time of booking, and travel season. During high-demand periods such as holidays, prices increase to maximize revenue.

It also supports API integration, allowing third-party travel agencies and aggregators to sell Greyhound tickets.

3. Challenges Encountered

- **Scalability:** The system must handle extremely high traffic volumes, especially during peak travel seasons.
- **Security Compliance:** For cross-border travel, the system must collect accurate passenger information and comply with border security regulations.
- **System Reliability:** Downtime can result in significant financial losses due to the scale of operations.

4. Additional Relevant Details

Greyhound's transition to a cloud-based architecture reflects modernization efforts. Cloud infrastructure improves uptime, scalability, and system resilience.

The TRIPS system demonstrates advanced concepts such as distributed databases, load balancing, and high-availability infrastructure. It serves as a model for large-scale transportation booking systems.

References

Greyhound Lines – Wikipedia.

https://en.wikipedia.org/wiki/Greyhound_Lines

Review 4: RedBus Global Aggregator Platform

Project Title: RedBus Global Aggregator Platform
Owner: redBus
Region of Use: India, Malaysia, Indonesia, Singapore, Peru, Colombia

1. Overview of the System

RedBus is not a bus operator but an online marketplace that connects passengers to multiple bus operators through one platform. It aggregates seat inventories from thousands of independent operators.

2. System Functionality

The platform provides:

- Unified route search
- Real-time seat availability
- Mobile-first booking interface
- SeatSeller API integration
- Digital payment processing

Operators upload schedules and seat data via the SeatSeller API. Customers search routes and compare options across multiple bus companies in one interface.

3. Challenges Encountered

- **Data Standardization:** Different operators use different seat layouts and booking formats. RedBus had to create standardized templates.
- **Technical Support:** Small operators may lack advanced IT systems.
- **Real-Time Updates:** Delays in updating seat status can cause booking conflicts.

4. Additional Relevant Details

RedBus controls a major share of India's online bus ticket market. Its system demonstrates how API-driven architecture can integrate multiple independent vendors into a single ecosystem.

The aggregator model reduces infrastructure burden for small operators and expands customer reach. The system also collects big data analytics to study consumer travel behavior.

References

RedBus – Wikipedia.

https://en.wikipedia.org/wiki/RedBus

Review 5: Intercape Passenger & Freight Management System

Project Title: Intercape Passenger & Freight Management System
Owner: Intercape Mainliner
Region of Use: South Africa, Namibia, Botswana, Zimbabwe, Malawi

1. Overview of the System

Intercape provides long-distance luxury and sleeper bus services in Southern Africa. Its booking portal allows passengers to choose between sleeper bunks and standard seating.

2. System Functionality

The system includes:

- Class-based pricing (Sleeper vs Mainliner)
- Loyalty points system
- Freight and cargo management
- Driver log-in compliance tracking
- Online seat selection and payment

Passengers earn loyalty points for each ticket purchased. Points can be redeemed for future discounts.

3. Challenges Encountered

- **Remote Connectivity:** Some routes pass through areas with limited mobile network coverage.
- **Multi-Leg Travel Coordination:** Managing luggage and passenger transfers between buses is complex.
- **Cross-Border Regulations:** Southern African routes involve immigration documentation and data compliance.

4. Additional Relevant Details

Intercape integrates passenger and freight logistics within one database. The loyalty program enhances customer retention and brand loyalty.

The system highlights the importance of compliance monitoring, route tracking, and integrated logistics management in long-distance bus transport.

References

Intercape Loyalty Terms and Conditions.

https://www.intercape.co.za/loyalty-terms-and-conditions/

System Analysis

A. Data Analysis

The system utilizes a relational database to maintain data integrity and referential links between various transport entities.

Core Entities & Attributes:

1. **Users (users)**:

- Contains: user_id, first_name, last_name, email (unique), phone, password (hashed), and role.
- *Purpose*: The central identity registry for authentication and authorization.

2. **Drivers (drivers)**:
   - Contains: driver_id, national_id, full_name, phone, email.
   - *Purpose*: Maintains professional records for the bus crew.
3. **Buses (buses)**:
   - Contains: bus_id, reg_no, max_passengers, seat_layout, and driver_id.
   - *Purpose*: Represents physical vehicles and their current driver assignments.
4. **Routes (routes)**:
   - Contains: route_id, from_location, to_location, departure_date/time, cost, and bus_id.
   - *Purpose*: The schedule/plan for individual trips.
5. **Bookings (bookings)**:
   - Contains: booking_id, timestamp, seat_number, status, user_id, route_id, bus_id, and passenger metadata (name, age, ID).
   - *Purpose*: The primary transaction table linking users to specific trips.
6. **Feedback (feedback)**:
   - Contains: rating, comments, and links to the user, bus, and route.
   - *Purpose*: Performance and satisfaction metrics

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| ibbs               |
| ibbs_prototype     |
| information_schema |
| mysql              |
| performance_schema |
| phpmyadmin         |
| test               |
| wema_transport     |
+--------------------+
8 rows in set (0.16 sec)

mysql> use ibbs_prototype;
Database changed
mysql> show tables;
+--------------------------+
| Tables_in_ibbs_prototype |
+--------------------------+
| bookings                 |
| buses                    |
| drivers                  |
| feedback                 |
| routes                   |
| users                    |
+--------------------------+
6 rows in set (0.01 sec)
```

```
mysql> desc bookings;
+---------------------+-------------------------------------------------------+------+-----+---------------------+----------------+
| Field               | Type                                                  | Null | Key | Default             | Extra          |
+---------------------+-------------------------------------------------------+------+-----+---------------------+----------------+
| booking_id          | int(11)                                               | NO   | PRI | NULL                | auto_increment |
| booking_time        | datetime                                              | NO   |     | current_timestamp() |                |
| seat_number         | varchar(10)                                           | NO   |     | NULL                |                |
| booking_status      | enum('CONFIRMED','CANCELLED','PAID','CHECKED_IN')     | YES  |     | PAID                |                |
| qr_token            | varchar(255)                                          | YES  |     | NULL                |                |
| user_id             | int(11)                                               | NO   | MUL | NULL                |                |
| route_id            | int(11)                                               | NO   | MUL | NULL                |                |
| bus_id              | int(11)                                               | NO   | MUL | NULL                |                |
| passenger_name      | varchar(255)                                          | NO   |     |                     |                |
| passenger_age       | int(11)                                               | NO   |     | 0                   |                |
| passenger_dob       | date                                                  | YES  |     | NULL                |                |
| passenger_id_number | varchar(50)                                           | NO   |     |                     |                |
+---------------------+-------------------------------------------------------+------+-----+---------------------+----------------+
12 rows in set (0.01 sec)
```

```
mysql> desc buses;
+----------------+-------------+------+-----+---------+----------------+
| Field          | Type        | Null | Key | Default | Extra          |
+----------------+-------------+------+-----+---------+----------------+
| bus_id         | int(11)     | NO   | PRI | NULL    | auto_increment |
| reg_no         | varchar(20) | NO   | UNI | NULL    |                |
| bus_name       | varchar(50) | YES  |     | NULL    |                |
| max_passengers | int(11)     | NO   |     | NULL    |                |
| seat_layout    | varchar(20) | NO   |     | NULL    |                |
| driver_id      | int(11)     | YES  | MUL | NULL    |                |
+----------------+-------------+------+-----+---------+----------------+
6 rows in set (0.06 sec)

mysql> desc drivers;
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| driver_id   | int(11)      | NO   | PRI | NULL    | auto_increment |
| national_id | varchar(20)  | NO   | UNI | NULL    |                |
| full_name   | varchar(100) | NO   |     | NULL    |                |
| phone       | varchar(15)  | NO   | UNI | NULL    |                |
| email       | varchar(100) | NO   | UNI | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
5 rows in set (0.06 sec)
```

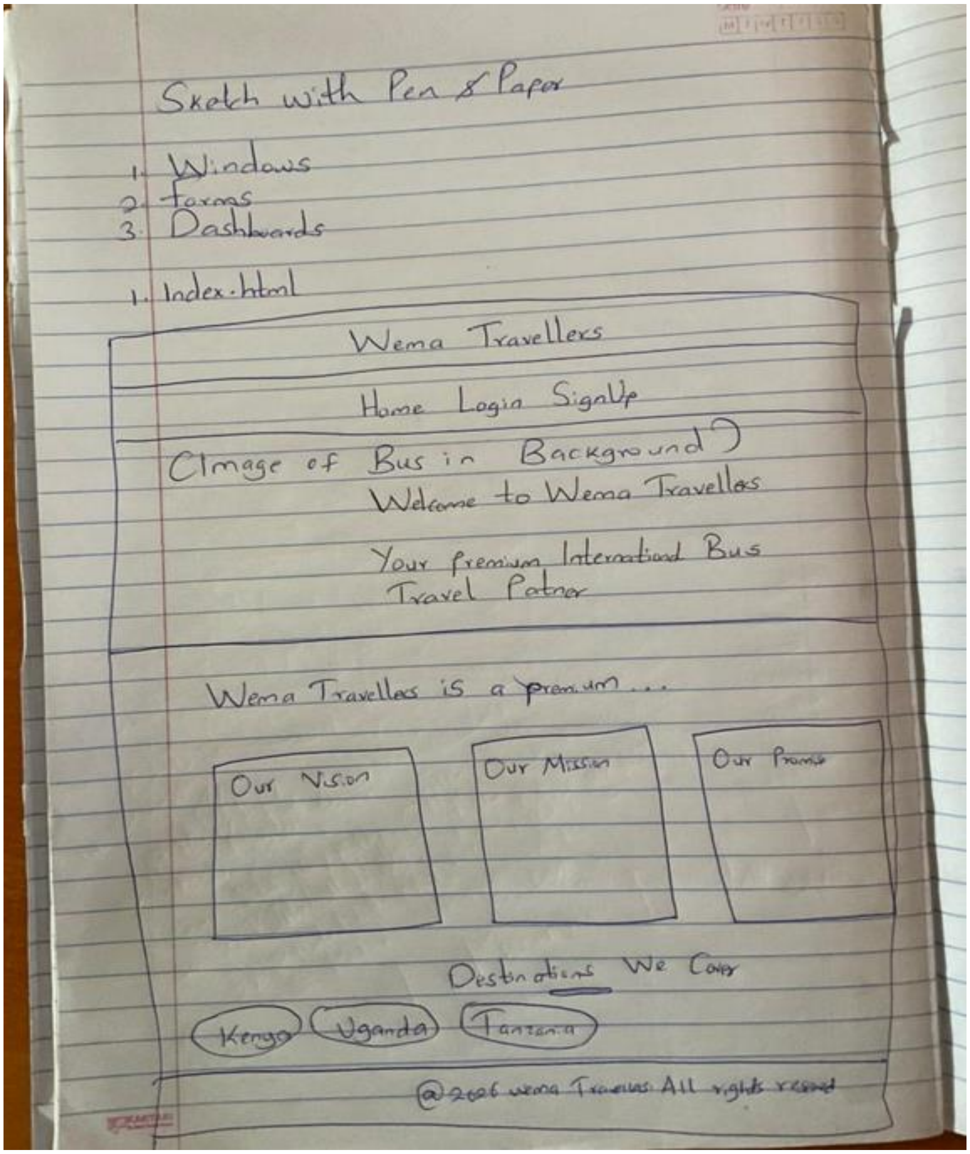
```
mysql> desc feedback;
+---------------+-----------+------+-----+---------------------+----------------+
| Field         | Type      | Null | Key | Default             | Extra          |
+---------------+-----------+------+-----+---------------------+----------------+
| feedback_id   | int(11)   | NO   | PRI | NULL                | auto_increment |
| rating        | int(11)   | YES  |     | 5                   |                |
| comments      | text      | NO   |     | NULL                |                |
| feedback_date | date      | NO   |     | NULL                |                |
| user_id       | int(11)   | NO   | MUL | NULL                |                |
| bus_id        | int(11)   | NO   | MUL | NULL                |                |
| route_id      | int(11)   | NO   | MUL | NULL                |                |
| submitted_at  | timestamp | NO   |     | current_timestamp() |                |
+---------------+-----------+------+-----+---------------------+----------------+
8 rows in set (0.06 sec)

mysql> desc routes;
+----------------+----------------------------------------------+------+-----+-----------+----------------+
| Field          | Type                                         | Null | Key | Default   | Extra          |
+----------------+----------------------------------------------+------+-----+-----------+----------------+
| route_id       | int(11)                                      | NO   | PRI | NULL      | auto_increment |
| from_location  | varchar(100)                                 | NO   |     | NULL      |                |
| to_location    | varchar(100)                                 | NO   |     | NULL      |                |
| departure_date | date                                         | NO   |     | NULL      |                |
| departure_time | time                                         | NO   |     | NULL      |                |
| cost           | decimal(10,2)                                | NO   |     | NULL      |                |
| bus_id         | int(11)                                      | NO   | MUL | NULL      |                |
| status         | enum('SCHEDULED','COMPLETED','CANCELLED')    | YES  |     | SCHEDULED |                |
+----------------+----------------------------------------------+------+-----+-----------+----------------+
8 rows in set (0.07 sec)
```

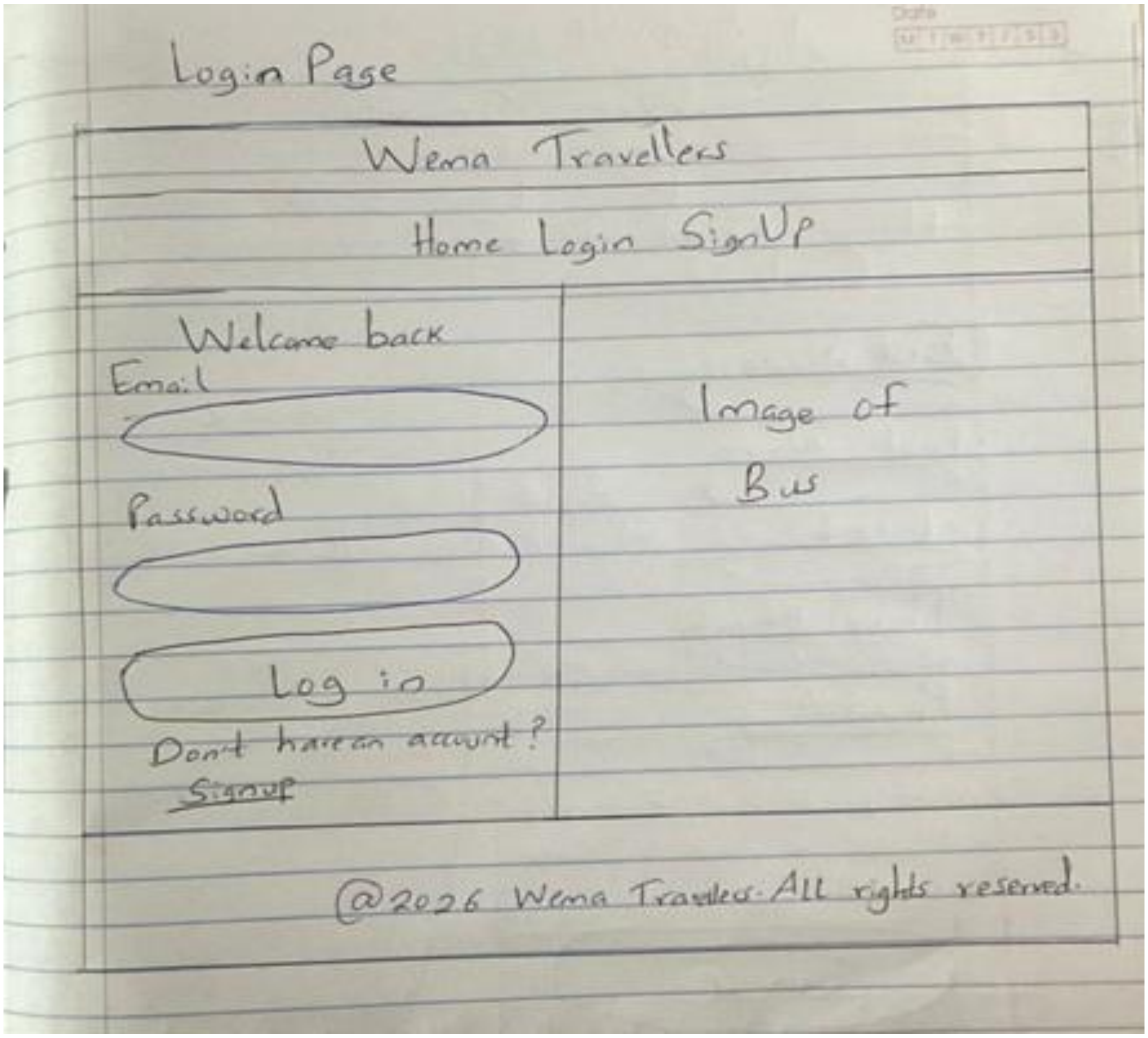
```
mysql> desc users;
+--------------+-----------------------------------+------+-----+---------------------+----------------+
| Field        | Type                              | Null | Key | Default             | Extra          |
+--------------+-----------------------------------+------+-----+---------------------+----------------+
| user_id      | int(11)                           | NO   | PRI | NULL                | auto_increment |
| first_name   | varchar(50)                       | NO   |     | NULL                |                |
| last_name    | varchar(50)                       | NO   |     | NULL                |                |
| email        | varchar(100)                      | NO   | UNI | NULL                |                |
| phone_number | varchar(15)                       | NO   | UNI | NULL                |                |
| password     | varchar(255)                      | NO   |     | NULL                |                |
| role         | enum('PASSENGER','ADMIN','AGENT') | YES  |     | PASSENGER           |                |
| created_at   | timestamp                         | NO   |     | current_timestamp() |                |
+--------------+-----------------------------------+------+-----+---------------------+----------------+
8 rows in set (0.06 sec)
```

B. UI Analysis

a) Index Page

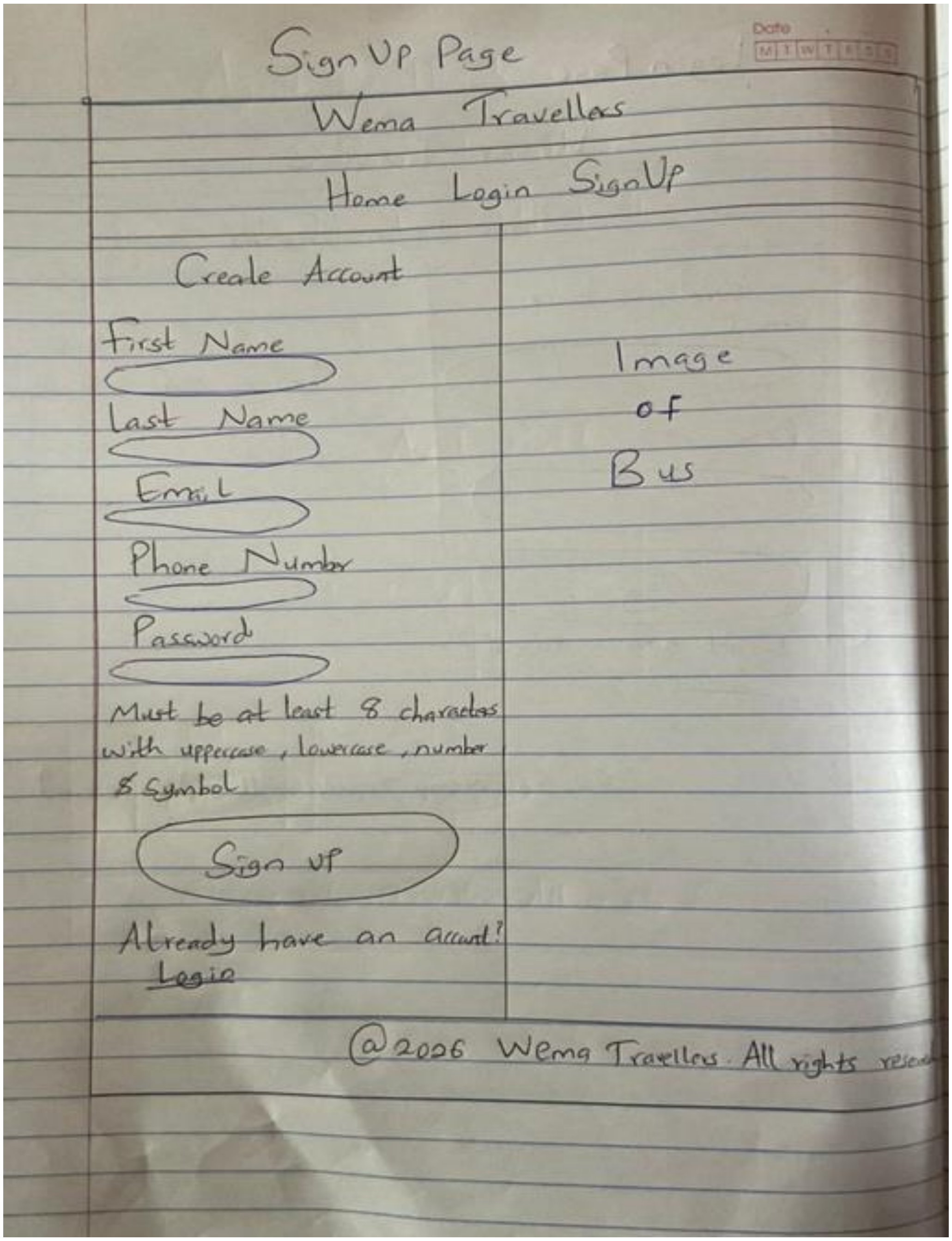


b) Login Page

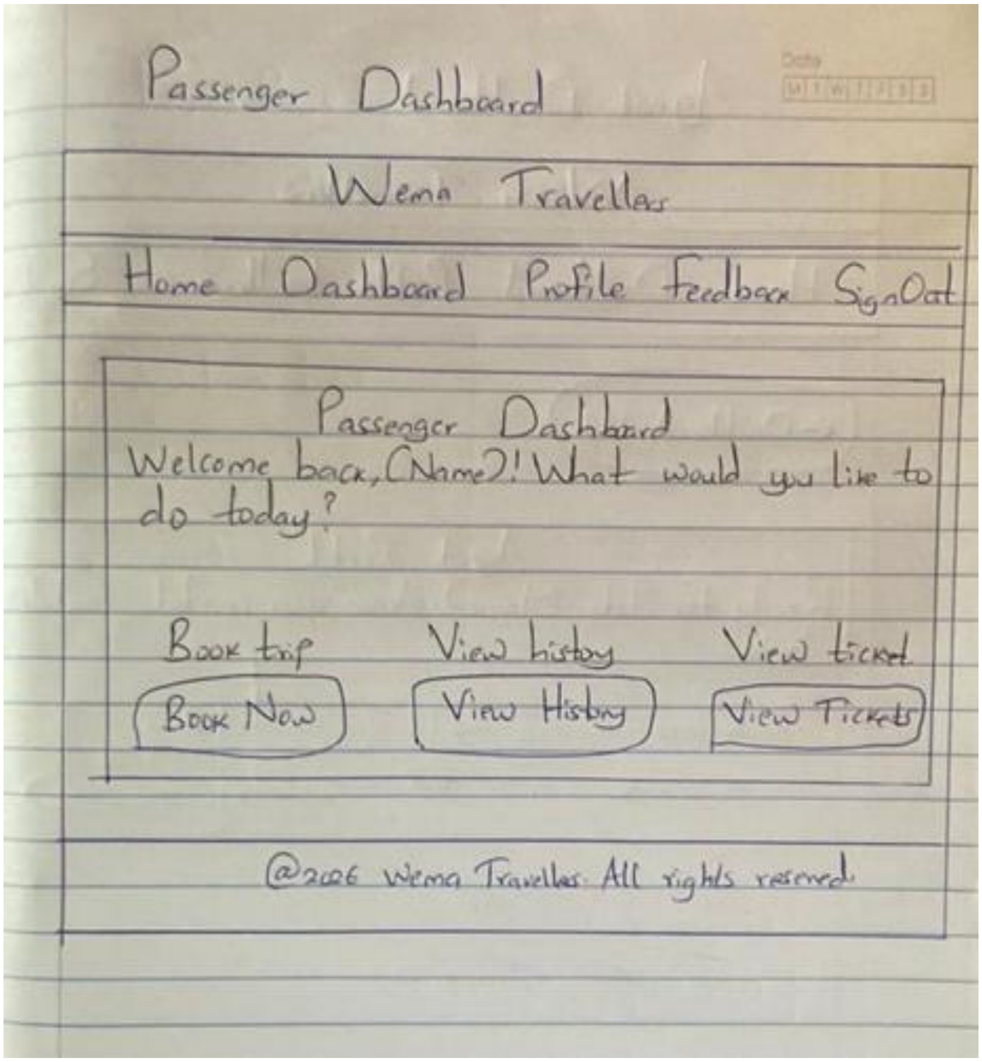


Login Page
Wema Travellers
Home Login SignUp
Welcome back
Email
Password
Log in
Don't have an account?
Signup
Image of
Bus
@2026 Wema Travelers. All rights reserved.

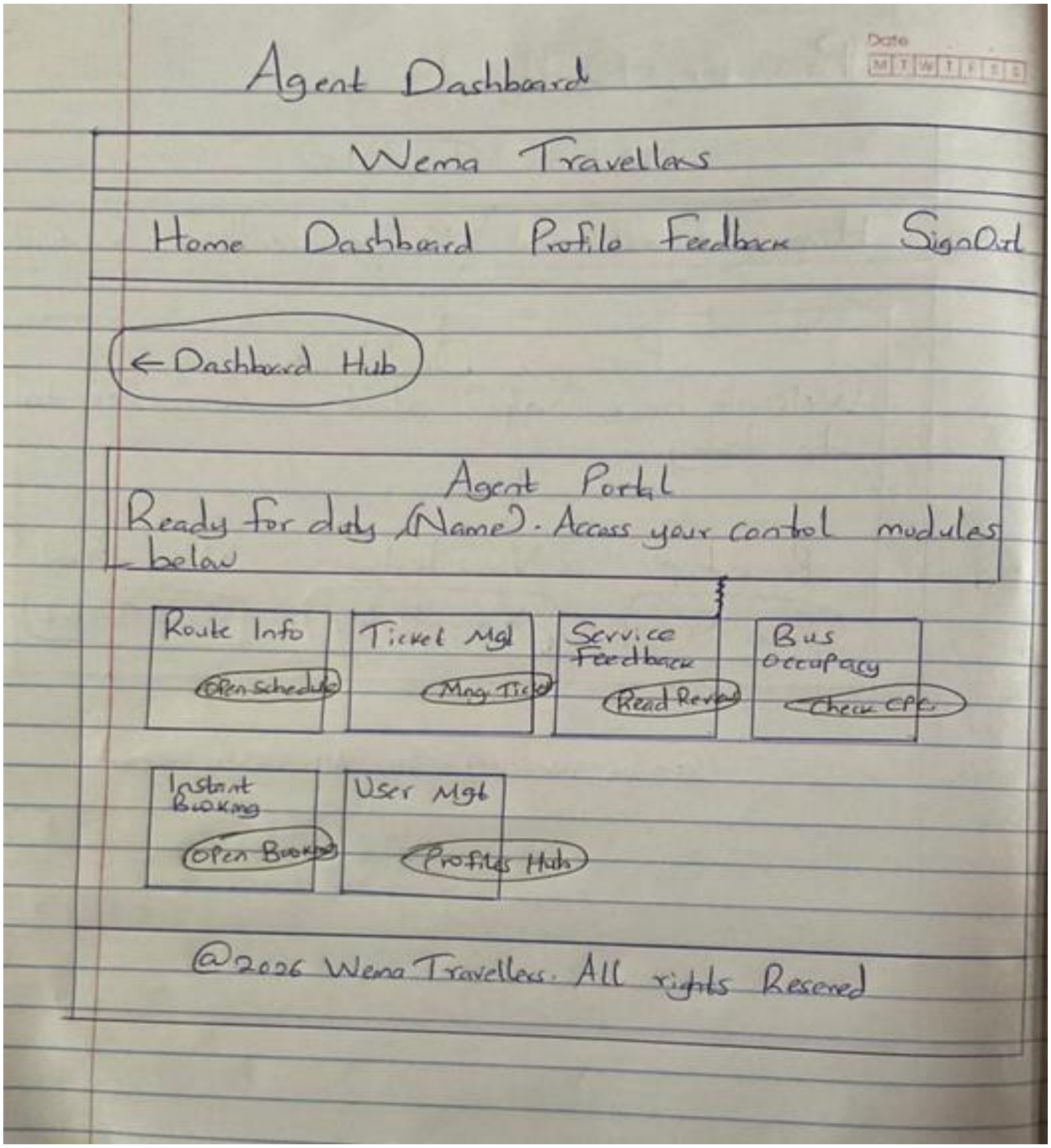
c) SignUp Page



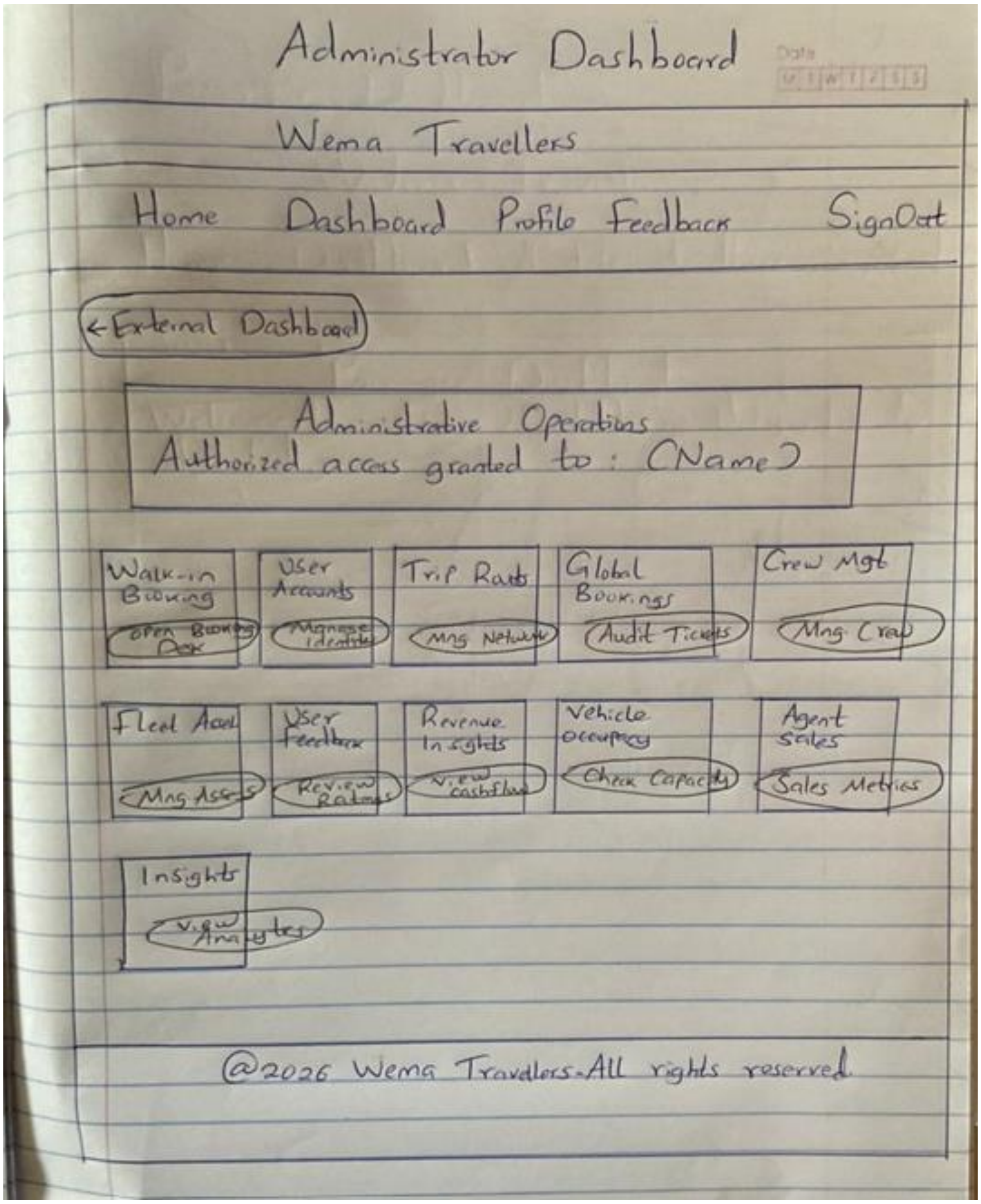
d) Passenger Dashboard



e) **Agent Dashboard**



f) Administrator Dashboard



IDENTIFIED USERS AND THEIR ROLES

1. Administrator (ADMIN)

The highest authority in the system. The Admin manages the business logic and fleet.

- **User Management**: Can create, edit, or delete accounts for Agents and Passengers.
- **Fleet Management**: Registers new buses and assigns/de-assigns drivers to them.
- **Schedule Management**: Responsible for creating and updating international travel routes and setting ticket prices.
- **Business Intelligence**: Accesses high-level reports, including **Revenue Reports**, **Agent Performance**, and **Bus Occupancy** trends.
- **System Oversight**: Can view and moderate all customer feedback.

2. Booking Agent (AGENT)

Staff members stationed at terminals or travel offices who facilitate the booking process.

- **Walk-in Bookings**: Authorized to book tickets for customers who visit the station physically.
- **Passenger Support**: Assists travellers with schedule information, seat availability, and booking cancellations.
- **Seat Monitoring**: Monitors real-time bus occupancy to prevent overbooking.
- **Identity Registration**: Can register new passengers into the system to facilitate their first booking.

3. Passenger (PASSENGER)

The primary customer or traveller using the portal.

- **Self-Service Booking**: Can search for routes, compare fares, and select their preferred seats.
- **Personal Dashboard**: Accesses their personal travel history and profile.
- **Digital Ticketing**: Retrieves and views their **Digital Travel Passes** (QR-code tokens) for boarding.
- **Feedback & Ratings**: Can rate their travel experience and leave comments about specific buses or routes.

System Actors (Managed but non-login)

While these individuals do not log into the system themselves, they are key "actors" tracked by the software:

- **Driver**: A registered professional assigned to specific buses. The system tracks their National ID, licensing status, and contact details to ensure every trip has a designated operator.

C. Functional Analysis

Each module in the system is designed with specific inputs and deterministic outputs.

1. Login Function

- **Input Data**: email, password.
- **Processing**: Validates email existence; verifies hashed password against DB; initializes session.
- **Successful Output**: Valid $_SESSION tokens (user_id, name, role) and redirection to the appropriate portal (Admin, Agent, or Home).

2. Signup Function

- **Input Data**: first_name, last_name, email, phone_number, password.
- **Processing**: Checks for email/phone uniqueness; hashes the password; inserts a new record with the 'PASSENGER' role.
- **Successful Output**: "Account Created" confirmation and redirection to the login portal.

3. Seat Booking Function

- **Input Data**: route_id, seat_number, passenger_name, passenger_age, passenger_id_number.
- **Processing**: Verifies seat availability for the specific route; links current user_id to the transaction; generates a unique qr_token.

- **Successful Output**: New entry in bookings table; "Payment Successful" simulation; entry in personal "Travel History".

4. Route Management (Admin)

- **Input Data**: departure_city, arrival_city, date, time, ticket_price, assigned_bus.
- **Processing**: Validates input formats; creates or updates the routes table entry.
- **Successful Output**: Updated schedule visible to all passengers and agents for booking.

5. Generate Insights (Admin)

- **Input Data**: Selection of time filter (Week, Month, Last Month, Year).
- **Processing**: Executes a SQL JOIN between bookings and users; filters based on booking_time interval; concatenates names for report display.
- **Successful Output**: A detailed, read-only table of all sales and passenger names for the selected period.

6. Driver/Bus Assignment

- **Input Data**: bus_id, driver_id.
- **Processing**: Updates the buses table with the new driver_id foreign key.
- **Successful Output**: Automatic reflection of assigned crew in the fleet report.

System Design

This documentation serves as the final technical blueprint for the **IBBS**. It bridges the gap between the initial conceptual analysis and the final implemented codebase, providing a detailed map of how data, visuals, and logic interact to create a cohesive transport management ecosystem.

---

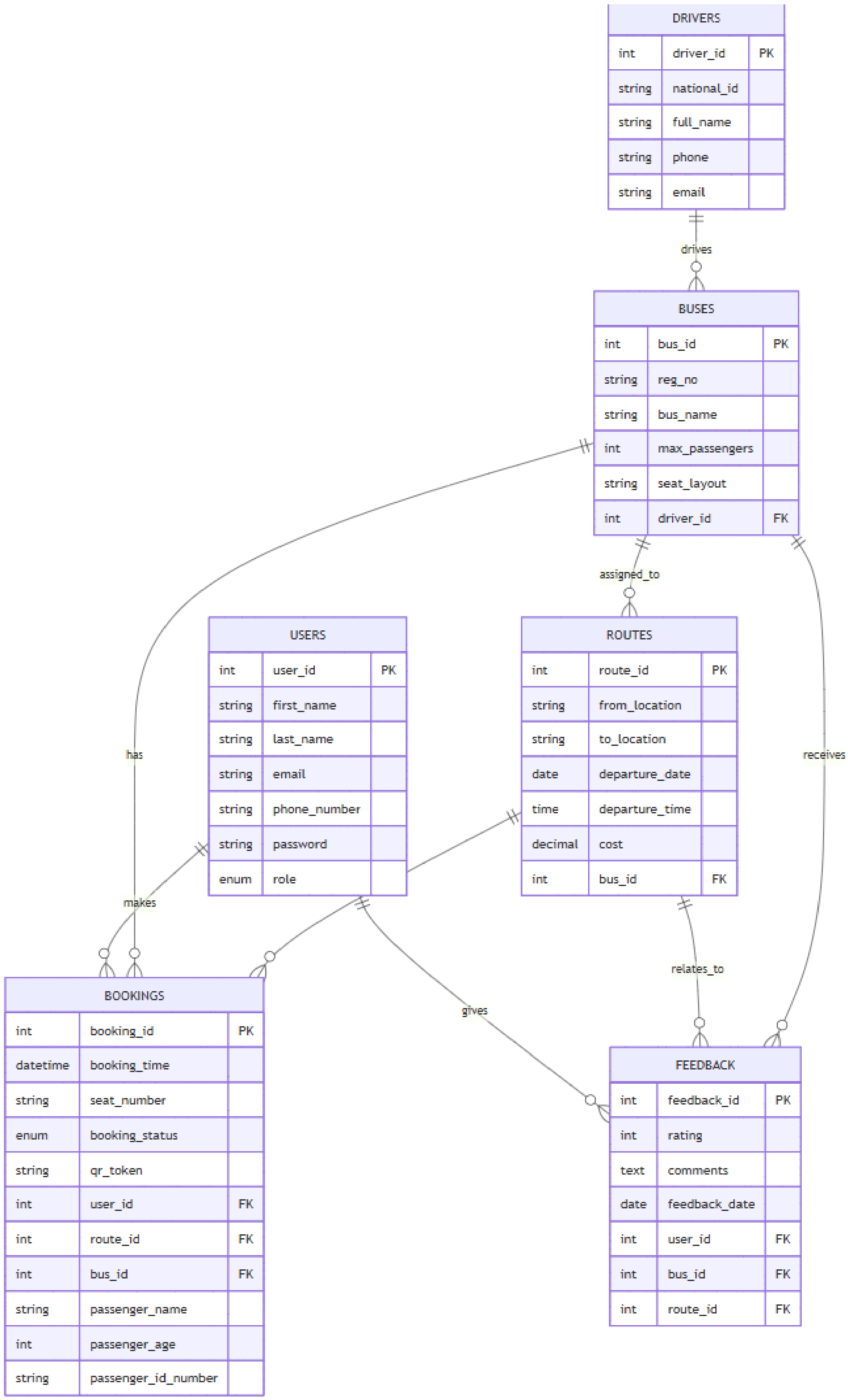
1. Data Design (The Structural Blueprint)

The data design focuses on the architecture of information—how it is captured, stored, and related within the MySQL relational engine.

A. **Entity Relationship Diagram (ERD) Logic**

The system is built on an interconnected relational model where the Bookings entity acts as the central hub.

- **Users to Bookings (1:N)**: One user can make multiple bookings over time, but each booking belongs to only one user account for auditability.
- **Routes to Bookings (1:N)**: A specific route (e.g., Nairobi to Kampala on 4th April) can contain many passenger bookings.
- **Buses to Drivers (1:1)**: To ensure operational accountability, each bus is assigned to exactly one driver at a time through the fleet management registry.
- **Buses to Routes (1:N)**: A physical bus is assigned to a specific scheduled route to fulfill the travel capacity.


DRIVERS
int driver_id PK
string national_id
string full_name
string phone
string email
drives
BUSES
int bus_id PK
string reg_no
string bus_name
int max_passengers
string seat_layout
int driver_id FK
assigned_to
has
receives
USERS
int user_id PK
string first_name
string last_name
string email
string phone_number
string password
enum role
ROUTES
int route_id PK
string from_location
string to_location
date departure_date
time departure_time
decimal cost
int bus_id FK
makes
relates_to
gives
BOOKINGS
int booking_id PK
datetime booking_time
string seat_number
enum booking_status
string qr_token
int user_id FK
int route_id FK
int bus_id FK
string passenger_name
int passenger_age
string passenger_id_number
FEEDBACK
int feedback_id PK
int rating
text comments
date feedback_date
int user_id FK
int bus_id FK
int route_id FK

B. Relational Schema

The database is normalized to the **3rd Normal Form (3NF)** to ensure zero data redundancy.

Table Name	Primary Key	Foreign Keys	Key Attributes
users	user_id	N/A	Email (Unique), Phone (Unique), Role (Enum)
drivers	driver_id	N/A	National ID, Full Name, Contact Info
buses	bus_id	driver_id	Registration No, Seat Layout, Max Capacity
routes	route_id	bus_id	Departure Date, Departure Time, Ticket Cost
bookings	booking_id	user_id, route_id, bus_id	Seat Number, QR Token, Passenger Metadata
feedback	feedback_id	user_id, bus_id, route_id	Rating (1-5), Comments, Timestamp

Relational Schema for IBBS

Below is the relational schema for the International Bus Booking System (IBBS) prototype database.

1. Users

Stores all user information including Passengers, Admins, and Agents.

- user_id (PK, INT, Auto Increment)
- first_name (VARCHAR, Not Null)
- last_name (VARCHAR, Not Null)
- email (VARCHAR, Unique, Not Null)
- phone_number (VARCHAR, Unique, Not Null)
- password (VARCHAR, Not Null)
- role (ENUM: 'PASSENGER', 'ADMIN', 'AGENT', Default: 'PASSENGER')

2. Drivers

Stores information about bus drivers.

- driver_id (PK, INT, Auto Increment)
- national_id (VARCHAR, Unique, Not Null)
- full_name (VARCHAR, Not Null)
- phone (VARCHAR, Unique, Not Null)
- email (VARCHAR, Unique, Not Null)

3. Buses

Stores information about the buses in the fleet.

- bus_id (PK, INT, Auto Increment)
- reg_no (VARCHAR, Unique, Not Null)
- bus_name (VARCHAR)
- max_passengers (INT, Not Null)
- seat_layout (VARCHAR, Not Null)
- driver_id (FK -> Drivers.driver_id, Nullable)

4. Routes

Stores available international travel routes.

- route_id (PK, INT, Auto Increment)
- from_location (VARCHAR, Not Null)
- to_location (VARCHAR, Not Null)
- departure_date (DATE, Not Null)
- departure_time (TIME, Not Null)
- cost (DECIMAL, Not Null)
- bus_id (FK -> Buses.bus_id, Not Null)

5. Bookings

Stores ticket bookings made by users.

- booking_id (PK, INT, Auto Increment)

- booking_time (DATETIME, Not Null)
- seat_number (VARCHAR, Not Null)
- booking_status (ENUM: 'PAID', 'CHECKED_IN', 'CANCELLED', Default: 'PAID')
- qr_token (VARCHAR, Unique, Not Null)
- user_id (FK -> Users.user_id, Not Null)
- route_id (FK -> Routes.route_id, Not Null)
- bus_id (FK -> Buses.bus_id, Not Null)

6. Feedback

Stores user feedback and ratings.

- feedback_id (PK, INT, Auto Increment)
- rating (INT, 1-5)
- comments (TEXT)
- feedback_date (DATE, Not Null)
- user_id (FK -> Users.user_id, Not Null)
- bus_id (FK -> Buses.bus_id, Not Null)
- route_id (FK -> Routes.route_id, Not Null)

Relationships

- **Users** have multiple **Bookings** (1:N).
- **Users** can leave multiple **Feedback** (1:N).
- **Drivers** are assigned to **Buses** (1:1 or 1:N, schema allows 1 driver per bus at a time).
- **Buses** have multiple **Routes** (1:N).
- **Routes** have multiple **Bookings** (1:N).
- **Routes** have multiple **Feedback** (1:N).

C. Data Dictionary (Sample Modules)

- **Role (users table)**: Defines access levels (ADMIN, AGENT, PASSENGER).
- **QR Token (bookings table)**: A unique MD5/SHA-1 string used as a digital secure key for ticket verification.
- **Seat Layout (buses table)**: A string descriptor (e.g., 2x2) that determines the visual grid shown during the booking process.

---

2. User Interface (UI) Design

The UI design implements a **High-Impact, Premium Aesthetic** focused on readability and professional "trust signals."

A. Visual Identity (The Design System)

- **Primary Palette**: Matte Purple (#9a4d9a) and Vibrant Pink (#d06dd0) create a modern, energetic brand identity.
- **Accent Colors**: Use of Action Green for "Success" and Deep Red for "Danger/Delete" ensures clear haptic feedback.
- **Typography**: The system utilizes a dual-font strategy:
    - **Branding Header**: *Comic Sans MS* for a friendly, approachable entrance.
    - **Dashboard Content**: *Segoe UI* or *System-UI* for high-legibility data reading in tables and reports.

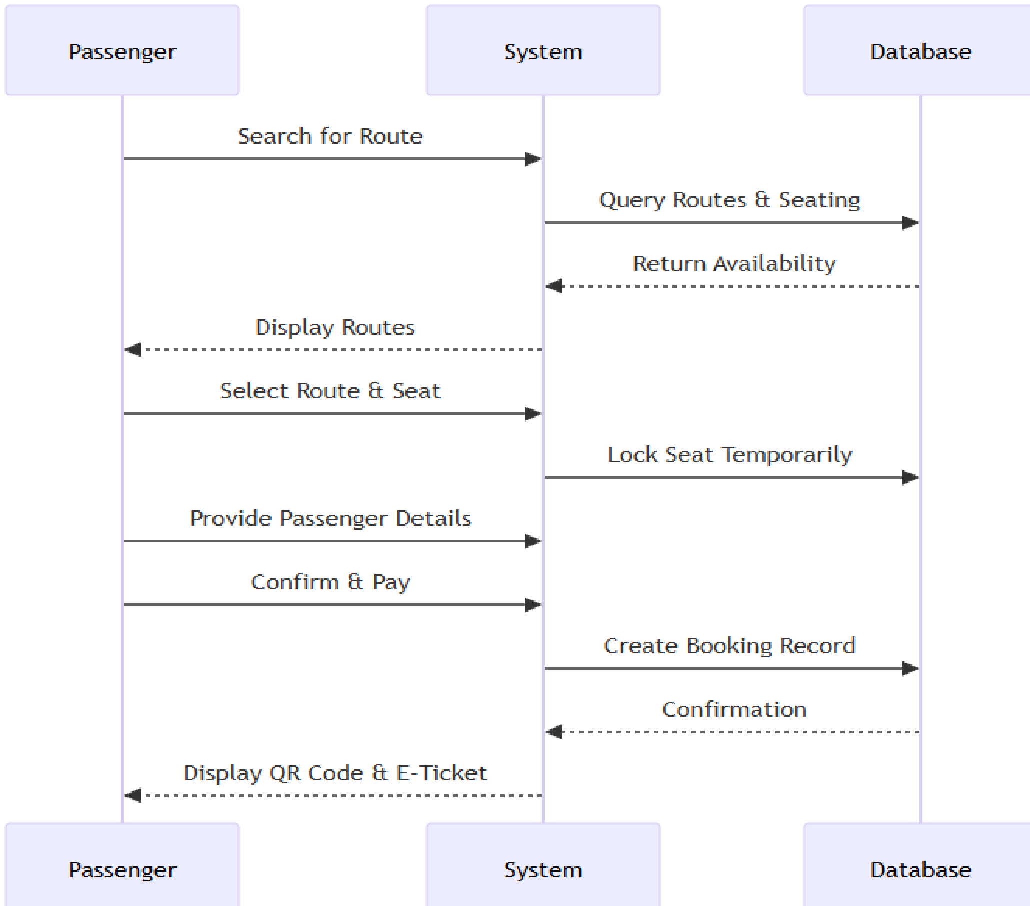
B. Component-Based Architecture

The system does not use ad-hoc styles but rather a consistent library of components:

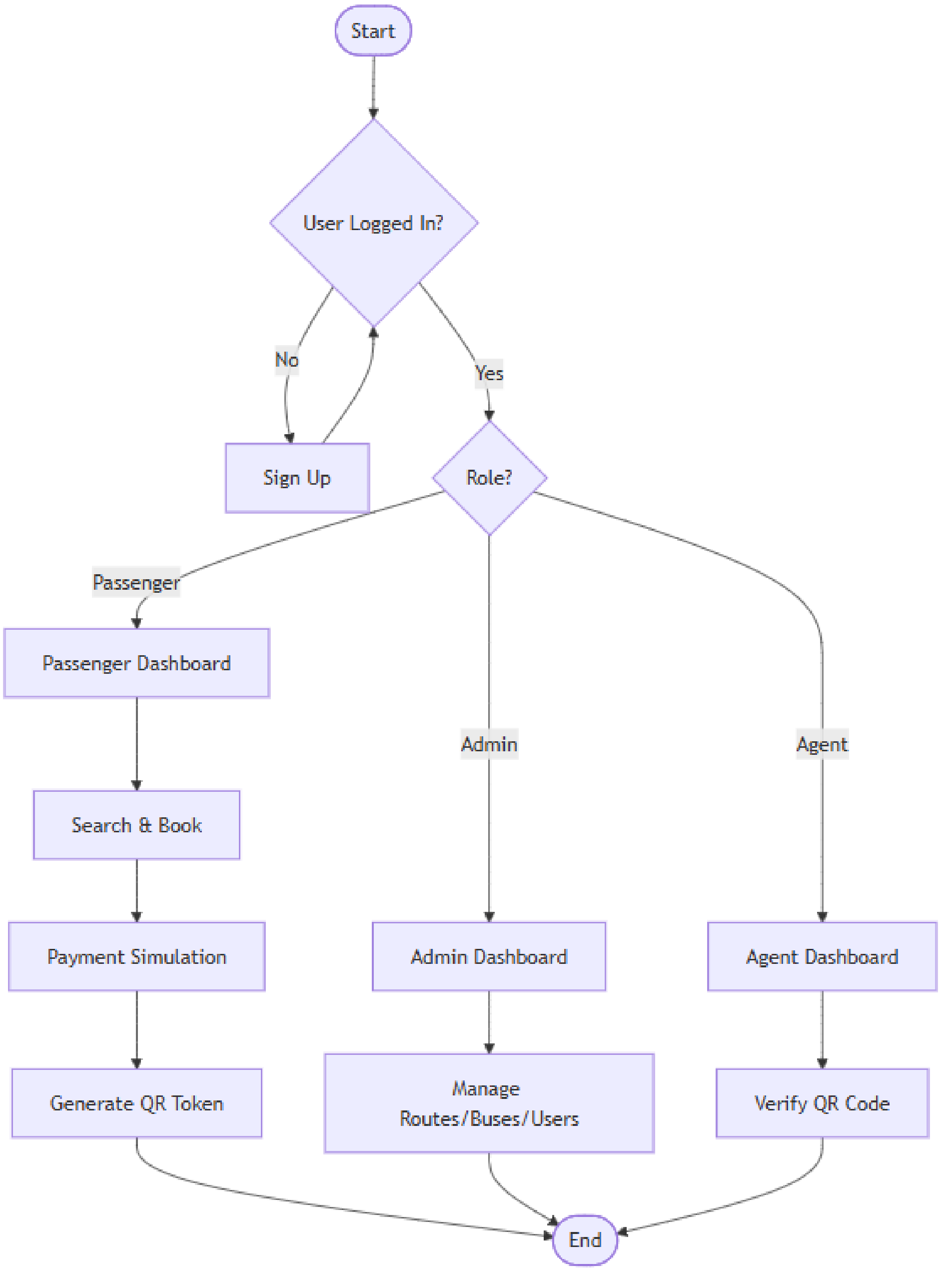
- **The "Pill" Button**: All primary actions use border-radius: 50px with a hard retro-shadow for a tactile, "clickable" look.
- **The Welcome Banner**: A full-width header block using var(--purple) with rounded corners (16px) to welcome users into their session.
- **Glassmorphism Effects**: Cards utilize subtle white semi-transparency (rgba(255, 255, 255, 0.5)) and purple borders to separate different functional modules.

D. Navigation Flow (Sitemap)

Process Flow (Booking)



System Flowchart



1. **Entry**:

index.html (Landing) →

login.html (Authentication).

2. **Redirection**: Based on role, users land in

dashboard.php (Passenger),

agent_dashboard.php, or

admin_dashboard.php.

3. **Process Flow**: Dashboard → Route Selection (

book.php) → Process Booking (

process_booking.php) → History View (

booking_history.php).

---

3. Functional Design

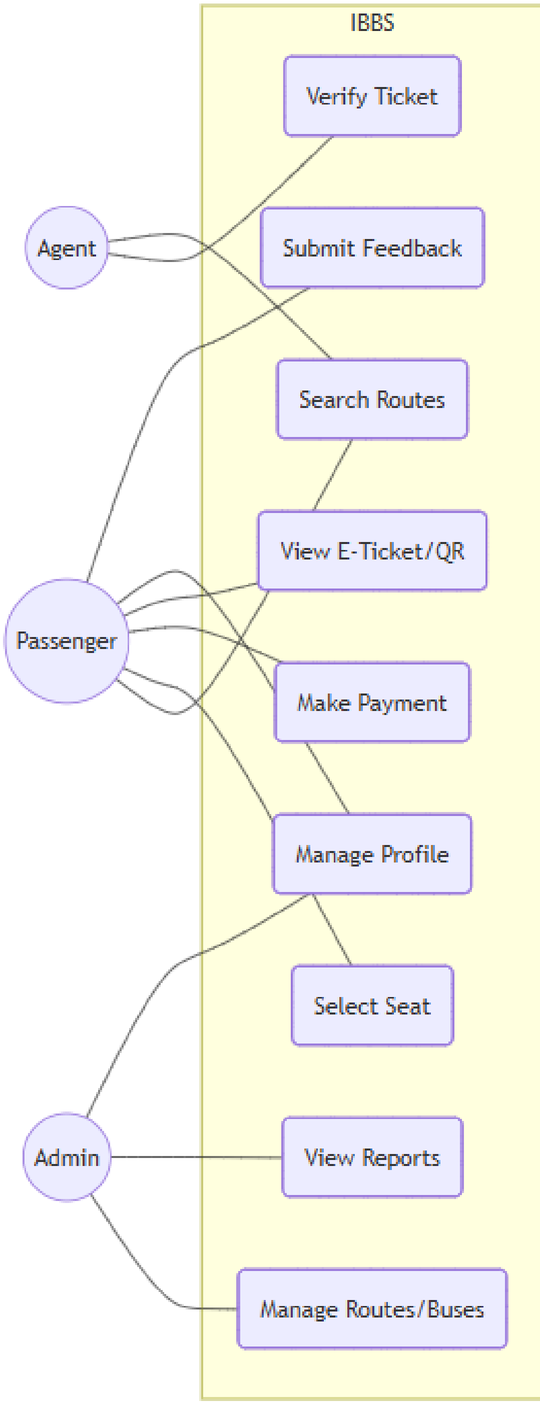
Functional design defines the actual "intelligence" of the system—how inputs are transformed into operational results.

A. Use-Case Scenarios

- **Use Case: Administrator Reporting**: The Admin accesses the **Insights Hub**, selects a timeframe, and the system executes a complex JOIN query to fetch real names from the users table and booking details from bookings.
- **Use Case: Intelligent Seat Selection**: When a user picks a route, the system dynamically queries the bookings table to find "Occupied" seats and disables them in the UI to prevent double-booking.

- **Use Case: Agent Walk-in Booking**: An agent registers a ticket for a third party. The system captures the agent's ID as the user_id but records the customer's name in passenger_metadata for ticket validity.

Use Case Diagram



B. **Process Flowcharts (The Logic Engine)**

The Booking Verification Logic (Functional Flow)

Start[User selects Route] --> QueryDB[Check assigned Bus & Capacity]

QueryDB --> DisplayMap[Show Seat Map]

DisplayMap --> SelectSeat[User selects Seat]

SelectSeat --> Validate[Verify if Seat is still free]

Validate -- No --> Fail[Error: Seat already taken]

Validate -- Yes --> Commit[Save Booking & Generate Hash]

Commit --> Finish[Show Success & Receipt]

C. Input-Process-Output (IPO) Analysis

Module	Inputs (What it needs)	Process (What it does)	Outputs (What it gives)
Authentication	Email + Password	Password hashing & DB comparison	Session IDs + Site Access
Booking Engine	Route ID + Seat No + Metadata	Database INSERT & QR Hash Gen	Digital Ticket + Reserved Seat
Insights Hub	Time Interval (e.g., Week)	SQL JOIN & Date Range Filtering	Detailed Financial/Sales Report
Fleet Mgmt	Bus Reg + Driver Selection	UPDATE foreign key links in DB	Assigned Crew & Active Fleet

---

4. Security & Performance Design

- **SQL Injection Prevention**: Implementation of **Prepared Statements** ($stmt->prepare) for all user-submitted data.
- **Identity Security**: Passwords are never stored as plain text but use one-way cryptographic hashing.
- **Access Control**: Every PHP page starts with a role-verification header that bounces unauthorized users to the login screen before any data is rendered.

Objectives

Main Objective

To build a digital bus booking and verification system that replaces manual ticketing with a secure digital token framework.

Specific Objectives

1. **Online Travel Portal**: To develop a web-based platform where passengers can search for international routes, compare costs, and select seats in real-time.
2. **Digital Token Generation**: To design a system that generates unique digital identifiers (tokens) for every booking, ensuring each ticket is one-of-a-kind and tamper-proof.
3. **Simulated Verification Framework**: To implement the database logic required for automated ticket validation, allowing staff to verify boarding passes instantly without manual logs.
4. **Comprehensive Manifest Management**: To create a database that captures detailed passenger information (Name, Age, ID) to satisfy cross-border travel requirements.
5. **Admin Control Center**: To provide bus operators with a centralized dashboard for managing fleets, travel schedules, and real-time occupancy.

Room for Improvement (Future Phase / Out of Scope)

6. **Physical QR Scanner API**: Integrating specialized mobile camera APIs or hardware scanners to physically "read" the generated tokens at the bus door.
7. **Live Notification Gateways**: Integrating SMS and Email APIs to send automated travel alerts and reminders (currently simulated within the UI).
8. **Automated Payment Settlement**: Integrating the M-Pesa Daraja API or Card Gateways for real-time financial reconciliation.

For an improvement I will make a page called verify_ticket.php where a staff member just pastes the token, and it says **"VALID: Seat 5A, John Doe"** or **"INVALID"** to validate the tokens.

IMPLIMENTING THE SYSTEM

DATABASE CREATION SCRIPTS

A. IMPLIMENTING DATA DESIGN

```
DROP DATABASE IF EXISTS IBBS_PROTOTYPE;
CREATE DATABASE IBBS_PROTOTYPE;
USE IBBS_PROTOTYPE;

1. USERS TABLE
CREATE TABLE users (
user_id INT(11) NOT NULL AUTO_INCREMENT,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
email VARCHAR(100) NOT NULL UNIQUE,
phone_number VARCHAR(15) NOT NULL UNIQUE,
password VARCHAR(255) NOT NULL,
role ENUM('PASSENGER','ADMIN','AGENT') DEFAULT 'PASSENGER',
created_at TIMESTAMP NOT NULL DEFAULT current_timestamp(),
PRIMARY KEY (user_id)
);
2. DRIVERS TABLE
CREATE TABLE drivers (
driver_id INT(11) NOT NULL AUTO_INCREMENT,
national_id VARCHAR(20) NOT NULL UNIQUE,
full_name VARCHAR(100) NOT NULL,
```

phone VARCHAR(15) NOT NULL UNIQUE,

email VARCHAR(100) NOT NULL UNIQUE,

PRIMARY KEY (driver_id)

);

3. BUSES TABLE

CREATE TABLE buses (

bus_id INT(11) NOT NULL AUTO_INCREMENT,

reg_no VARCHAR(20) NOT NULL UNIQUE,

bus_name VARCHAR(50) DEFAULT NULL,

max_passengers INT(11) NOT NULL,

seat_layout VARCHAR(20) NOT NULL,

driver_id INT(11) DEFAULT NULL,

PRIMARY KEY (bus_id),

FOREIGN KEY (driver_id) REFERENCES drivers (driver_id) ON DELETE SET NULL

);

4. ROUTES TABLE

CREATE TABLE routes (

route_id INT(11) NOT NULL AUTO_INCREMENT,

from_location VARCHAR(100) NOT NULL,

to_location VARCHAR(100) NOT NULL,

departure_date DATE NOT NULL,

departure_time TIME NOT NULL,

cost DECIMAL(10,2) NOT NULL,

bus_id INT(11) NOT NULL,

status ENUM('SCHEDULED','COMPLETED','CANCELLED') DEFAULT 'SCHEDULED',

PRIMARY KEY (route_id),

FOREIGN KEY (bus_id) REFERENCES buses (bus_id)

);

5. BOOKINGS TABLE

CREATE TABLE bookings (

booking_id INT AUTO_INCREMENT PRIMARY KEY,

booking_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

seat_number VARCHAR(10) NOT NULL,

booking_status ENUM('CONFIRMED','CANCELLED','PAID','CHECKED_IN') DEFAULT 'PAID',

qr_token VARCHAR(255) UNIQUE,

user_id INT NOT NULL,

route_id INT NOT NULL,

bus_id INT NOT NULL,

passenger_name VARCHAR(255) NOT NULL DEFAULT '',

passenger_age INT NOT NULL DEFAULT 0,

passenger_id_number VARCHAR(50) NOT NULL DEFAULT '',

passenger_dob DATE DEFAULT NULL,

FOREIGN KEY (user_id) REFERENCES users(user_id),

FOREIGN KEY (route_id) REFERENCES routes(route_id) ON DELETE CASCADE,

FOREIGN KEY (bus_id) REFERENCES buses(bus_id),

UNIQUE KEY unique_booking (route_id, seat_number)

);

6. FEEDBACK TABLE

CREATE TABLE feedback (

feedback_id INT(11) NOT NULL AUTO_INCREMENT,

rating INT(11) DEFAULT 5,

comments TEXT NOT NULL,

feedback_date DATE NOT NULL,

user_id INT(11) NOT NULL,

bus_id INT(11) NOT NULL,

route_id INT(11) NOT NULL,

submitted_at TIMESTAMP NOT NULL DEFAULT current_timestamp(),

PRIMARY KEY (feedback_id),

FOREIGN KEY (user_id) REFERENCES users (user_id) ON DELETE CASCADE,

FOREIGN KEY (bus_id) REFERENCES buses (bus_id),

FOREIGN KEY (route_id) REFERENCES routes (route_id) ON DELETE CASCADE

);

DATA INSERTION SCRIPTS

Inserting ADMIN Users

INSERT INTO users (first_name, last_name, email, phone_number, password, role) VALUES

('Alice', 'Kamau', 'alice1@gmail.com', '0700000000', '$2y$10$6AP9mQLbsAlcc2yDxdX3.Oed376Faioz4iG2NG7Z5ZTsEOQU8huQe', 'ADMIN'),

('Bob', 'Ochieng', 'bob2@gmail.com', '0700000001', '$2y$10$eOws.VlRQIGeg7axR6s9pOcAEHNSQ7ogTbPqG4uGzsnXQJ1jf2GUK', 'ADMIN'),

('Charlie', 'Kipkorir', 'charlie3@gmail.com', '0700000002', '$2y$10$nnpgJAAK5jr6CwCNQbFJTuRXCZZY017LneYwnjbpX2IhOa1aHKZqW', 'ADMIN'),

('David', 'Maina', 'david4@gmail.com', '0700000003', '$2y$10$KPSYO2TIhGX2uVCcycR08uyYkqVoQMnq7t1xWLGIP0j8d5Hhz7y32', 'ADMIN'),

('Eve', 'Wanjiku', 'eve5@gmail.com', '0700000004', '$2y$10$07KWtn7qvTnlWR4AWJprzujaiG4W0QuNot90mOJaD61jwkIHcBOVS', 'ADMIN'),

('Frank', 'Otieno', 'frank6@gmail.com', '0700000005', '$2y$10$Dw4bGCBQoh6he42VVOKyqegyTzNd81KDtP8J0hg3r79pAOfZWll/e', 'ADMIN'),

('Grace', 'Achieng', 'grace7@gmail.com', '0700000006', '$2y$10$Kf8pw34A0CbitzLb2W1uhOrJuy3Jpz8YaBkHZnFrCGUEWvToaukuC', 'ADMIN'),

('Hank', 'Musyoka', 'hank8@gmail.com', '0700000007', '$2y$10$7AfpBfCzt0V0ID.iVXutLOdQjeT6YfQQTz7toR776amoEqHSKpApK', 'ADMIN'), ('Ivy', 'Njeri', 'ivy9@gmail.com', '0700000008', '$2y$10$l7XO37p6M77kIXQmFT/7au5acAaWJyqXPueFCs6QEDK2rT7iAE10u', 'ADMIN'),

('Jack', 'Mutua', 'jack10@gmail.com', '0700000009', '$2y$10$faynGAGhm8/w.YVgf42B1eSL7Kta9AdKvGCSz93ZfhT/Oge6Wu2Z2', 'ADMIN');

Inserting AGENT Users

INSERT INTO users (first_name, last_name, email, phone_number, password, role) VALUES

('Karen', 'Njoroge', 'karen1@gmail.com', '0711000000', '$2y$10$vorDX2PZQq6CNyKhnEd6muNccbOp0/kAsmf/VoOvsdL9VixyFckha', 'AGENT'),

('Leo', 'Mwangi', 'leo2@gmail.com', '0711000001', '$2y$10$n63JIZe9OeTTLHSgl9HrEuSQsam528/xT2XNpUo/jhNka2HVBWrYK', 'AGENT'),

('Mia', 'Odhiambo', 'mia3@gmail.com', '0711000002', '$2y$10$e2UZmFWu0tvbMmaWdt/vtObHNqAnVEaY3JuZL2q6rKn9gDYQWXcPW', 'AGENT'),

('Noah', 'Kimani', 'noah4@gmail.com', '0711000003', '$2y$10$OtaT8IBKXLzF7TwwxVjmpe8UyX7iXI2a991g2Bnf7BUbt4tjApzMu', 'AGENT'),

('Olivia', 'Chebet', 'olivia5@gmail.com', '0711000004', '$2y$10$Vajn3ZpLHhurmfDz/CsvluQznpN7kTYjjk.Dxl9ktS4VJt9SicYmm', 'AGENT'),

('Paul', 'Kariuki', 'paul6@gmail.com', '0711000005', '$2y$10$QTQGUhYKwJv9krYSEI/sku0vRIsACDQ2eE3FD3Ac07/b8tYf2C9m6', 'AGENT'),

('Quinn', 'Awere', 'quinn7@gmail.com', '0711000006', '$2y$10$LntrFhaxjyTKyGEShGKFsuY/j7E5McloZPjzKoDRDozZjMmnaRjqS', 'AGENT'),

('Ryan', 'Omondi', 'ryan8@gmail.com', '0711000007', '$2y$10$/Y6W3rm6Ug1L5sWVHCeQn.QwOFX0brsk7jyUpxNXLus.aayOwA1SO', 'AGENT'),

('Sarah', 'Wambui', 'sarah9@gmail.com', '0711000008', '$2y$10$ZTnq0FaW5YapQmPJCahjredJ3fs.VlquxaIxWfSQhS7u82DBdGBOK', 'AGENT'),

('Tom', 'Ndegwa', 'tom10@gmail.com', '0711000009', '$2y$10$0L.4AyomYevMyybNyBoGue4ZEUqn8I1S..h8MV7jyyx1Mi5sW3S5G', 'AGENT');

Inserting PASSENGER Users

INSERT INTO users (first_name, last_name, email, phone_number, password, role) VALUES

('Uma', 'Abdi', 'uma1@gmail.com', '0722000000', '$2y$10$1YvBeMq4Ot4OIzluMOkL8eSPlDm/vf1RvJDhD/S94gnKL7hVgKHx2', 'PASSENGER'),

('Vin', 'Ndlovu', 'vin2@gmail.com', '0722000001',
'$2y$10$NHV5qfiqIxq8CX08oMIAzu1HUU1aAzAg/VeCszLban9to86DGxr7O',
'PASSENGER'),

('Will', 'Chamele', 'will3@gmail.com', '0722000002',
'$2y$10$isNOgS0qXW3/jaKOHK/5aeMDajNAnJPxU5NhcAzUdRPr5NzdSReiu',
'PASSENGER'),

('Xena', 'Tesfaye', 'xena4@gmail.com', '0722000003',
'$2y$10$KY1KRlHaK7fU.fPjWQzrGehf4EtXzqD6cZKOU55Ysbwwv9ojHR3Fi',
'PASSENGER'),

('Yara', 'Mensah', 'yara5@gmail.com', '0722000004',
'$2y$10$/cYOmtVIWUga.v0oLa6kEukgJT922bVzTMjCmkLuuBLK6WlPUC6Fi',
'PASSENGER'),

('Zac', 'Diallo', 'zac6@gmail.com', '0722000005',
'$2y$10$uI27jAHOQfMptYojSuWiQ.U/ZrbkbPRZsk8wG0IQr8CXWljPEWGv2',
'PASSENGER'),

('Adam', 'Keita', 'adam7@gmail.com', '0722000006',
'$2y$10$cKzfWlu7dfxQaVZ9h4YdOumimgSVLp8isYQzHvspOMom9.RUuUAhG',
'PASSENGER'),

('Bella', 'Sow', 'bella8@gmail.com', '0722000007',
'$2y$10$CE03fm6meXEWMKeL48fXsunPESkwcP.krgkbZSUDv.v2.YMioHu6q',
'PASSENGER'),

('Chris', 'Traore', 'chris9@gmail.com', '0722000008',
'$2y$10$J/xI66qvwJoFOei8mACNSueIhH9xVv6va7E.4tgaolFxQLyx86xvW', 'PASSENGER'),

('Drake', 'Kone', 'drake10@gmail.com', '0722000009',
'$2y$10$FjK8MDssTU5GMnetQ9FAIe1NIK/yi0JNY7bIbqAoxi2i9MouNqxfS',
'PASSENGER');

INSERT DRIVERS

INSERT INTO drivers (national_id, full_name, phone, email) VALUES

('ID00000001', 'John Mwangi', '0733000000', 'john.mwangi@gmail.com'),

('ID00000002', 'Samuel Okello', '0733000001', 'samuel.okello@gmail.com'),

('ID00000003', 'David Mengistu', '0733000002', 'david.mengistu@gmail.com'),

('ID00000004', 'Mohammed Hassan', '0733000003', 'mohammed.hassan@gmail.com'),

('ID00000005', 'Peter Kamau', '0733000004', 'peter.kamau@gmail.com'), ('ID00000006', 'James Omondi', '0733000005', 'james.omondi@gmail.com'),

('ID00000007', 'Benson Kiprop', '0733000006', 'benson.kiprop@gmail.com'),

('ID00000008', 'Charles Odhiambo', '0733000007', 'charles.odhiambo@gmail.com'),

('ID00000009', 'Joseph Kariuki', '0733000008', 'joseph.kariuki@gmail.com'),

('ID00000010', 'Thomas Njoroge', '0733000009', 'thomas.njoroge@gmail.com'),

('ID00000011', 'Emmanuel Abebe', '0733000010', 'emmanuel.abebe@gmail.com'),

('ID00000012', 'Isaac Tesfaye', '0733000011', 'isaac.tesfaye@gmail.com'),

('ID00000013', 'Gabriel Selassie', '0733000012', 'gabriel.selassie@gmail.com'),

('ID00000014', 'Michael Afewerki', '0733000013', 'michael.afewerki@gmail.com'),

('ID00000015', 'Daniel Tekle', '0733000014', 'daniel.tekle@gmail.com'),

('ID00000016', 'Richard Mwanza', '0733000015', 'richard.mwanza@gmail.com'),

('ID00000017', 'Patrick Mutua', '0733000016', 'patrick.mutua@gmail.com'),

('ID00000018', 'Stephen Musyoka', '0733000017', 'stephen.musyoka@gmail.com'),

('ID00000019', 'George Otieno', '0733000018', 'george.otieno@gmail.com'),

('ID00000020', 'Edward Wanyama', '0733000019', 'edward.wanyama@gmail.com'),

('ID00000021', 'Brian Kibet', '0733000020', 'brian.kibet@gmail.com'),

('ID00000022', 'Kevin Cheruiyot', '0733000021', 'kevin.cheruiyot@gmail.com'),

('ID00000023', 'Dennis Kemboi', '0733000022', 'dennis.kemboi@gmail.com'),

('ID00000024', 'Alex Rotich', '0733000023', 'alex.rotich@gmail.com'),

('ID00000025', 'Felix Langat', '0733000024', 'felix.langat@gmail.com'),

('ID00000026', 'Victor Ochieng', '0733000025', 'victor.ochieng@gmail.com'),

('ID00000027', 'Collins Okeyo', '0733000026', 'collins.okeyo@gmail.com'),

('ID00000028', 'Fredrick Owino', '0733000027', 'fredrick.owino@gmail.com'),

('ID00000029', 'Walter Aketch', '0733000028', 'walter.aketch@gmail.com'),

('ID00000030', 'Moses Odongo', '0733000029', 'moses.odongo@gmail.com');

INSERT BUSES

INSERT INTO buses (reg_no, bus_name, max_passengers, seat_layout, driver_id) VALUES

('KWI 435Z', 'Wema Executive 1', 40, '2x2', 1),

('KIP 893K', 'Wema Executive 2', 40, '2x2', 2),

('KHN 719W', 'Wema Executive 3', 40, '2x2', 3),

('KOY 346P', 'Wema Executive 4', 40, '2x2', 4),

('UPP 520A', 'Wema Executive 5', 40, '2x2', 5),

('UUH 503W', 'Wema Executive 6', 40, '2x2', 6),

('T 435 VWM', 'Wema Executive 7', 40, '2x2', 7),

('T 155 UII', 'Wema Executive 8', 40, '2x2', 8),

('RLR 718 H', 'Wema Executive 9', 40, '2x2', 9),

('SIJ 854 GP', 'Wema Executive 10', 40, '2x2', 10),

('KJI 112Z', 'Wema Executive 11', 40, '2x2', 11),

('KOT 120F', 'Wema Executive 12', 40, '2x2', 12),

('KJX 893M', 'Wema Executive 13', 40, '2x2', 13),

('KAL 785Y', 'Wema Executive 14', 40, '2x2', 14),

('UAP 465F', 'Wema Executive 15', 40, '2x2', 15),

('UTK 299E', 'Wema Executive 16', 40, '2x2', 16),

('T 175 CCH', 'Wema Executive 17', 40, '2x2', 17),

('T 676 VKM', 'Wema Executive 18', 40, '2x2', 18),

('RQI 605 S', 'Wema Executive 19', 40, '2x2', 19),

('ENM 318 GP', 'Wema Executive 20', 40, '2x2', 20),

('KHJ 537G', 'Wema Executive 21', 40, '2x2', 21),

('KZQ 699X', 'Wema Executive 22', 40, '2x2', 22),

('KUI 133D', 'Wema Executive 23', 40, '2x2', 23),

('KIS 653J', 'Wema Executive 24', 40, '2x2', 24),

('UIH 545W', 'Wema Executive 25', 40, '2x2', 25),

('UOF 319L', 'Wema Executive 26', 40, '2x2', 26),

('T 788 UOC', 'Wema Executive 27', 40, '2x2', 27), ('T 909 NSY', 'Wema Executive 28', 40, '2x2', 28),

('RFA 598 Q', 'Wema Executive 29', 40, '2x2', 29),

('YET 839 GP', 'Wema Executive 30', 40, '2x2', 30);

INSERT ROUTES (International)

INSERT INTO routes (from_location, to_location, departure_date, departure_time, cost, bus_id) VALUES

('Nairobi, Kenya', 'Kampala, Uganda', '2026-04-07', '12:00:00', 3500, 1),

('Kampala, Uganda', 'Nairobi, Kenya', '2026-06-14', '15:10:00', 3500, 2),

('Nairobi, Kenya', 'Arusha, Tanzania', '2026-11-25', '14:10:00', 2500, 3),

('Arusha, Tanzania', 'Nairobi, Kenya', '2026-09-20', '20:40:00', 2500, 4),

('Johannesburg, SA', 'Asmara, Eritrea', '2026-05-22', '13:30:00', 25000, 5),

('Mombasa, Kenya', 'Dar es Salaam, Tanzania', '2026-10-05', '17:40:00', 4000, 6),

('Kigali, Rwanda', 'Kampala, Uganda', '2026-11-03', '13:40:00', 2000, 7),

('Bujumbura, Burundi', 'Kigali, Rwanda', '2026-03-18', '14:10:00', 1500, 8),

('Addis Ababa, Ethiopia', 'Nairobi, Kenya', '2026-08-18', '20:00:00', 6000, 9),

('Lusaka, Zambia', 'Dar es Salaam, Tanzania', '2026-06-12', '12:40:00', 8000, 10),

('Nairobi, Kenya', 'Kampala, Uganda', '2026-11-15', '07:10:00', 3500, 11),

('Kampala, Uganda', 'Nairobi, Kenya', '2026-09-01', '10:40:00', 3500, 12),

('Nairobi, Kenya', 'Arusha, Tanzania', '2026-10-05', '17:10:00', 2500, 13),

('Arusha, Tanzania', 'Nairobi, Kenya', '2026-01-16', '20:20:00', 2500, 14),

('Johannesburg, SA', 'Asmara, Eritrea', '2026-05-25', '10:40:00', 25000, 15),

('Mombasa, Kenya', 'Dar es Salaam, Tanzania', '2026-01-20', '20:10:00', 4000, 16),

('Kigali, Rwanda', 'Kampala, Uganda', '2026-04-07', '06:50:00', 2000, 17),

('Bujumbura, Burundi', 'Kigali, Rwanda', '2026-03-26', '06:50:00', 1500, 18),

('Addis Ababa, Ethiopia', 'Nairobi, Kenya', '2026-10-26', '10:20:00', 6000, 19),

('Lusaka, Zambia', 'Dar es Salaam, Tanzania', '2026-04-12', '15:10:00', 8000, 20),

('Nairobi, Kenya', 'Kampala, Uganda', '2026-09-26', '17:20:00', 3500, 21), ('Kampala, Uganda', 'Nairobi, Kenya', '2026-10-24', '16:10:00', 3500, 22),

('Nairobi, Kenya', 'Arusha, Tanzania', '2026-09-05', '09:30:00', 2500, 23),

('Arusha, Tanzania', 'Nairobi, Kenya', '2026-11-04', '09:10:00', 2500, 24),

('Johannesburg, SA', 'Asmara, Eritrea', '2026-08-01', '18:10:00', 25000, 25),

('Mombasa, Kenya', 'Dar es Salaam, Tanzania', '2026-09-09', '08:30:00', 4000, 26),

('Kigali, Rwanda', 'Kampala, Uganda', '2026-09-11', '20:40:00', 2000, 27),

('Bujumbura, Burundi', 'Kigali, Rwanda', '2026-12-07', '20:30:00', 1500, 28),

('Addis Ababa, Ethiopia', 'Nairobi, Kenya', '2026-08-11', '12:20:00', 6000, 29),

('Lusaka, Zambia', 'Dar es Salaam, Tanzania', '2026-06-20', '10:50:00', 8000, 30);

INSERT BOOKINGS

INSERT INTO bookings (booking_time, seat_number, booking_status, qr_token, user_id, route_id, bus_id) VALUES

('2026-06-05 20:00:00', '2A', 'PAID', 'e6233292db93c159fca80cd558ffbfd9', 22, 2, 2),

('2026-10-03 15:00:00', '3A', 'PAID', '0db1515fc46edd6e578051bcd29375b6', 23, 3, 3),

('2026-02-16 20:00:00', '4A', 'PAID', '730e6e438d5f966a527f4f538f132003', 24, 4, 4),

('2026-04-26 14:00:00', '5A', 'PAID', '66fc8bc02fd8ac40242e137f1830bfb9', 25, 5, 5),

('2026-03-27 12:00:00', '6A', 'PAID', '622d6ca0678c67e7fdd6a7d368465fa4', 26, 6, 6),

('2026-02-04 13:00:00', '7A', 'PAID', '26f3029aeed588071f58b38de5757653', 27, 7, 7),

('2026-09-05 20:00:00', '8A', 'PAID', 'dc5486d63381738a637789a92b8085fa', 28, 8, 8),

('2026-04-20 16:00:00', '9A', 'PAID', '71b059ece856b4830ff8068af95eec29', 29, 9, 9),

('2026-06-05 15:00:00', '10A', 'PAID', '4477b9b52f1ae4f00c21a4ea55af147a', 30, 10, 10),

('2026-12-24 10:00:00', '11A', 'PAID', '8a51b1f62ed3c7abd8fd96a67b744c63', 21, 11, 11),

('2026-07-03 19:00:00', '12A', 'PAID', 'eabec05090e648641aca30b6dc9dde54', 22, 12, 12),

('2026-09-18 09:00:00', '13A', 'PAID', '0688f64043a983e55793317de1d00730', 23, 13, 13),

('2026-06-20 14:00:00', '14A', 'PAID', '61d1e57c0708a4edc51066ae81067bac', 24, 14, 14),

('2026-06-19 17:00:00', '15A', 'PAID', '0d5ba3994aa82106ff5e852aa841f395', 25, 15, 15),

('2026-04-14 08:00:00', '16A', 'PAID', '52a4dd281e934256937909057dad588c', 26, 16, 16), ('2026-11-11 12:00:00', '17A', 'PAID', '32fe983757e7eb28580a7a519bc5cd7a', 27, 17, 17),

('2026-01-11 10:00:00', '18A', 'PAID', '373db2a35abc3e3b8b1f6a9eaf88b9bf', 28, 18, 18),

('2026-08-20 19:00:00', '19A', 'PAID', '31400a08f0666ae9067b670816f3a7b7', 29, 19, 19),

('2026-06-01 08:00:00', '20A', 'PAID', '747d0627ee743985911357a1bb6c2185', 30, 20, 20),

('2026-07-14 16:00:00', '21A', 'PAID', 'da61c9e0d991ec6df279d64349ac43b8', 21, 21, 21),

('2026-06-22 14:00:00', '22A', 'PAID', '8d9c0cd02e567801f2a89bcdbd17a720', 22, 22, 22),

('2026-08-24 12:00:00', '23A', 'PAID', '99ac1e189752a4d8b44304de6cd6bbe9', 23, 23, 23),

('2026-07-22 14:00:00', '24A', 'PAID', '98cd95edd59baf6a2333c93be1ecd4b8', 24, 24, 24),

('2026-11-18 14:00:00', '25A', 'PAID', '7c100301a8e2ff2b11e6df1d95730ba5', 25, 25, 25),

('2026-01-05 17:00:00', '26A', 'PAID', '149cf7b391f41ebd4e0151b3c1619c62', 26, 26, 26),

('2026-06-23 16:00:00', '27A', 'PAID', '46f3d5ae88f84387aa08bdf6afd424ed', 27, 27, 27),

('2026-10-01 11:00:00', '28A', 'PAID', 'a23cc6ba5f5ab6e6b4d1d044f30a245d', 28, 28, 28),

('2026-08-04 08:00:00', '29A', 'PAID', '7f89ebecb0dc36b7d2f55fd2480667e1', 29, 29, 29),

('2026-04-21 13:00:00', '30A', 'PAID', '5095b52c5c74e2c358469a1b4fd6755b', 30, 30, 30),

('2026-09-19 14:00:00', '31A', 'PAID', 'e6b951a4a4eb2d2543f08052046565d1', 21, 1, 1);

INSERT FEEDBACK

INSERT INTO feedback (rating, comments, feedback_date, user_id, bus_id, route_id) VALUES

(4, 'Charging ports were not working, please fix.', '2026-12-11', 22, 2, 2),

(3, 'Charging ports were not working, please fix.', '2026-08-04', 23, 3, 3),

(5, 'Great service, very comfortable bus.', '2026-03-25', 24, 4, 4),

(4, 'Love the new buses, very modern and fresh.', '2026-12-25', 25, 5, 5),

(5, 'Bus was a bit late but the ride was okay.', '2026-02-08', 26, 6, 6),

(5, 'Excellent staff and safe driving. Will book again.', '2026-07-17', 27, 7, 7),

(5, 'The driver drove carefully, I felt safe throughout.', '2026-06-02', 28, 8, 8),

(5, 'Best travel experience I have had in a long time.', '2026-06-28', 29, 9, 9),

(5, 'Excellent staff and safe driving. Will book again.', '2026-09-26', 30, 10, 10),

(3, 'Enjoyed the free Wi-Fi, it was surprisingly fast.', '2026-08-09', 21, 11, 11), (3, 'The snacks provided were a nice touch.', '2026-09-17', 22, 12, 12),

(5, 'Great service, very comfortable bus.', '2026-07-18', 23, 13, 13),

(4, 'Enjoyed the free Wi-Fi, it was surprisingly fast.', '2026-03-17', 24, 14, 14),

(3, 'Too many stops along the way made the trip longer.', '2026-03-13', 25, 15, 15),

(3, 'Booking process was easy and the bus was on time.', '2026-04-11', 26, 16, 16),

(3, 'Too many stops along the way made the trip longer.', '2026-01-15', 27, 17, 17),

(5, 'The bus was noisy, could not sleep well.', '2026-09-15', 28, 18, 18),

(5, 'Booking process was easy and the bus was on time.', '2026-04-16', 29, 19, 19),

(5, 'Bus was a bit late but the ride was okay.', '2026-07-14', 30, 20, 20),

(3, 'Booking process was easy and the bus was on time.', '2026-03-27', 21, 21, 21),

(5, 'Excellent staff and safe driving. Will book again.', '2026-01-11', 22, 22, 22),

(4, 'The journey was smooth and the driver was professional.', '2026-08-25', 23, 23, 23),

(3, 'Enjoyed the free Wi-Fi, it was surprisingly fast.', '2026-06-09', 24, 24, 24),

(4, 'Customer service was helpful when I needed to change my seat.', '2026-02-18', 25, 25, 25),

(5, 'Great service, very comfortable bus.', '2026-04-05', 26, 26, 26),

(4, 'The bus was noisy, could not sleep well.', '2026-12-26', 27, 27, 27),

(4, 'Bus was a bit late but the ride was okay.', '2026-10-01', 28, 28, 28),

(5, 'The journey was smooth and the driver was professional.', '2026-07-06', 29, 29, 29),

(3, 'Arrived at the destination ahead of schedule!', '2026-06-20', 30, 30, 30),

(4, 'Love the new buses, very modern and fresh.', '2026-07-26', 21, 1, 1);

MY SQL CLI SNAPSHOTS

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| ibbs               |
| ibbs_prototype     |
| information_schema |
| mysql              |
| performance_schema |
| phpmyadmin         |
| test               |
| wema_transport     |
+--------------------+
8 rows in set (0.16 sec)

mysql> use ibbs_prototype;
Database changed
mysql> show tables;
+--------------------------+
| Tables_in_ibbs_prototype |
+--------------------------+
| bookings                 |
| buses                    |
| drivers                  |
| feedback                 |
| routes                   |
| users                    |
+--------------------------+
6 rows in set (0.01 sec)
```

```
mysql> desc bookings;
+-------------------+------------------------------------------------+------+-----+---------------------+----------------+
| Field             | Type                                           | Null | Key | Default             | Extra          |
+-------------------+------------------------------------------------+------+-----+---------------------+----------------+
| booking_id        | int(11)                                        | NO   | PRI | NULL                | auto_increment |
| booking_time      | datetime                                       | NO   |     | current_timestamp() |                |
| seat_number       | varchar(10)                                    | NO   |     | NULL                |                |
| booking_status    | enum('CONFIRMED','CANCELLED','PAID','CHECKED_IN') | YES  |     | PAID                |                |
| qr_token          | varchar(255)                                   | YES  |     | NULL                |                |
| user_id           | int(11)                                        | NO   | MUL | NULL                |                |
| route_id          | int(11)                                        | NO   | MUL | NULL                |                |
| bus_id            | int(11)                                        | NO   | MUL | NULL                |                |
| passenger_name    | varchar(255)                                   | NO   |     |                     |                |
| passenger_age     | int(11)                                        | NO   |     | 0                   |                |
| passenger_dob     | date                                           | YES  |     | NULL                |                |
| passenger_id_number | varchar(50)                                  | NO   |     |                     |                |
+-------------------+------------------------------------------------+------+-----+---------------------+----------------+
12 rows in set (0.01 sec)
```

```
mysql> desc buses;
+----------------+-------------+------+-----+---------+----------------+
| Field          | Type        | Null | Key | Default | Extra          |
+----------------+-------------+------+-----+---------+----------------+
| bus_id         | int(11)     | NO   | PRI | NULL    | auto_increment |
| reg_no         | varchar(20) | NO   | UNI | NULL    |                |
| bus_name       | varchar(50) | YES  |     | NULL    |                |
| max_passengers | int(11)     | NO   |     | NULL    |                |
| seat_layout    | varchar(20) | NO   |     | NULL    |                |
| driver_id      | int(11)     | YES  | MUL | NULL    |                |
+----------------+-------------+------+-----+---------+----------------+
6 rows in set (0.06 sec)

mysql> desc drivers;
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| driver_id   | int(11)      | NO   | PRI | NULL    | auto_increment |
| national_id | varchar(20)  | NO   | UNI | NULL    |                |
| full_name   | varchar(100) | NO   |     | NULL    |                |
| phone       | varchar(15)  | NO   | UNI | NULL    |                |
| email       | varchar(100) | NO   | UNI | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
5 rows in set (0.06 sec)
```

```
mysql> desc feedback;
+---------------+-----------+------+-----+---------------------+----------------+
| Field         | Type      | Null | Key | Default             | Extra          |
+---------------+-----------+------+-----+---------------------+----------------+
| feedback_id   | int(11)   | NO   | PRI | NULL                | auto_increment |
| rating        | int(11)   | YES  |     | 5                   |                |
| comments      | text      | NO   |     | NULL                |                |
| feedback_date | date      | NO   |     | NULL                |                |
| user_id       | int(11)   | NO   | MUL | NULL                |                |
| bus_id        | int(11)   | NO   | MUL | NULL                |                |
| route_id      | int(11)   | NO   | MUL | NULL                |                |
| submitted_at  | timestamp | NO   |     | current_timestamp() |                |
+---------------+-----------+------+-----+---------------------+----------------+
8 rows in set (0.06 sec)

mysql> desc routes;
+----------------+-----------------------------------------------+------+-----+-----------+----------------+
| Field          | Type                                          | Null | Key | Default   | Extra          |
+----------------+-----------------------------------------------+------+-----+-----------+----------------+
| route_id       | int(11)                                       | NO   | PRI | NULL      | auto_increment |
| from_location  | varchar(100)                                  | NO   |     | NULL      |                |
| to_location    | varchar(100)                                  | NO   |     | NULL      |                |
| departure_date | date                                          | NO   |     | NULL      |                |
| departure_time | time                                          | NO   |     | NULL      |                |
| cost           | decimal(10,2)                                 | NO   |     | NULL      |                |
| bus_id         | int(11)                                       | NO   | MUL | NULL      |                |
| status         | enum('SCHEDULED','COMPLETED','CANCELLED')     | YES  |     | SCHEDULED |                |
+----------------+-----------------------------------------------+------+-----+-----------+----------------+
8 rows in set (0.07 sec)
```

```
mysql> desc users;
+--------------+------------------------------------+------+-----+---------------------+----------------+
| Field        | Type                               | Null | Key | Default             | Extra          |
+--------------+------------------------------------+------+-----+---------------------+----------------+
| user_id      | int(11)                            | NO   | PRI | NULL                | auto_increment |
| first_name   | varchar(50)                        | NO   |     | NULL                |                |
| last_name    | varchar(50)                        | NO   |     | NULL                |                |
| email        | varchar(100)                       | NO   | UNI | NULL                |                |
| phone_number | varchar(15)                        | NO   | UNI | NULL                |                |
| password     | varchar(255)                       | NO   |     | NULL                |                |
| role         | enum('PASSENGER','ADMIN','AGENT')  | YES  |     | PASSENGER           |                |
| created_at   | timestamp                          | NO   |     | current_timestamp() |                |
+--------------+------------------------------------+------+-----+---------------------+----------------+
8 rows in set (0.06 sec)
```

DB CONNECTION SCRIPT

```
<?php
// ====================================================================
// THE DATABASE PLUG-IN (CONNECTION CONFIGURATION)
```

```
// ===========================================================================
// Think of this script like a "plug" that connects our website to the
// filing cabinet (Database) where all our data is stored.
// We include this file at the top of every page that needs to READ or WRITE data.
// ===========================================================================

// --- STEP 1: ENABLE ERROR REPORTING ---
// When we are learning to code, mistakes happen!
// These two lines tell the computer: "If there is a mistake in my code,
// please show me a big message so I can find and fix it."
error_reporting(E_ALL);
ini_set('display_errors', '1');

// --- STEP 2: DEFINE THE FILING CABINET (DB) LOCATION ---
// $server_name: Where is the database? "localhost" means it's on the same computer as the code.
// $username: The ID used to enter the database. "root" is the default for XAMPP.
// $password: The secret key. By default in XAMPP, this is empty ("").
// $database_name: The specific folder/cabinet we created in MySQL.
$server_name = "localhost";
$username = "root";
$password = "";
$database_name = "IBBS_PROTOTYPE";
$port = 3306; // This is the default "door number" (port) that MySQL uses.

// --- STEP 3: ESTABLISH THE CONNECTION ---
// We create a new 'mysqli' object. This is like picking up the phone and calling the database.
$conn = new mysqli($server_name, $username, $password, $database_name, $port);
```

```php
// --- STEP 4: CHECK IF THE CALL WAS SUCCESSFUL ---
// if ($conn->connect_error) checks if someone "picked up the phone".
if ($conn->connect_error) {
    // If it failed (like a busy signal), we stop everything (die) and show the error.
    die("Connection Failed: " . $conn->connect_error);
}

// If we reach this point, the connection is working!
// The $conn variable is now ready to be used to run SQL commands.
?>
```

CODE FOR SENDING DATA TO DATABASE (SIGNUP & LOGIN LOGIC)

```php
<?php
// This is the main "Account Engine" script. It handles two big jobs:
// 1. SIGNING UP: Letting new people create an account.
// 2. LOGGING IN: Letting existing people enter the site.

// We tell PHP to show all errors. This is very helpful for us as students because
// if something goes wrong, the computer will tell us exactly which line has a problem!
error_reporting(E_ALL);
ini_set('display_errors', '1');

// session_start() is like opening a user's personal notebook on the server.
```

```
// Anything we write in there (like their name or ID) will be remembered on every page they visit.
session_start();

// We need to talk to the database to save or find users, so we include our connection file.
require_once 'db_connection.php';

// ===========================================================================
// PART 1: SIGNING UP (CREATE A NEW ACCOUNT)
// ===========================================================================

// We check if the user clicked the button named 'save' in the signup form.
if (isset($_POST['save'])) {

    // We collect the information the user typed.
    // trim() removes any accidental spaces at the beginning or end of what they typed.
    $first_name = trim($_POST['first_name']);
    $last_name = trim($_POST['last_name']);
    $email = trim($_POST['email']);
    $phone_number = trim($_POST['phone_number']);
    $password_raw = trim($_POST['password']);

    // --- STEP A: SECURITY CHECK (PASSWORD STRENGTH) ---
    // We want to make sure passwords are hard to guess.
    // This 'preg_match' is a pattern checker. It makes sure the password has:
    // One big letter, one small letter, one number, and one symbol.
    if (!preg_match('/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*()\-_=+{};:,<.>]).{8,}$/', $password_raw)) {
```

```
        die("Password must contain at least one uppercase, one lowercase, one digit, one special character, and be at least 8 characters long.");
    }

    // --- STEP B: CHECK IF EMAIL IS REAL ---
    // filter_var helps us check if the email has an @ symbol and a proper structure.
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        die("Invalid email address.");
    }

    // --- STEP C: LIMIT DOMAINS (Project Rule) ---
    // We only want to allow common email services like Gmail or Yahoo for this prototype.
    $allowed_domains = ["gmail.com", "yahoo.com", "hotmail.com"];
    $email_domain = explode("@", $email)[1]; // This grabs the part AFTER the '@'
    if (!in_array($email_domain, $allowed_domains)) {
        die("Email domain not allowed. Only Gmail, Yahoo, and Hotmail accepted.");
    }

    // --- STEP D: CHECK FOR DUPLICATES ---
    // We don't want two people using the same email or phone number.
    // We use a 'prepared statement' (?) to protect our database from hackers.
    $stmt = $conn->prepare("SELECT email FROM users WHERE email = ? OR phone_number = ?");
    $stmt->bind_param("ss", $email, $phone_number);
    $stmt->execute();
    $stmt->store_result();

    // If the computer found a row (num_rows > 0), it means the email/phone is already in the list!
```

```
    if ($stmt->num_rows > 0) {
        $stmt->close();
        die("User with this email or phone number already exists!");
    }
    $stmt->close();

    // --- STEP E: HASHING (SCRAMBLING) THE PASSWORD ---
    // We NEVER save real passwords. Instead, we scramble them into a long code.
    // Even if a hacker sees the database, they won't know the real password.
    $hashed_password = password_hash($password_raw, PASSWORD_DEFAULT);

    // Everyone who signs up through the public form starts as a "PASSENGER".
    $role = "PASSENGER";

    // --- STEP F: SAVE TO DATABASE ---
    // Now we insert the new user into our 'users' table.
    $stmt = $conn->prepare("INSERT INTO users (first_name, last_name, email, phone_number,
password, role) VALUES (?, ?, ?, ?, ?, ?)");
    // "ssssss" means we are providing 6 fragments of text (strings).
    $stmt->bind_param("ssssss", $first_name, $last_name, $email, $phone_number,
$hashed_password, $role);

    if ($stmt->execute()) {
        // SUCCESS! Now we automatically log them in.
        $new_user_id = $stmt->insert_id; // Get the unique ID the database just made for them.

        // Write their info into our server notebook (Session).
        $_SESSION['user_id'] = $new_user_id;
```

```
        $_SESSION['role'] = $role;
        $_SESSION['name'] = $first_name . " " . $last_name;

        $stmt->close();

        // Send them to their new Dashboard!
        echo "<script>alert('Sign-up successful! Welcome to Wema Travellers.');
window.location.href='dashboard.php';</script>";
        exit();
    } else {
        $stmt->close();
        die("Signup failed: " . $conn->error);
    }
}

// ======================================================================
// PART 2: LOGGING IN (ENTER THE SITE)
// ======================================================================

// We check if the user clicked the 'login' button.
if (isset($_POST['login'])) {

    // Collect what they typed in the login boxes.
    $email = trim($_POST['email']);
    $password_input = trim($_POST['password']);

    // --- STEP A: FIND THE USER ---
```

```
    // We ask the database: "Find the person who has this email."
    $stmt = $conn->prepare("SELECT user_id, password, role, first_name, last_name FROM
users WHERE email = ?");
    $stmt->bind_param("s", $email);
    $stmt->execute();
    $stmt->store_result();
    // Link the database results to our variables.
    $stmt->bind_result($user_id, $stored_password, $role, $f_name, $l_name);

    // If we found the email (rows > 0), now we check if they typed the right password.
    if ($stmt->num_rows > 0) {
        $stmt->fetch();

        // --- STEP B: CHECK PASSWORD ---
        // 'password_verify' compares the typed password with the scrambled one in the database.
        if (password_verify($password_input, $stored_password)) {
            // SUCCESS! Let's remember them in our notebook.
            $_SESSION['user_id'] = $user_id;
            $_SESSION['role'] = $role;
            $_SESSION['name'] = $f_name . " " . $l_name;
            $stmt->close();

            // --- STEP C: ROLE-BASED REDIRECTION ---
            // We send users to different places depending on their job (role).
            if($role == 'ADMIN') {
                // Administrators go to their secret Control Panel.
                echo "<script>alert('Login successful! Welcome Admin.');
window.location.href='admin_dashboard.php';</script>";
```

```
            } elseif ($role == 'AGENT') {
                // Agents go to their Workspace.
                echo "<script>alert('Login successful! Welcome Agent.');
window.location.href='agent_dashboard.php';</script>";
            } else {
                // Regular passengers go to their simple Dashboard.
                echo "<script>alert('Login successful!');
window.location.href='dashboard.php';</script>";
            }
            exit();
        } else {
            // WRONG PASSWORD: Tell them and send them back to try again.
            $stmt->close();
            echo "<script>alert('Incorrect password!'); window.location.href='login.html';</script>";
            exit();
        }
    } else {
        // EMAIL NOT FOUND: If the email isn't in our list at all.
        $stmt->close();
        echo "<script>alert('Email not found!'); window.location.href='login.html';</script>";
        exit();
    }
}

// Close the database connection when the engine is finished.
$conn->close();
?>
```

B. IMPLIMENTING USER INTERFACE DESIGN

INDEX.HTML

```
<!DOCTYPE html> <!-- Declares this document as an HTML5 document, the modern standard for web pages. -->

<html lang="en">

<!-- The root element of the HTML page, specifying "en" (English) as the primary language for SEO and accessibility. -->


<head>

  <!-- Container for metadata: information about the page that isn't displayed directly to the user but is used by the browser and search engines. -->

  <meta charset="UTF-8">

  <!-- Sets the character encoding to UTF-8, which supports almost all characters and symbols across all languages. -->

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <!-- Ensures the page is responsive, setting the initial zoom level to 100% and matching the device's width. -->

  <title>Wema Travellers - Home</title>

  <!-- The text shown in the browser tab and search engine results for this page. -->

  <link rel="stylesheet" href="css/style.css">

  <!-- Links an external CSS file for theme variables and common styles across multiple pages. -->

  <link rel="stylesheet" href="css/main.css">

  <!-- Links an external CSS file for structural and layout-specific styles. -->

  <style>

    /* Opens an internal stylesheet block to define styles specific only to this home page. */

    /* Hero Section - The large, visually striking header at the top of the page. */
```

.hero {

/* background-image: Applies a visual layer. linear-gradient creates a dark overlay for text readability. */

/* rgba(0, 0, 0, 0.5): r=red(0), g=green(0), b=blue(0), a=alpha/transparency(0.5 or 50%). It creates a semi-transparent black. */

/* url('img/wema_bus_sunset.png'): The actual background image path. */

background-image: linear-gradient(rgba(0, 0, 0, 0.5), rgba(0, 0, 0, 0.5)), url('img/wema_bus_sunset.png');

height: 60vh;

/* Sets the height to 60% of the viewport height (the user's screen height). */

background-position: center;

/* Ensures the focal point of the background image stays in the center of the box. */

background-size: cover;

/* Commands the image to stretch or shrink to fill the entire container without losing its aspect ratio. */

display: flex;

/* Enables Flexbox, an advanced layout tool for aligning child elements. */

justify-content: center;

/* Horizontally centers all content within this flex container. */

align-items: center;

/* Vertically centers all content within this flex container. */

text-align: center;

/* Centers any wrapped or inline text inside this container. */

color: white;

/* Sets the default text color to white for high contrast against the dark background. */

margin-bottom: 40px;

/* Adds 40 pixels of empty space below the hero section to separate it from the content. */

}

```
.hero h1 {
  font-size: 3rem;
  /* Sets a large font size relative to the default (typically 3 times the standard size). */
  margin-bottom: 20px;
  /* Adds space below the main heading. */
  /* text-shadow: Adds a subtle shadow behind the text for better readability on complex backgrounds. */
  /* 2px(horizontal offset) 2px(vertical offset) 4px(blur radius) rgba(0,0,0,0.5)(black at 50% opacity). */
  text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.5);
}

.hero p {
  font-size: 1.5rem;
  /* Sets a slightly smaller but still significant font size for the sub-heading. */
  max-width: 800px;
  /* Limits the text width to 800 pixels so it stays readable on wide monitors. */
  margin: 0 auto;
  /* Centers the paragraph element itself horizontally within its parent. */
}

/* Section Container - Standardizes the width and padding for content blocks on the page. */
.section-container {
  max-width: 1200px;
  /* Prevents the content from stretching too wide on large screens, maintaining a focus center. */
  margin: 0 auto;
```

```css
  /* Centers the container block horizontally. */
  padding: 40px 20px;
  /* Adds 40px padding to the top/bottom and 20px to the left/right. */
}

h2.section-title {
  text-align: center;
  /* Centers the heading text. */
  color: #333;
  /* Sets a dark grey color (#333) for a professional look. */
  font-size: 2.5rem;
  /* Sets a large, prominent font size. */
  margin-bottom: 40px;
  /* Adds space below the title. */
}

h2.section-title::after {
  content: '';
  /* Required property for pseudo-elements (::after); creates a blank virtual box. */
  display: block;
  /* Makes the box occupy its own line, allowing us to size it like an underline. */
  width: 80px;
  /* Determines the length of the underline. */
  height: 4px;
  /* Determines the thickness of the underline. */
  background-color: var(--pink);
  /* Applies the brand's primary pink color using a CSS variable. */
```

```
  margin: 10px auto 0;
  /* Adds 10px top margin and uses 'auto' flags to center the box. */
  border-radius: 2px;
  /* Curves the edges of the underline for a softer, more modern look. */
}

/* Vision Mission Goals Grid - Arrangement for internal company value cards. */
.vmg-grid {
  display: grid;
  /* Enables CSS Grid, the most powerful system for 2-dimensional layouts (rows and
columns). */
  /* repeat(auto-fit...): Automatically adds as many columns as fit the screen, with a minimum
of 300px width. */
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 30px;
  /* Adds a 30-pixel gap between each card in the grid. */
  margin-bottom: 60px;
  /* Adds significant space below the grid section. */
}

.vmg-card {
  background: white;
  /* Ensures the card has a solid white backdrop. */
  padding: 30px;
  /* Adds internal space between the card's edge and its content. */
  border-radius: 12px;
  /* Rounds the corners of the card. */
```

```
    /* box-shadow: Creates a soft elevation effect. 0(x) 4px(y) 15px(blur) rgba(black at 10% transparency). */
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
    text-align: center;
    /* Centers all text content inside the card. */
    transition: transform 0.3s ease;
    /* Prepares a 0.3-second smooth animation for the 'hover' transform below. */
    border-top: 5px solid var(--purple);
    /* Adds a thick purple border at the top for branding and visual interest. */
  }

  .vmg-card:hover {
    transform: translateY(-10px);
    /* Lifts the card upwards by 10 pixels when the mouse cursor is over it. */
  }

  .vmg-card h3 {
    color: var(--purple);
    /* Applies branding purple to the card title. */
    font-size: 1.8rem;
    /* Sets a mid-range font size for headings in cards. */
    margin-bottom: 15px;
    /* Adds space between the title and the paragraph below. */
  }

  .vmg-card p {
    color: #666;
```

```css
    /* Uses a softer grey for the body text to reduce eye strain. */
    line-height: 1.6;
    /* Increases vertical space between lines of text for better readability. */
  }

  /* Countries Grid - Layout for the destination list labels. */
  .countries-grid {
    display: grid;
    /* Activates grid layout mode. */
    /* auto-fill: Similar to auto-fit, it populates the row dynamically. Each pill is min 150px wide.
*/
    grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
    gap: 20px;
    /* Spacing between the country pills. */
    text-align: center;
    /* Centers text within the pills. */
  }

  .country-pill {
    background-color: #f0f0f0;
    /* Sets a very light grey background for the pills. */
    padding: 15px;
    /* Internal spacing for the pill shape. */
    border-radius: 50px;
    /* Fully rounds the sides to create a 'pill' or 'button' shape. */
    font-weight: 600;
    /* Makes the text semi-bold. */
```

```css
  color: #444;
  /* Dark grey text. */
  transition: background 0.3s, color 0.3s;
  /* Smoothly fades the background and text color on interaction. */
}

/*
HOVER EFFECT: Enhances the pill appearance when hovered.
*/
.country-pill:hover {
  background-color: #4a1a4a !important;
  /* Forces a dark purple background on hover. */
  border: 2px solid #4a1a4a !important;
  /* Adds a matching border. */
  color: white !important;
  /* Flips text to white for clarity against the dark purple. */
  cursor: pointer;
  /* Changes mouse to the hand 'clicking' icon. */
  transform: scale(1.05);
  /* Slightly grows the button size by 5%. */
  font-weight: bold;
  /* Makes the text even bolder. */
}

/* Intro Text - The welcoming blurb below the hero section. */
.intro-text {
  text-align: center;
```

```
      /* Centers the introduction paragraph. */
      font-size: 1.2rem;
      /* Sets a slightly larger than normal size for emphasis. */
      color: #555;
      /* Medium grey text color. */
      max-width: 900px;
      /* Prevents the lines from becoming too long and tiring to read. */
      margin: 0 auto 60px;
      /* Centers the block and adds 60px space below it. */
      line-height: 1.8;
      /* Generous spacing between text lines for a clean, open feel. */
    }
  </style> <!-- Closes the internal style block. -->
</head>

<body> <!-- The visible content of the page starts here. -->

  <!-- Public Header - Dynamically injects the standard navigation bar for visitors. -->
  <script src="js/header1.js"></script>

  <!-- Hero Section - Visual branding area. -->
  <div class="hero"> <!-- Background container. -->
    <div class="hero-content"> <!-- Inner container for text centering. -->
      <h1>Welcome to Wema Travellers</h1> <!-- Main tagline. -->
      <p>Your Premium International Bus Travel Partner</p> <!-- Mission summary. -->
    </div>
  </div>
```

```html
 <div class="section-container"> <!-- Main content wrapper for standard alignment. -->


  <!-- Introduction Paragraph -->
  <div class="intro-text">
   <p>
    Wema Travellers is a premier international transport company dedicated to connecting East
and South Africa.
    With a modern fleet of luxury buses and a commitment to punctuality, safety, and comfort,
we bridge the gap
    between nations.
    Our seamless cross-border travel experience is designed for the modern traveler.
   </p>
  </div>


  <!-- Vision, Mission, and Promises (Value Cards) -->
  <div class="vmg-grid"> <!-- Grid wrapper. -->
   <div class="vmg-card"> <!-- Individual value card. -->
    <h3>Our Vision</h3> <!-- Card Title. -->
    <p>To be the absolute leader in safe, reliable, and comfortable international road transport
across Africa,
     connecting communities and business hubs with excellence.</p> <!-- Vision description. --
>
   </div>
   <div class="vmg-card"> <!-- Individual value card. -->
    <h3>Our Mission</h3> <!-- Card Title. -->
    <p>To provide affordable, high-quality, and safe transportation services while exceeding
customer expectations
```

```
          through innovation, professional staff, and a world-class fleet.</p> <!-- Mission
description. -->
      </div>
      <div class="vmg-card"> <!-- Individual value card. -->
        <h3>Our Promise</h3> <!-- Card Title. -->
        <p>We promise punctuality, hospitality, and a hassle-free border crossing experience. Your
safety and comfort
          are our top priorities on every mile of the journey.</p> <!-- Promise description. -->
      </div>
    </div>

    <!-- Countries Coverage (Visual List) -->
    <h2 class="section-title">Destinations We Cover</h2> <!-- Section heading with custom
underline. -->
    <div class="countries-grid"> <!-- Grid of country labels. -->
      <div class="country-pill">Kenya</div> <!-- Individual country label. -->
      <div class="country-pill">Uganda</div>
      <div class="country-pill">Tanzania</div>
      <div class="country-pill">Rwanda</div>
      <div class="country-pill">South Africa</div>
      <div class="country-pill">Ethiopia</div>
      <div class="country-pill">Zambia</div>
      <div class="country-pill">Malawi</div>
      <div class="country-pill">Zimbabwe</div>
      <div class="country-pill">Burundi</div>
    </div>

  </div> <!-- Closes the section-container. -->
```

```html
  <!-- Footer - Dynamically injects the standard bottom bar (copyright, etc.). -->
  <script src="js/footer.js"></script>

</body> <!-- End of the page container. -->

</html> <!-- End of the HTML document. -->
```

Login.html

```html
<!DOCTYPE html>
<html lang="en">
<!--
  ========================================================================
  LOGIN PAGE
  ========================================================================
  This page allows existing users to log in.

  ========================================================================
  -->

<head>
  <meta charset="UTF-8" />
  <meta http-equiv="X-UA-Compatible" content="IE=edge" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Wema Travellers - Login</title>
```

```html
  <!-- CSS Links -->
  <link rel="stylesheet" href="css/main.css" />
  <link rel="stylesheet" href="css/entry-page.css" />
  <link rel="stylesheet" href="css/style.css" />
</head>

<body>

  <!-- Include Header and Footer Scripts -->
  <script src="js/header1.js"></script>
  <script src="js/footer.js"></script>

  <div class="row height-full">
    <!--
      LEFT SIDE: FORM
      Contains email and password fields for login.
      -->
    <div class="left-column flex flex-column height-full justify-center items-center">
      <h1 class="welcoming-title">Welcome back</h1>

      <!-- login.html Form: Sends credentials to the backend for verification. -->
      <!-- action="account2.php": Points to the central login/signup processing script. -->
      <!-- method="POST": Hides user passwords from the URL log. -->
      <form class="form" action="account2.php" method="POST" autocomplete="off">

        <!-- EMAIL INPUT -->
        <label for="email" class="label">Email</label> <!-- Instructional label. -->
```

```
        <input type="text" name="email" id="email" class="input" required /> <!-- The unique user
email address. -->

        <!-- PASSWORD INPUT -->
        <label for="password" class="label">Password</label> <!-- Instructional label. -->
        <input type="password" name="password" id="password" class="input" required />
        <!-- Masked password field for privacy. -->

        <!-- LOGIN ACTION -->
        <!-- name="login": Tells the PHP code to perform the 'Authentication' check instead of
'Registration'. -->
        <button type="submit" name="login" class="button regular-button pink-background cta-
btn">
          Log in
        </button>
      </form>

      <p class="sign-up-prompt">
        Don’t have an account?
        <a href="./signup.html" class="sign-up-link">Sign up</a>
      </p>
    </div>

    <!--
      RIGHT SIDE: IMAGE
      Displays the Wema Travellers bus image.
      -->
    <div class="right-column"></div>
```

```
  </div>
</body>

</html>
```

Signup.html

```
<!DOCTYPE html>
<html lang="en">
<!--
  ==========================================================================
  SIGNUP PAGE
  ==========================================================================
  This page allows new users to register for the system.
  It collects First Name, Last Name, Email, Phone Number, and Password.

  ==========================================================================
  -->

<head>
  <meta charset="UTF-8" />
  <meta http-equiv="X-UA-Compatible" content="IE=edge" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Wema Travellers - Sign up</title>
```

```
  <!--
      System fonts will be used via CSS.
    -->

  <!-- CSS Links -->
  <link rel="stylesheet" href="css/main.css" />
  <link rel="stylesheet" href="css/entry-page.css" />
  <link rel="stylesheet" href="css/style.css" />
</head>

<body>

  <!-- Include Header and Footer Scripts (Local JS) -->
  <script src="js/header1.js"></script>
  <script src="js/footer.js"></script>

  <div class="row height-full">
    <!--
      LEFT SIDE: FORM
      Contains the input fields for registration.
      -->
    <div class="left-column flex flex-column height-full justify-center items-center">
      <h1 class="welcoming-title">Create Account</h1>

      <!-- signup.html Form: This collects user data and sends it to the server for permanent
storage. -->
      <!-- action="account2.php": Tells the browser to send data to the account processing script. --
>
```

```
<!-- method="POST": Sends data securely in the background (hidden from the URL bar). -->
<form class="form" action="account2.php" method="POST" autocomplete="off">

  <!-- FIRST NAME -->
  <label for="first_name" class="label">First Name</label> <!-- UI Label. -->
  <input type="text" name="first_name" id="first_name" class="input" required />
  <!-- Text input for personal name. -->

  <!-- LAST NAME -->
  <label for="last_name" class="label">Last Name</label> <!-- UI Label. -->
  <input type="text" name="last_name" id="last_name" class="input" required />
  <!-- Text input for family name. -->

  <!-- EMAIL ADDRESS -->
  <label for="email" class="label">Email</label> <!-- UI Label. -->
  <input type="text" name="email" id="email" class="input" /> <!-- User's unique
identification email. -->

  <!-- PHONE NUMBER (Added for SMS notifications/contact) -->
  <label for="phone_number" class="label">Phone Number</label> <!-- UI Label. -->
  <input type="text" name="phone_number" id="phone_number" class="input"
placeholder="e.g. 0712345678" required />
  <!-- Contact phone. -->

  <!-- PASSWORD -->
  <label for="password" class="label">Password</label> <!-- UI Label. -->
  <input type="password" name="password" id="password" class="input" required />
  <!-- Secure password field (dots appear). -->
```

```
        <!-- Security hint text to improve user account strength. -->
        <p style="font-size: 0.85rem; color: gray; margin-bottom: 1rem;">
          Must be at least 8 characters with uppercase, lowercase, number & symbol.
        </p>

        <!-- SUBMIT BUTTON -->
        <!-- name="save": This specific name tells the PHP script that we want to CREATE a new
account. -->
        <button type="submit" name="save" class="button regular-button pink-background cta-
btn">
          Sign up
        </button>
      </form>

      <p class="login-prompt">
        Already have an account?
        <a href="./login.html" class="log-in-link">Log in</a>
      </p>
    </div>

    <!--
      RIGHT SIDE: IMAGE
      Displays the Wema Travellers bus image defined in css/entry-page.css
      -->
    <div class="right-column"></div>
  </div>
</body>
```

</html>

Main.css

```
/*
=============================================================================
===== */

/* MAIN STYLESHEET (main.css) */

/* This is the primary CSS file that defines the global look and feel of the application. */

/*
=============================================================================
===== */

/* CORE STYLES & VARIABLES */

/* We define global design tokens here (colors, spacing) to ensure consistency across all pages.
*/

/*
=============================================================================
===== */


:root { /* The :root pseudo-class represents the <html> element and is used to store global CSS
variables. */

  /* --- COLOR PALETTE (Based on Branding requirements) --- */

  --header-bg: #9a4d9a; /* Defines a Medium Purple color for the top header section. */

  /* Medium Purple (Header Background) */

  --nav-bg: #d06dd0; /* Defines a Lighter Purple for the navigation link area. */

  /* Lighter Purple (Nav Bar Background) */

  --body-bg: #ffccf9; /* Defines a soft Light Pink for the main page background. */

  /* Light Pink (Main Background) */

  --input-bg: #ffe6ff; /* Defines an even lighter pink/white tint for the backgrounds of input
boxes. */

  /* Very Light Pink (Input Fields) */
```

```css
  --text-color: #5a2a5a; /* Defines a high-contrast Dark Purple specifically for text readability. */

  /* Dark Purple (Text) */

  --button-color: #ff9999; /* Defines a 'Peachy Pink' shade used for primary interaction buttons. */

  /* Peachy Pink (Buttons) */

  --button-border: #000000; /* Defines Solid Black for borders to give a bold, defined look. */

  /* Black Border for Buttons/Inputs */


  /* --- BRAND COLORS (Used in new dashboard components) --- */

  --purple: #9a4d9a; /* Re-declaring branding purple for specific component usage. */

  --pink: #d06dd0; /* Re-declaring branding pink for specific component usage. */

}


/* --- GLOBAL RESET --- */

body { /* Targeted styles for the entire page body. */

  margin: 0; /* Removes default browser margin (white space around the edges). */

  padding: 0; /* Removes default browser padding. */

  font-family: 'Comic Sans MS', 'Chalkboard SE', sans-serif; /* Sets a playful, rounded typography style. */

  /* Comic Sans MS is the primary choice; Chalkboard SE is the Mac fallback; sans-serif is the generic system fallback. */

  background-color: var(--body-bg); /* Applies the global pink background defined in the :root variables. */

  color: var(--text-color); /* Applies the global dark purple text color for all content. */

}


/* --- HEADER / BANNER --- */

#banner { /* Styles for the unique element with ID 'banner' (usually at the very top). */
```

```css
  background-color: var(--header-bg); /* Sets background to the branding purple. */
  color: white; /* Forces text inside the banner to be pure white. */
  text-align: center; /* Centers any text or child elements horizontally. */
  padding: 15px; /* Adds 15 pixels of internal cushion space. */
  font-size: 1.5rem; /* Makes the banner text 50% larger than the base font (1.5x size). */
  font-weight: bold; /* Makes the text thick/heavy. */
  font-style: italic; /* Slants the text to the right for a stylized look. */
  /* text-shadow: Adds a drop shadow. 1px(right) 1px(down) 2px(blur) rgba(black at 20% opacity). */
  text-shadow: 1px 1px 2px rgba(0, 0, 0, 0.2);
}

#banner h1 { /* Targeted styles for any <h1> tag nested inside the banner. */
  margin: 0; /* Removes default heading margins to keep the banner height compact and predictable. */
}

/* --- NAVIGATION BAR --- */
#nav-links { /* Container for the site's primary menu links. */
  background-color: var(--nav-bg); /* Sets the lighter purple background. */
  padding: 10px; /* Internal cushion. */
  text-align: center; /* Centers content for browsers that don't support flexbox. */
  display: flex; /* Activates Flexbox layout mode. */
  justify-content: center; /* Centers the menu links horizontally. */
  align-items: center; /* Aligns the links vertically within the 10px padding. */
  position: relative; /* Allows child elements to be positioned relative to this bar if needed. */
}
```

```css
#nav-links a { /* Styles for the links (anchors) inside the navigation bar. */
  color: white; /* Forces links to be white. */
  text-decoration: none; /* Removes the default blue underline from links. */
  font-size: 1.2rem; /* Sets a clear, readable size. */
  margin: 0 15px; /* Adds 15px gap on the left and right of EACH link to separate them. */
  font-weight: bold; /* Makes the menu items stand out. */
}

#nav-links a:hover { /* Visual feedback when the user's mouse is over a link. */
  text-decoration: underline; /* Briefly adds an underline to signal that the item is clickable. */
}

/* --- INPUT FIELDS (Pill Shape) --- */
input.input { /* Styles for <input> elements with the class 'input'. */
  width: 100%; /* Forces the field to stretch to the full width of its container. */
  padding: 12px 20px; /* 12px top/bottom and 20px left/right for comfortable typing space. */
  margin: 8px 0; /* adds vertical spacing to separate fields. */
  display: inline-block; /* Allows the element to respect width/padding while staying in the flow. */
  border: 2px solid var(--button-border); /* Applies a solid 2px black border. */
  border-radius: 25px; /* Curves the edges significantly to create a smooth 'pill' shape. */
  box-sizing: border-box; /* Ensures padding and border are included in the 100% width calculation. */
  background-color: var(--input-bg); /* Applies the light pink background color. */
  font-family: inherit; /* Inherits the playful 'Comic Sans' font from the body. */
  font-size: 1rem; /* Standard text size for inputs. */
}
```

```
/* --- BUTTONS (Pill Shape + Shadow) --- */
.button,
button { /* Shared styles for links styled as buttons (.button) and actual <button> elements. */
  background-color: var(--button-color); /* The signature peach color. */
  color: black; /* Dark text for high contrast. */
  padding: 12px 20px; /* Comfortable clicking area. */
  margin: 10px 0; /* Vertical spacing. */
  border: 2px solid var(--button-border); /* Strong black border. */
  border-radius: 25px; /* Pill shape matching the inputs. */
  cursor: pointer; /* Mandatory visual cue: changes cursor to pointer (hand). */
  width: 100%; /* Full-width buttons for a bold mobile-friendly UI. */
  font-size: 1.1rem; /* Slightly larger text for calls to action. */
  font-weight: bold; /* Bold text for importance. */
  /* box-shadow: Creates a 'retro' hard shadow. 3px(right) 3px(down) 0px(no blur) Black at
100% opacity. */
  box-shadow: 3px 3px 0px rgba(0, 0, 0, 1);
  transition: transform 0.1s, box-shadow 0.1s; /* Enables quick 0.1s animation when clicked. */
}

.button:active,
button:active { /* State triggered while the button is being held down by the user's mouse/finger.
*/
  transform: translate(2px, 2px); /* Physically shifts the button 2px right and 2px down. */
  /* Shrinks the shadow to 1px to simulate the button being 'pressed' into the page. */
  box-shadow: 1px 1px 0px rgba(0, 0, 0, 1);
}

/* Specific Button Colors override */
```

```css
.cta-btn { /* 'Call To Action' specific class. */
  background-color: #ff9999 !important; /* !important ensures the peachy peach color is always used regardless of other cascade rules. */
}

/* "Action Buttons" - Smaller buttons used inside table rows or detail views. */
.action-btn {
  display: inline-block; /* Allows buttons to sit side-by-side. */
  width: auto; /* Does not stretch to 100%. */
  padding: 5px 15px; /* Slimmer padding for a compact look. */
  font-size: 0.9rem; /* Smaller text for secondary actions. */
  color: white !important; /* Forces text to white. */
  text-decoration: none; /* Removes underlines. */
  border-radius: 5px; /* Soft rectangular corners instead of pill shape. */
  box-shadow: none; /* Removes the heavy hard shadow for a cleaner look in tables. */
  border: none; /* Removes the heavy border. */
}

.btn-view { /* Stylized specifically for viewing records. */
  background-color: #9a4d9a; /* Deep purple background. */
}

.btn-delete { /* Color-coding specifically for destructive/permanent actions. */
  background-color: #ff4d4d; /* Bright red signals 'Danger' or 'Delete'. */
}

.btn-cancel { /* Color-coding for non-destructive removals (cancellations). */
```

```css
  background-color: #ba68c8; /* Soft purple/lavender. */
}

/* --- LABELS --- */
label { /* Styles for the text descriptive tags above input fields. */
  font-weight: bold; /* Bold labels for clarity and professional structure. */
  display: block; /* Moves the label to its own line so the input field follows below it. */
  margin-top: 10px; /* Gap above the label. */
  margin-bottom: 5px; /* Gap between the label and its input field. */
  color: var(--text-color); /* Uses branding dark purple. */
}

/* --- LINKS --- */
a { /* Global styles for hyperlink tags. */
  color: var(--header-bg); /* Sets links to branding purple by default. */
  text-decoration: none; /* Removes underlines to keep the UI clean. */
}

a:hover { /* Visual feedback. */
  text-decoration: underline; /* Re-adds underline to show interaction. */
}
```

Header2.js

```javascript
//
===========================================================================
=====
// HEADER2.JS (Dashboard Navigation Loader)
```

```
// ==========================================================================
// Purpose: This script manages the navigation bar for the internal "Customer/Staff Portal" (Dashboards, Booking, etc).
// Role: It provides deep links to internal logic pages where session-specific data is active.
// ==========================================================================


// document.addEventListener: Monitors the state of the webpage.
// 'DOMContentLoaded': Fires when the initial HTML document has been completely loaded and parsed.
document.addEventListener('DOMContentLoaded', function () {


  // const header: Stores the HTML code for the complex dashboard navigation bar.
  // We utilize a standard div-based structure to ensure consistent layout across PHP-powered pages.
  const header = `
      <!-- Banner Section with Company Name - Branding persistence -->
      <div id="banner"> <!-- Div ID used for global typography and background purple color application. -->
        <h1>Wema Travellers</h1> <!-- Site-wide header title. -->
      </div> <!-- End of banner div. -->


      <!-- Navigation Links Container - Functional menu for authenticated users -->
      <div id="nav-links"> <!-- Flexbox-driven bar for layout management. -->


        <!-- Standard Links (Centered Group) - Primary portal destinations -->
```

```
      <div style="display: flex; gap: 20px;"> <!-- Flexbox container with a fixed 20px gutter between links. -->
        <a href="home.php">Home</a> <!-- Link to the landing dashboard. -->
        <a href="dashboard.php">Dashboard</a> <!-- Link to the main user/admin control panel. -->
        <a href="profile.php">Profile</a> <!-- Link to view/edit personal account details. -->
        <a href="feedback.php">Feedback</a> <!-- Link for submitting user experience ratings. -->
      </div> <!-- Closes the flex link-group. -->

      <!-- SignOut Link (Absolute Right) - Safety exit specifically positioned for UX clarity -->
      <a href="logout.php" style="position: absolute; right: 20px;">SignOut</a> <!-- Absolute positioning pins this specifically to the right edge. -->
     </div> <!-- Closes the navigation bar. -->
   `; // Ends the header definition.

  // insertAdjacentHTML: Integrates the string directly into the browser's memory of the page content.
  // 'afterbegin': Places it at the top of the body, before any other dashboard content.
  document.body.insertAdjacentHTML('afterbegin', header);
}); // Ends the script.
```

Footer.js

```
// =========================================================================
// FOOTER.JS (Footer Section Loader)
```

```
//
======================================================================
=====

// Purpose: This script dynamically injects a consistent copyright footer at the absolute bottom of
every webpage it is included in.

// This design pattern avoids the need to manually type the footer code into dozens of different
HTML files.

//
======================================================================
=====


// document.addEventListener: Adds an event listener to the entire HTML document.

// 'DOMContentLoaded': Tells the browser to wait until the basic HTML structure (DOM) is
fully loaded and parsed.

// function (): The block of code that will execute as soon as the DOM is ready.

document.addEventListener('DOMContentLoaded', function () {


  // const footer: Declares a constant variable (it won't change) to hold the HTML template.

  // The ` (backtick) allows us to create a multi-line string in JavaScript (Template Literal).

  const footer = `

    <div id="footer"> <!-- A div container with ID 'footer', which is specifically styled in
main.css for theme consistency. -->

      <!-- &copy;: This is a special HTML Entity that instructs the browser to render the circular
Copyright $(\copyright)$ symbol. -->

      <p>&copy; 2026 Wema Travellers. All rights reserved.</p> <!-- The copyright notice text. -
->

    </div> <!-- Closes the footer container. -->

   `; // Ends the string definition.


  // document.body: Selects the <body> element of the current page.
```

// insertAdjacentHTML: A high-performance method to add HTML content into the document.

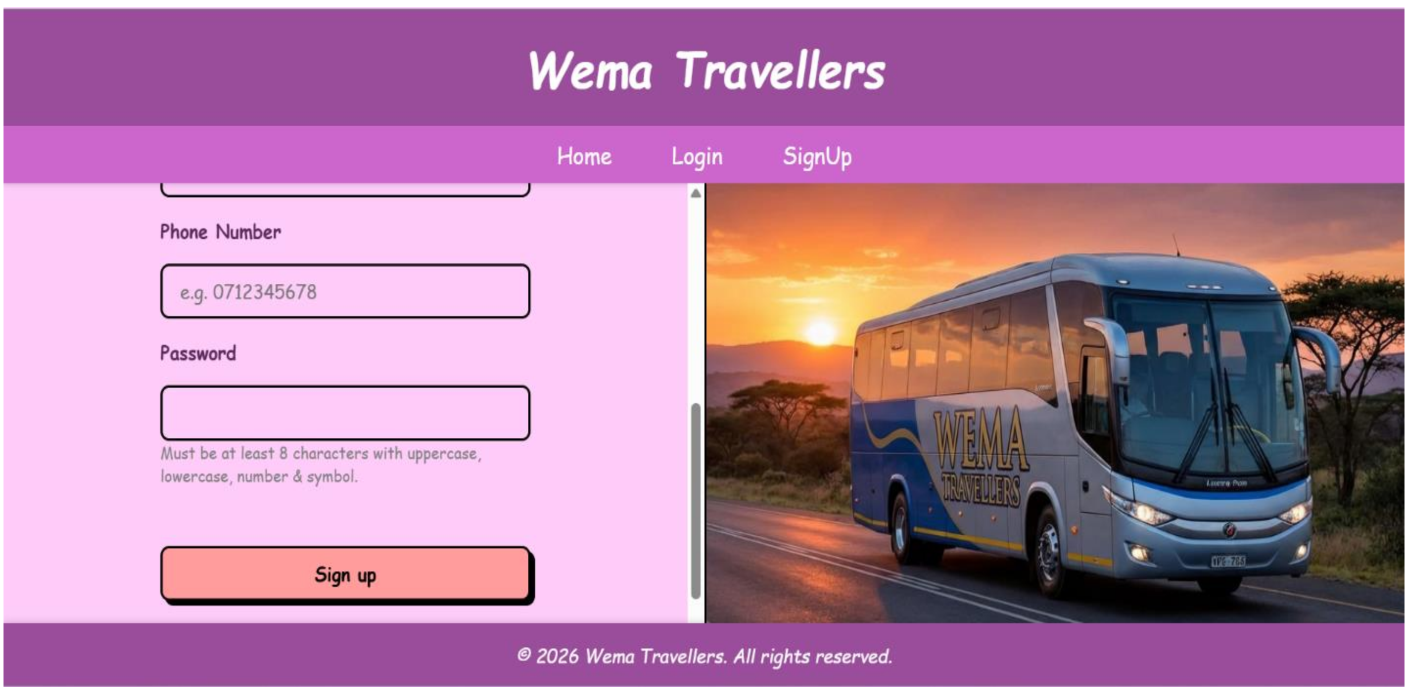
// 'beforeend': A positioning flag that means "place this new HTML just before the </body> tag closes" (i.e., at the very bottom).

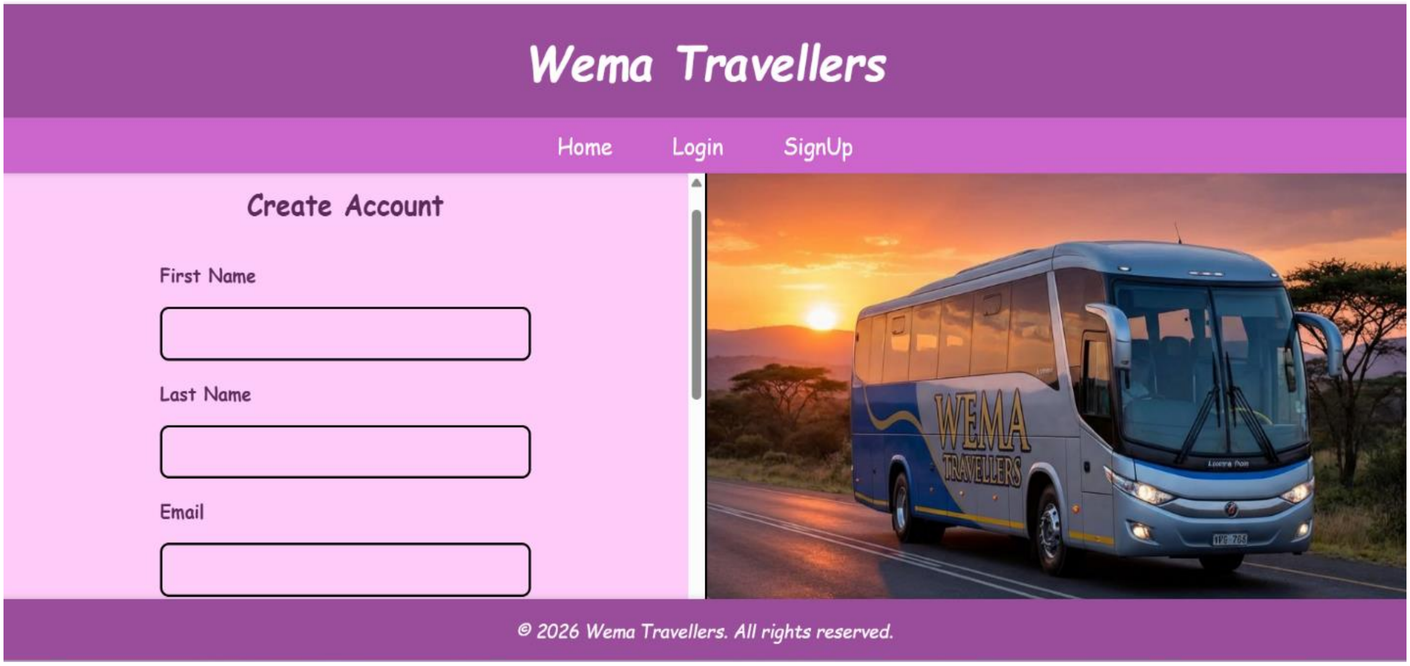
// footer: The variable containing the HTML code we just defined above.

document.body.insertAdjacentHTML('beforeend', footer);

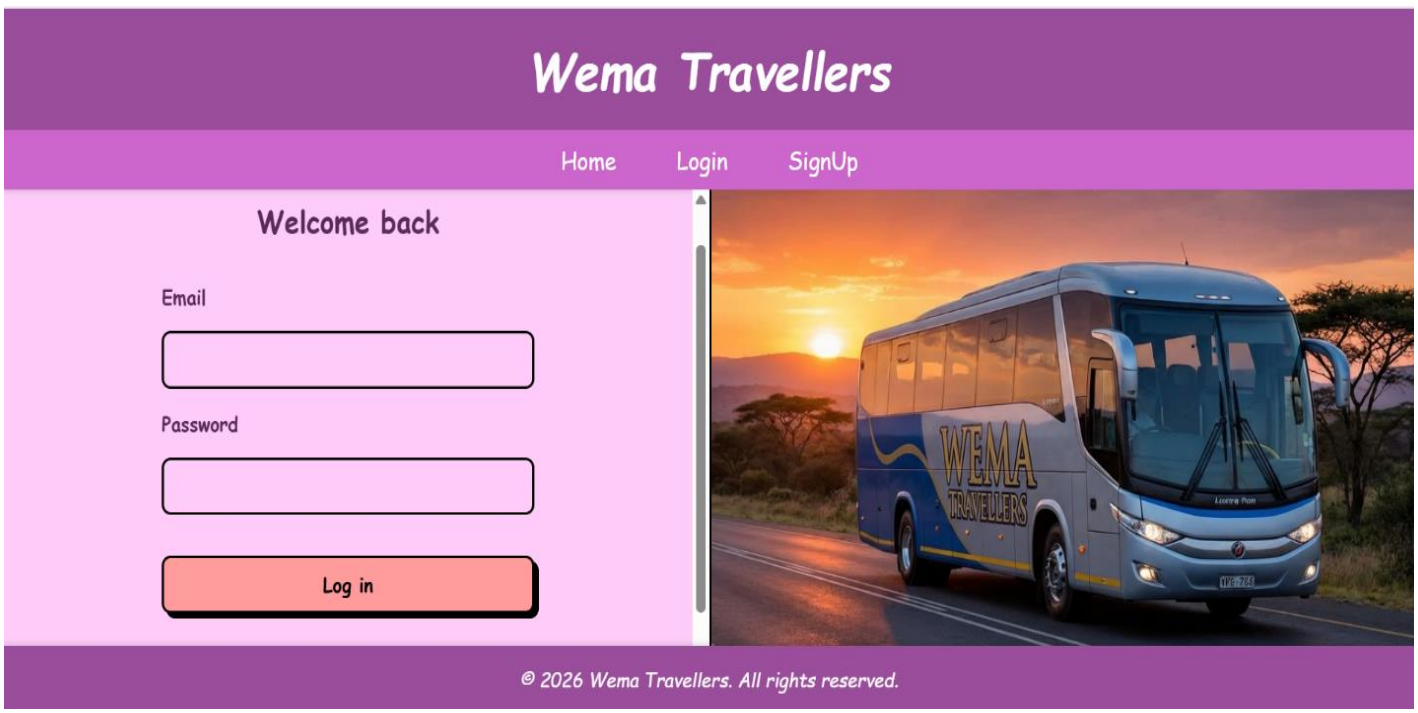
}); // Closes the event listener function.

SIGN UP PAGE

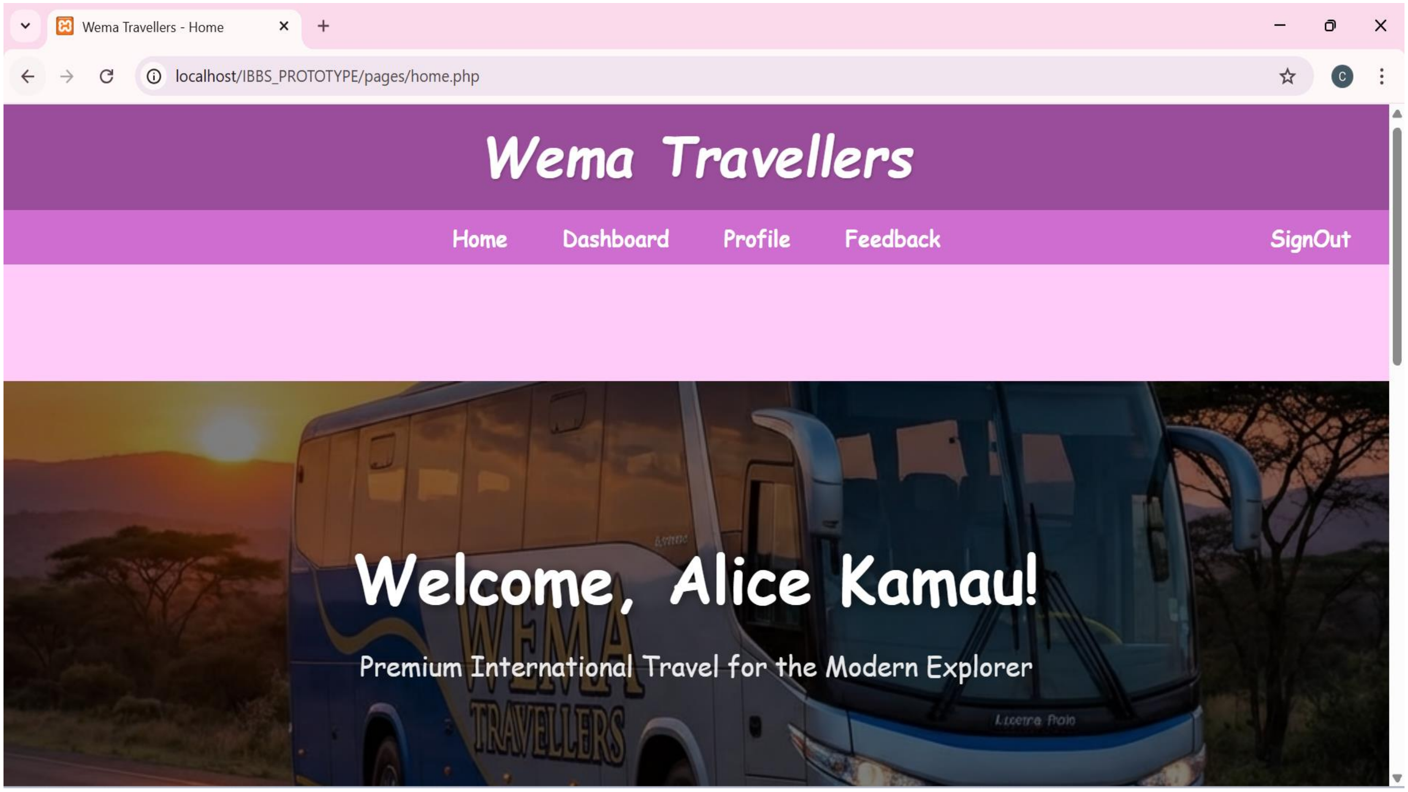


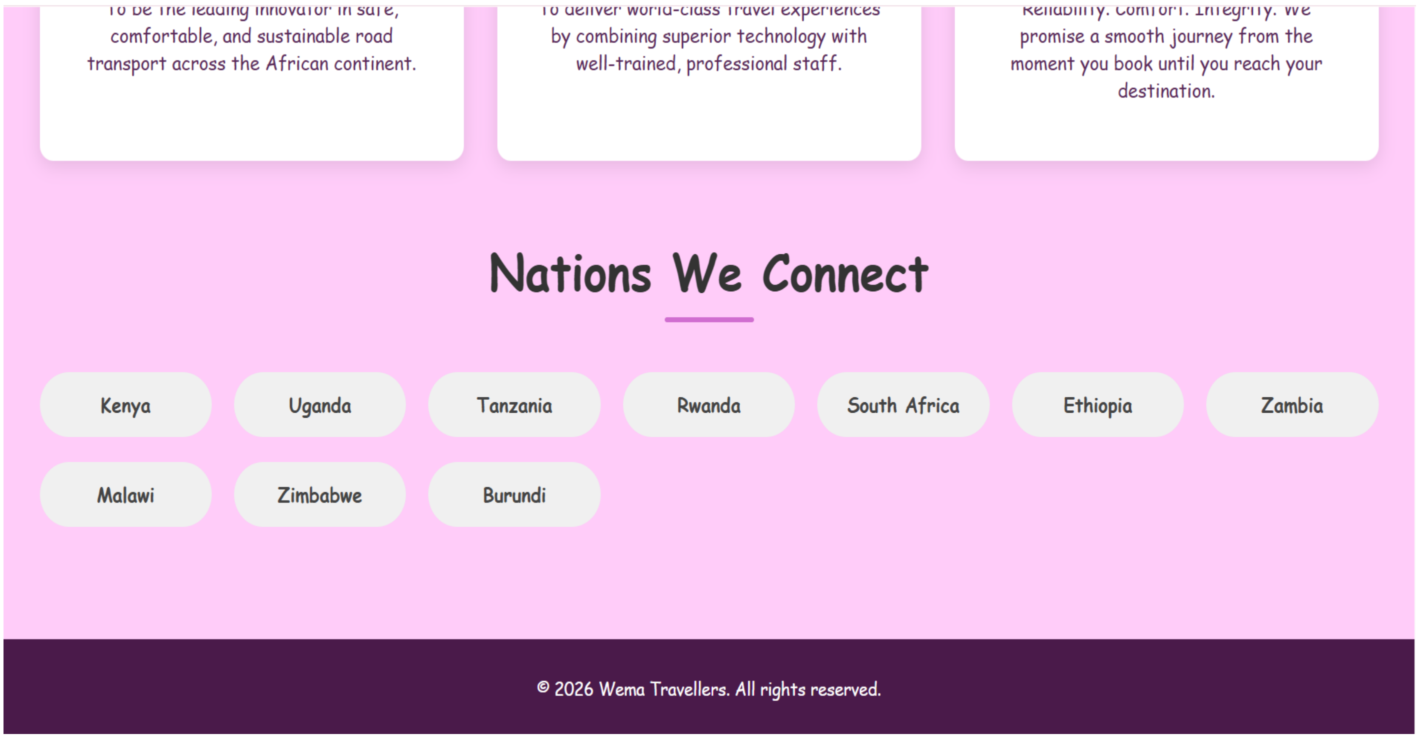


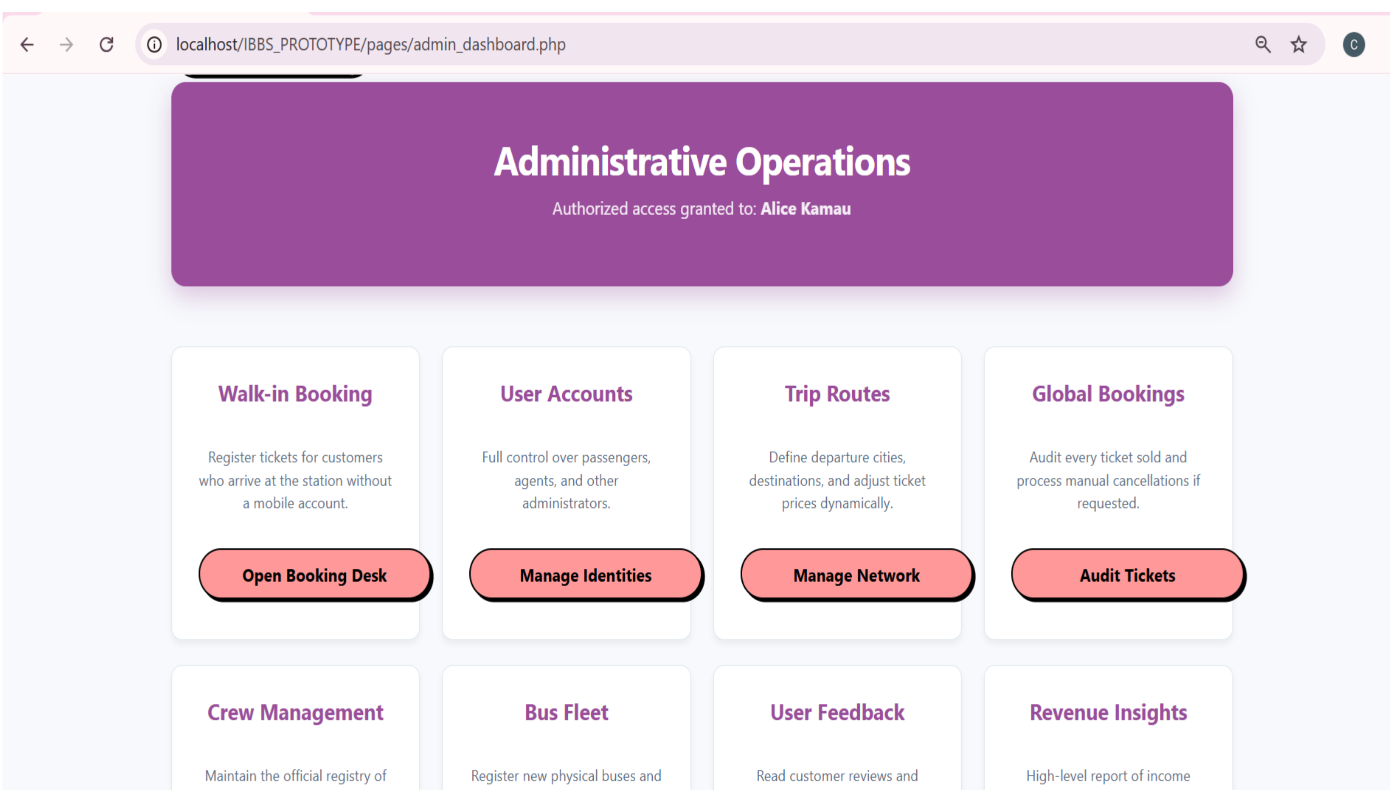
LOGIN FORM SNAPSHOT

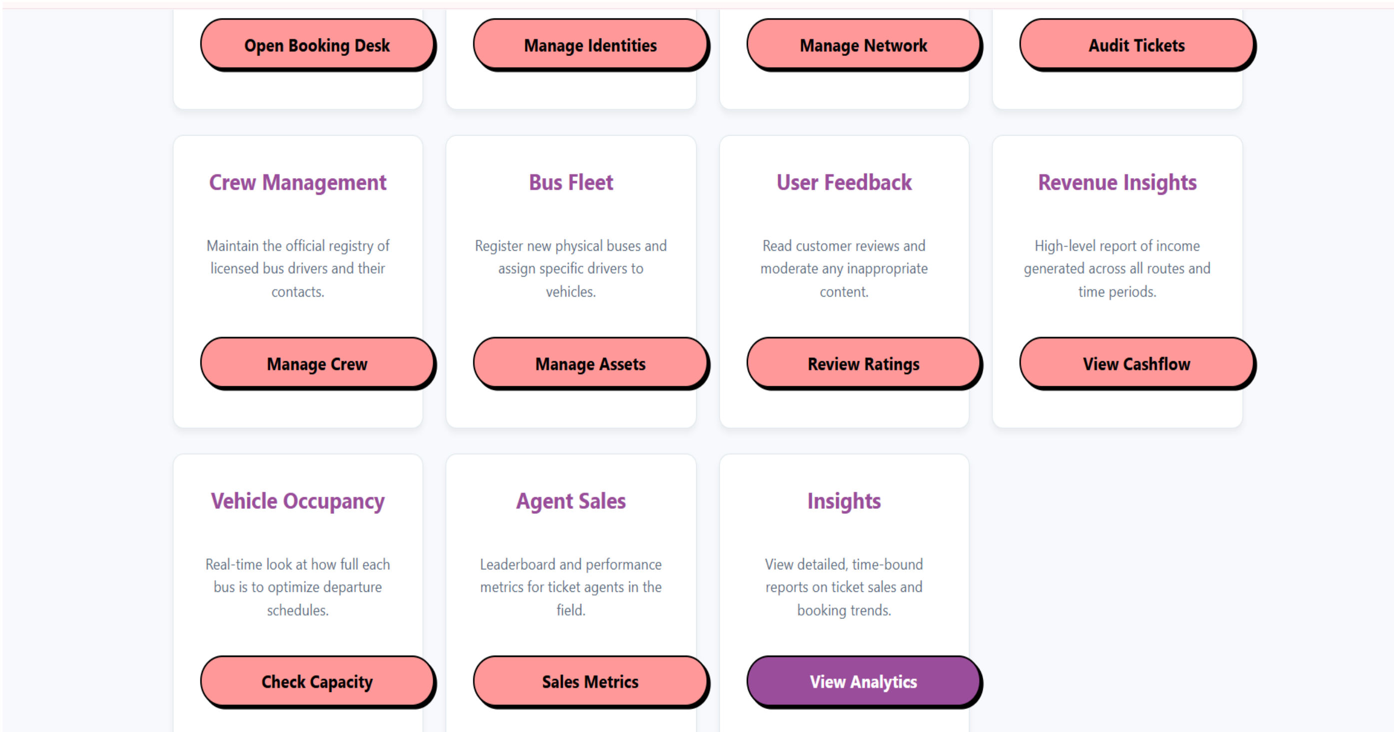


ADMINISTRATOR DASHBOARD

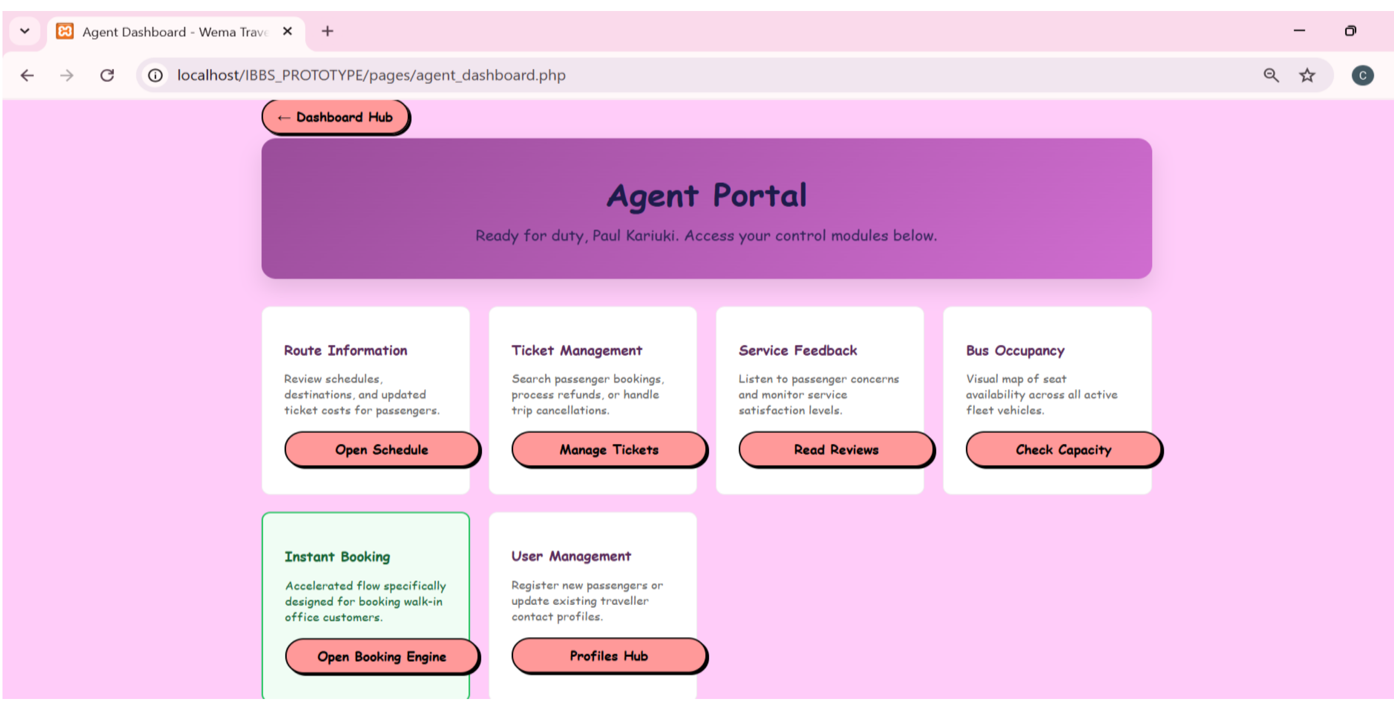



comfortable, and sustainable road transport across the African continent.
by combining superior technology with well-trained, professional staff.
promise a smooth journey from the moment you book until you reach your destination.
Nations We Connect
Kenya
Uganda
Tanzania
Rwanda
South Africa
Ethiopia
Zambia
Malawi
Zimbabwe
Burundi
© 2026 Wema Travellers. All rights reserved.


localhost/IBBS_PROTOTYPE/pages/admin_dashboard.php
Administrative Operations
Authorized access granted to: Alice Kamau
Walk-in Booking
Register tickets for customers who arrive at the station without a mobile account.
Open Booking Desk
User Accounts
Full control over passengers, agents, and other administrators.
Manage Identities
Trip Routes
Define departure cities, destinations, and adjust ticket prices dynamically.
Manage Network
Global Bookings
Audit every ticket sold and process manual cancellations if requested.
Audit Tickets
Crew Management
Maintain the official registry of
Bus Fleet
Register new physical buses and
User Feedback
Read customer reviews and
Revenue Insights
High-level report of income



AGENT DASHBOARD



C. IMPLIMENTING THE FUNCTIONAL DESIGN

- ✓ **process_booking.php**: Implements the **Transaction Logic**. This code handles seat validation, QR token generation, and the automated "payment" flow.

```php
<?php
// ====================================================================
// BOOKING PROCESSING ENGINE (process_booking.php)
// ====================================================================
// This script runs in the background (via AJAX fetch) to handle ticket bookings.
// It receives data from 'book.php', validates it, and saves it to the database.
// It returns a JSON response indicating success or failure.
// ====================================================================

// Include the database connection so we can talk to MySQL.
require_once 'db_connection.php';

// Start the session to identify the currently logged-in user.
session_start();

// Tell the browser that the output of this script is strictly JSON data.
// This ensures the JavaScript on the frontend parses the response correctly.
header('Content-Type: application/json');

// --- SECURITY CHECK ---
// We verify if the user is logged in by checking the session variable.
if (!isset($_SESSION['user_id'])) {
```

```php
    // If not logged in, return a failure message.
    echo json_encode(['success' => false, 'message' => 'Not authenticated']);
    // Stop the script immediately.
    exit();
}

// --- RECEIVE INPUT DATA ---
// We read the raw data sent in the request body (because it's JSON, not a standard form POST).
$raw_input = file_get_contents("php://input");
// We decode the JSON string into a PHP associative array.
$data = json_decode($raw_input, true);

// Extract specific pieces of information from the decoded data.
// We use the '??' operator (null coalescing) to provide defaults if data is missing.
$route_id = $data['route_id'] ?? null;              // The ID of the trip being booked
$target_user_id = $data['target_user_id'] ?? $_SESSION['user_id']; // ID of user who gets the ticket
$bookings_to_process = $data['bookings'] ?? [];     // List of seats and passenger details

// --- VALIDATION ---
// Check if we have the essential information needed to proceed.
if (!$route_id || empty($bookings_to_process)) {
    // If route ID is missing or the list of bookings is empty...
    echo json_encode(['success' => false, 'message' => 'Missing booking selection']);
    exit();
}
```

```
// --- FETCH TRIP DETAILS ---
// We need to know which bus is used for this route to get its Bus ID.
// We Prepare a SQL statement to prevent SQL Injection attacks.
$stmt = $conn->prepare("SELECT r.*, b.bus_id, b.max_passengers
                        FROM routes r
                        JOIN buses b ON r.bus_id = b.bus_id
                        WHERE r.route_id = ?");
// Bind the route_id parameter (integer type 'i').
$stmt->bind_param("i", $route_id);
// Execute the query.
$stmt->execute();
// Get the result and fetch it as an associative array.
$route = $stmt->get_result()->fetch_assoc();
// Close the statement to free up resources.
$stmt->close();

// If the route ID did not match any trip in the database...
if (!$route) {
    echo json_encode(['success' => false, 'message' => 'Route not found']);
    exit();
}

// --- NEW PRIVILEGE CHECK: PREVENT DOUBLE BOOKING ---
// We check if this passenger (target_user_id) already has an active booking for this specific trip.
// This prevents users from hoarding seats or booking the same trip multiple times by mistake.
$stmt_check = $conn->prepare("SELECT booking_id FROM bookings WHERE user_id = ?
AND route_id = ? AND booking_status != 'CANCELLED'");
```

```php
$stmt_check->bind_param("ii", $target_user_id, $route_id);
$stmt_check->execute();
$existing_res = $stmt_check->get_result();
if ($existing_res->num_rows > 0) {
    echo json_encode(['success' => false, 'message' => 'This passenger already has an active booking for this route. To book again, the previous ticket must be cancelled.']);
    $stmt_check->close();
    exit();
}
$stmt_check->close();

// --- START TRANSACTION ---
// A transaction ensures that either ALL seats are booked successfully, or NONE are.
// This prevents "partial bookings" if an error occurs halfway through.
$conn->begin_transaction();

try {
    // Loop through each seat requested by the user.
    foreach ($bookings_to_process as $b) {
        // Extract details for this specific seat.
        $seat_no = $b['seat_number'];
        $p_name = $b['passenger_name']; // Passenger Name
        $p_age = $b['passenger_age'];   // Passenger Age
        $p_id = $b['passenger_id_number']; // Passenger ID

        // --- STEP A: CHECK FOR DOUBLE BOOKING ---
        // Check if this specific seat was taken by someone else just seconds ago.
```

```
// We look for any active booking (not cancelled) for this route and seat.
$stmt = $conn->prepare("SELECT booking_id FROM bookings WHERE route_id = ? AND seat_number = ? AND booking_status != 'CANCELLED'");
$stmt->bind_param("is", $route_id, $seat_no);
$stmt->execute();

// If the query returns any rows, it means the seat is occupied.
if ($stmt->get_result()->num_rows > 0) {
   // We throw an 'Exception' to jump straight to the 'catch' block.
   throw new Exception("Seat $seat_no is already taken.");
}
$stmt->close();

// --- STEP B: GENERATE DIGITAL TOKEN ---
// ------------------------------------------------------------------------------------
// DETAILED TOKEN GENERATION EXPLANATION (Line 137):
// 1. random_bytes(16):
//    This function generates 16 bytes (128 bits) of cryptographically secure pseudo-random bytes.
//    It pulls "entropy" (unpredictable data) from the operating system's kernel (like /dev/urandom on Linux
//    or the CryptGenRandom API on Windows). This makes the output statistically unique and impossible
//     for an attacker to predict or reverse-engineer.
//
// 2. bin2hex(...):
//    The 16 bytes generated are raw binary data (not readable by humans).
//    bin2hex() converts each byte into its two-character hexadecimal representation (0-9, a-f).
```

```
//    Result: A 32-character unique string (e.g., "7f3e8a2b...") that serves as the ticket's "Hash" or
//    Digital Fingerprint. This token is stored in the database and used to generate the QR code.
//
// WHY NOT USE md5() or sha1()?
//    Simple hashes of predictable data (like user_id + time) can be "brute-forced."
//    By using random_bytes, we ensure that even if two people book at the exact same microsecond,
//    their tokens will be completely different.
// --------------------------------------------------------------------------------
// ENCRYPTION / SECURITY EXPLANATION:
// We use the function 'random_bytes(16)' to generate 16 bytes of cryptographically secure random data.
// Unlike 'rand()', which is predictable, 'random_bytes()' uses the operating system's entropy source.
// This makes it impossible for a hacker to guess the next token.
// 'bin2hex()' converts these raw binary bytes into a readable 32-character hexadecimal string.
// This string (e.g., "a3f9...") becomes the unique digital fingerprint for this ticket.
// --------------------------------------------------------------------------------
$qr_token = bin2hex(random_bytes(16));

// Get the current date and time for the booking record.
$booking_time = date('Y-m-d H:i:s');

// Set the status. Since this is a prototype, we assume instant payment verification.
$status = 'PAID';
$bus_id = $route['bus_id'];
```

```
        // --- STEP C: INSERT BOOKING RECORD ---
        // We construct the SQL command to save all the data into the 'bookings' table.
        $sql = "INSERT INTO bookings (booking_time, seat_number, booking_status, qr_token,
user_id, route_id, bus_id, passenger_name, passenger_age, passenger_id_number)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";

        // Prepare the statement.
        $stmt = $conn->prepare($sql);

        // Bind all 10 parameters.
        // Types: s=string, i=integer.
        // Ordering matches the '?' placeholders exactly.
        $stmt->bind_param("ssssiiisis", $booking_time, $seat_no, $status, $qr_token,
$target_user_id, $route_id, $bus_id, $p_name, $p_age, $p_id);

        // Execute the insertion. If it fails (returns false), we throw an error.
        if (!$stmt->execute()) {
            throw new Exception("Database error: " . $stmt->error);
        }
        // Close the statement for this iteration.
        $stmt->close();
    }

    // --- COMMIT TRANSACTION ---
    // If the loop finishes without throwing any exceptions, it means all seats are valid.
    // We 'permit' the changes to be permanently saved to the database.
    $conn->commit();
```

```
    // Determine where to redirect the user.
    // If the admin booked it ($target_user_id != session ID), stay on admin page.
    $is_self = ($target_user_id == $_SESSION['user_id']);

    // Send back a success JSON response.
    echo json_encode([
        'success' => true,
        'message' => 'Booking successfully confirmed for ' . count($bookings_to_process) . '
seat(s)!',
        'redirect' => $is_self ? 'view_user_history.php' : 'view_admin_bookings.php'
    ]);

} catch (Exception $e) {
    // --- ROLLBACK TRANSACTION ---
    // If ANY error occurred (like a taken seat or DB failure), we jump here.
    // 'rollback()' undoes ANY changes made during this transaction block.
    // This ensures we don't end up with one booked seat and one failed seat.
    $conn->rollback();

    // Return a failure JSON response with the error message.
    echo json_encode(['success' => false, 'message' => $e->getMessage()]);
}

// Close the main database connection.
$conn->close();
?>
```

✓ **admin_insights.php & the report_*.php files**: These implement **Analytical Logic**, using complex SQL JOINs to transform raw database rows into readable business reports.

```php
<?php
/**
 * ADMIN INSIGHTS HUB (admin_insights.php)
 * Purpose: This page acts as the central dashboard for all time-bound reports.
 * It allows Administrators to jump to specific data views (Weekly, Monthly, Yearly).
 */

// --- SESSION START ---
// We start the session to verify who is visiting.
session_start();

// --- SECURITY CHECK ---
// Only 'ADMIN' users are allowed here. If an AGENT or PASSENGER tries to access,
// they are sent back to the login page.
if (!isset($_SESSION['role']) || $_SESSION['role'] !== 'ADMIN') {
    header("Location: login.html");
    exit();
}
?>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Business Insights - Wema Travellers</title>
  <!-- Responsive viewport for mobile devices -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <!-- CSS Links: Reusing existing styles for consistency -->
  <link rel="stylesheet" href="css/style.css">
  <link rel="stylesheet" href="css/main.css">

  <style>
    /* Shared Styles for the Insights Hub */
    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background-color: #f4f7f6;
      margin: 0;
      padding: 0;
    }

    .container {
      width: 90%;
      max-width: 1000px;
      margin: 40px auto;
      background: white;
      padding: 30px;
```

```css
    border-radius: 12px;
    box-shadow: 0 4px 15px rgba(0,0,0,0.1);
    text-align: center;
}

.back-btn-container {
    margin-bottom: 20px;
    text-align: left;
}

/* The Insight Hub's inner title styling */
.container h1 {
    color: var(--purple);
    margin-bottom: 10px;
}

p.subtitle {
    color: #666;
    margin-bottom: 40px;
}

/* Grid for the report buttons */
.report-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 20px;
}
```

```
        /* Individual report link buttons */
        .report-link {
            display: block;
            padding: 30px 20px;
            background: #ffffff;
            border: 2px solid #e1e8ed;
            border-radius: 10px;
            color: var(--purple);
            text-decoration: none;
            font-weight: 700;
            font-size: 1.1rem;
            transition: all 0.3s ease;
        }

        .report-link:hover {
            background: var(--purple);
            color: white;
            border-color: var(--purple);
            transform: translateY(-5px);
            box-shadow: 0 5px 15px rgba(106, 17, 203, 0.2);
        }
    </style>
</head>
<body>
    <!-- Inject Global Header/Navbar -->
    <script src="js/header2.js"></script>
```

```
    <!-- Spacer for the sticky header -->
    <div style="height: 100px;"></div>


    <div class="container">
        <!-- Back navigation link: Styled as a fully rounded purple button for consistency -->
        <div class="back-btn-container">
            <a href="admin_dashboard.php" class="button regular-button" style="text-decoration:none; background-color: var(--purple); color: white; border-radius: 50px; display: inline-block; width: auto; padding: 10px 30px;">← Admin Dashboard</a>
        </div>


        <h1>Business Insights</h1>
        <p class="subtitle">Select a report period to view booking data and performance metrics.</p>


        <!-- The Report Selection Grid -->
        <div class="report-grid">
            <!-- Button for This Week's Data -->
            <a href="report_this_week.php" class="report-link">This Week's Bookings</a>


            <!-- Button for This Month's Data -->
            <a href="report_this_month.php" class="report-link">This Month's Bookings</a>


            <!-- Button for Last Month's Data -->
            <a href="report_last_month.php" class="report-link">Last Month's Bookings</a>


            <!-- Button for This Year's Data -->
            <a href="report_this_year.php" class="report-link">This Year's Bookings</a>
```

```html
        </div>
    </div>

    <!-- Spacer for the footer -->
    <div style="height: 100px;"></div>
    <!-- Inject Global Footer -->
    <script src="js/footer.js"></script>
</body>
</html>
```

- ✓ **book.php:** Implements the **Sales Workflow**, coordinating the interaction between route selection, user input, and reservation.

```php
<?php
/**
 * CORE BOOKING ENGINE (book.php)
 * ==============================================================
 * Purpose: This is the flagship interface of the IBBS Prototype.
 * It allows Passengers to find trips and Agents to book for walk-ins.
 *
 * Technical Highlights:
 * - Real-time seat availability logic.
 * - Dynamic Seat Map injection.
 * - Multi-passenger data capture (Name, Age, ID).
```

```php
 * - AJAX-based submission to prevent page reloads.
 * ===========================================================================
 */

// 1. DATA BRIDGE: Connect to the MySQL database by including the header file.
require_once 'db_connection.php';

// 2. IDENTITY: Start the PHP session mechanism to track the current logged-in user's state.
session_start();

/**
 * --- SECURITY ACCESS CONTROL ---
 * Booking is a privileged action. We check if the user is logged in.
 * If the 'user_id' index is missing from the $_SESSION array, they are blocked.
 */
if (!isset($_SESSION['user_id'])) {
    header("Location: login.html"); // Force redirect to the login portal.
    exit(); // Prevent any further HTML from loading.
}

/**
 * --- USER CONTEXT ---
 * Retrieve the specific identity and role of the person accessing the page.
 * We use this to toggle between "Passenger Mode" and "Staff/Agent Mode".
 */
$user_id = $_SESSION['user_id']; // The unique database ID of the user.
$role    = $_SESSION['role'];    // The designation (PASSENGER, AGENT, or ADMIN).
```

```
// $is_staff: A boolean (True/False) that checks if the role matches internal staff types.
$is_staff = ($role === 'ADMIN' || $role === 'AGENT');



/**
 * --- QUERY: AVAILABLE TRIPS ---
 * We write a complex SQL query to find all valid journeys.
 * Logic:
 * - Select all columns (*) from the 'routes' table (r).
 * - JOIN with 'buses' (b) to retrieve seat capacity (max_passengers).
 * - Use a correlated subquery to dynamically COUNT active bookings for each route.
 * - FILTER: Only show trips that haven't happened yet (>= CURDATE()).
 * - FILTER: Hide trips that this specific user has already booked to avoid double-booking.
 * - SORT: Kenya routes first, then Tanzania, then others, then by date.
 */
$sql_available = "
   SELECT r.*, b.bus_name, b.max_passengers,
          (SELECT COUNT(*) FROM bookings WHERE route_id = r.route_id AND
booking_status != 'CANCELLED') as booked_seats
   FROM routes r
   JOIN buses b ON r.bus_id = b.bus_id
   WHERE r.departure_date >= CURDATE()
   AND r.route_id NOT IN (
      SELECT route_id FROM bookings WHERE user_id = $user_id AND booking_status !=
'CANCELLED'
   )
   ORDER BY
      CASE
```

```
            WHEN r.from_location LIKE '%Kenya%' THEN 1
            WHEN r.from_location LIKE '%Tanzania%' THEN 2
            ELSE 3
        END ASC,
        r.departure_date ASC
";
// $result_available: Executes the query and stores the rows in memory.
$result_available = $conn->query($sql_available);


/**
 * --- DATA: PASSENGER REGISTRY (Staff Only) ---
 * If an AGENT is booking, they need to select WHICH customer they are booking for.
 * We fetch all users who have the 'PASSENGER' role.
 */
$passengers = []; // Initialize an empty container.
if ($is_staff) {
    // Run a query to get names and emails for the dropdown.
    $pass_res = $conn->query("SELECT user_id, first_name, last_name, email FROM users
WHERE role = 'PASSENGER' ORDER BY first_name");
    // Loop through results and store them in the $passengers array.
    while($p = $pass_res->fetch_assoc()) $passengers[] = $p;
}
?>

<!DOCTYPE html>
<html lang="en">
```

```html
<head>
    <meta charset="UTF-8">
    <title>Book your Journey - Wema Travellers</title>

    <!-- Branding & Design System -->
    <link rel="stylesheet" href="css/style.css">
    <link rel="stylesheet" href="css/main.css">

    <style>
        /* UI DESIGN: Core layout for the booking engine */
        .booking-container {
            max-width: 1200px;
            margin: 40px auto;
            padding: 30px;
            background: white;
            border-radius: 15px;
            box-shadow: 0 10px 30px rgba(0,0,0,0.05);
        }

        /* Staff Controls: Blue highlight for the Admin-only section */
        .staff-panel {
            background: #f0f4ff;
            border: 2px solid var(--purple);
            padding: 25px;
            border-radius: 12px;
            margin-bottom: 40px;
        }
```

```
.staff-panel label {
    display: block;
    font-weight: 700;
    color: #1e1b4b;
    margin-bottom: 12px;
    font-size: 0.9em;
    text-transform: uppercase;
    letter-spacing: 0.05em;
}

.passenger-select {
    width: 100%;
    max-width: 500px;
    padding: 12px;
    border: 1px solid #cbd5e1;
    border-radius: 8px;
    font-size: 1.1em;
    background: white;
}

/* Grid Table for Route Listing */
.crud-table {
    width: 100%;
    border-collapse: collapse;
}
.crud-table th, .crud-table td {
```

```css
    padding: 18px;
    border-bottom: 1px solid #f1f5f9;
    text-align: left;
}
.crud-table th {
    background-color: #f8fafc;
    color: #64748b;
    font-size: 0.75rem;
    text-transform: uppercase;
    letter-spacing: 0.1em;
}

/* -----------------------------------------------------------
   SEAT MAP MODAL: The Interactive Seating UI
   ----------------------------------------------------------- */

/* Overlay: Darkens the background */
#seat-modal {
    display: none;
    position: fixed;
    z-index: 9999;
    left: 0;
    top: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(15, 23, 42, 0.75);
    backdrop-filter: blur(8px);
```

```css
    overflow-y: auto;
}

/* Modal Body */
.seat-content {
    background-color: #ffffff;
    margin: 3% auto;
    padding: 40px;
    border-radius: 20px;
    width: 90%;
    max-width: 850px;
    box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.25);
}

/* The Virtual Bus: CSS Grid of 5 columns (Standard Bus Layout) */
.bus-layout {
    display: grid;
    grid-template-columns: repeat(5, 1fr);
    gap: 12px;
    background: #f1f5f9;
    padding: 25px;
    border-radius: 18px;
    border: 1px solid #e2e8f0;
    margin: 30px auto;
    max-width: 450px;
}
```

```
/* Individual Seat Interaction */
.seat {
    aspect-ratio: 1;
    display: flex;
    align-items: center;
    justify-content: center;
    border-radius: 10px;
    font-size: 0.85em;
    font-weight: 700;
    cursor: pointer;
    transition: all 0.2s cubic-bezier(0.4, 0, 0.2, 1);
    border: 1px solid #cbd5e1;
    background: white;
    color: #475569;
}

/* Seat States */
/* Available: Clean White */
.seat.available:hover {
    border-color: #22c55e;
    background-color: #f0fdf4;
    transform: scale(1.05);
}
/* Occupied: Solid Red (Blocked) */
.seat.occupied {
    background-color: #ef4444;
    color: white;
```

```css
    border-color: #dc2626;
    cursor: not-allowed;
    box-shadow: inset 0 2px 4px rgba(0,0,0,0.1);
}
/* Selected: Vibrant Green (Active) */
.seat.selected {
    background-color: #22c55e;
    color: white;
    border-color: #15803d;
    box-shadow: 0 0 15px rgba(34, 197, 94, 0.4);
}

/* PASSENGER DATA FORMS: Generated dynamically based on selected seats */
.passenger-details-section {
    text-align: left;
    margin-top: 40px;
    padding-top: 30px;
    border-top: 1px solid #e2e8f0;
    display: none;
}

.passenger-info-card {
    background: #f8fafc;
    padding: 20px;
    border-radius: 12px;
    margin-bottom: 20px;
    border-left: 6px solid var(--purple);
```

```
}
.passenger-info-card h4 { margin: 0 0 15px 0; color: #1e293b; font-size: 1em; }

.info-grid {
   display: grid;
   grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
   gap: 20px;
}
.info-group label { display: block; font-size: 0.8em; font-weight: 600; color: #64748b;
margin-bottom: 8px; }
.info-group input {
   width: 100%;
   padding: 10px;
   border: 1px solid #cbd5e1;
   border-radius: 6px;
   background: white;
   font-size: 0.95em;
}

/* UI Legend: Helping users understand colors */
.legend {
   display: flex;
   justify-content: center;
   gap: 25px;
   margin: 25px 0;
   font-size: 0.85em;
   color: #64748b;
```

```
        }
        .legend-item { display: flex; align-items: center; gap: 8px; }
        .box { width: 18px; height: 18px; border-radius: 4px; border: 1px solid #e2e8f0; }
    </style>
</head>
<body>
    <!-- Standard Navigation -->
    <script src="js/header2.js"></script>

    <!-- Buffer for the sticky header -->
    <div style="height: 100px;"></div>

    <div class="booking-container"> <!-- Main visual wrapper for the booking software. -->
        <h2 style="color: var(--purple); margin-bottom: 30px; font-weight: 800;">🎫 Trip
Reservation Engine</h2> <!-- Main Page Title. -->

        <!-- STAFF CONTROLS (Only visible if the logged-in user is an ADMIN or AGENT) -->
        <?php if ($is_staff): ?>
        <div class="staff-panel"> <!-- Styles this area with a light blue background. -->
            <label for="target_user_id">Booking Representative Control</label> <!-- Label for
instructions. -->
            <select id="target_user_id" class="passenger-select"> <!-- Dropdown to pick the
customer. -->
                <!-- Default option: The Agent books for themselves. -->
                <option value="<?= $user_id ?>">Agent Action: (<?=
htmlspecialchars($_SESSION['name']) ?>)</option>
                <!-- Grouped list of all registered passengers in the system. -->
                <optgroup label="Select Authorized Passenger">
```

```
            <?php foreach($passengers as $p): ?> <!-- Loop through the $passengers array created in PHP above. -->
                <option value="<?= $p['user_id'] ?>">
                    <?= htmlspecialchars($p['first_name'] . ' ' . $p['last_name']) ?> | <?= htmlspecialchars($p['email']) ?>
                </option>
            <?php endforeach; ?>
        </optgroup>
    </select>
    <p style="margin-top: 15px; font-size: 0.85em; color: #475569;">
        <!-- Link to manual registration if the customer isn't in the list yet. -->
        Need a new account? <a href="view_users_sorted.php" style="color:var(--purple); font-weight:700;">Create Passenger Profile →</a>
    </p>
</div>
<?php else: ?>
    <!-- Regular Passenger Logic: We hide the selection and force the user_id to be their own. -->
    <input type="hidden" id="target_user_id" value="<?= $user_id ?>">
<?php endif; ?>


<!-- ACTIVE ROUTES: The visual table schedule -->
<div class="table-container">
    <table class="crud-table"> <!-- Uses the standardized project table styles. -->
        <thead>
            <tr>
                <th>Destination</th> <!-- Origin and Destination cities. -->
                <th>Departure Schedule</th> <!-- Date and Time. -->
```

```
            <th>Assigned Vehicle</th> <!-- The bus name. -->
            <th>Cost (KES)</th> <!-- Ticket price. -->
            <th>Availability</th> <!-- Seats free. -->
            <th>Action</th> <!-- The 'Reserve' button. -->
          </tr>
        </thead>
        <tbody>
          <?php
          // Render each active route row from the $result_available database object.
          $result_available->data_seek(0); // Reset pointer to the start of the results.
          while ($row = $result_available->fetch_assoc()): // Fetch one row at a time.
            // Calculate remaining seats by subtracting bookings from max capacity.
            $remaining = $row['max_passengers'] - $row['booked_seats'];
            // $is_full: Boolean check to see if the bus is sold out.
            $is_full = ($remaining <= 0);
          ?>
          <tr>
            <!-- Location Pair -->
            <td>
              <div style="font-weight: 700; color: #1e293b;"><?=
htmlspecialchars($row['from_location']) ?></div> <!-- Origin city. -->
              <div style="font-size: 0.85em; color: #64748b;">to <?=
htmlspecialchars($row['to_location']) ?></div> <!-- Destination city. -->
            </td>
            <!-- Time Tracking -->
            <td>
              <div style="font-weight: 600;"><?= $row['departure_date'] ?></div> <!--
Calendar date. -->
```

```
            <div style="font-size: 0.85em; font-family: monospace; color: var(--purple);"><?= $row['departure_time'] ?></div> <!-- Clock time. -->
        </td>
        <!-- Bus Identity -->
        <td><?= htmlspecialchars($row['bus_name']) ?></td> <!-- Name of the bus vehicle. -->
        <!-- Fare Calculation -->
        <td style="font-weight: 700; color: #1e293b;"><?= number_format($row['cost'], 2) ?></td> <!-- Price formatted for currency. -->
        <!-- Stock Status -->
        <td>
            <?php if ($is_full): ?>
                <!-- Red pill showing the bus is full. -->
                <span style="background: #fee2e2; color: #b91c1c; padding: 6px 12px; border-radius: 99px; font-size: 0.75em; font-weight: 800;">BUS FULL</span>
            <?php else: ?>
                <!-- Green pill showing available seats count. -->
                <span style="background: #f0fdf4; color: #166534; padding: 6px 12px; border-radius: 99px; font-size: 0.75em; font-weight: 800;"><?= $remaining ?> SEATS OPEN</span>
            <?php endif; ?>
        </td>
        <!-- Entry Button -->
        <td>
            <?php if (!$is_full): ?>
                <!-- Active button that triggers the JS Seat Map logic. -->
                <button type="button" class="button regular-button pink-background" onclick="openSeatMap(<?= $row['route_id'] ?>, <?= $row['max_passengers'] ?>)">Reserve Seats</button>
            <?php else: ?>
```

```
                <!-- Faded button if no seats are available. -->
                <button disabled class="button regular-button" style="opacity:0.3;
cursor:not-allowed;">Sold Out</button>
              <?php endif; ?>
            </td>
          </tr>
          <?php endwhile; ?> <!-- End of trip loop. -->
        </tbody>
      </table>
    </div>
  </div>

  <!-- ---------------------------------------------------------------
       INTERFACE: INTERACTIVE SEATING MODAL
       --------------------------------------------------------------- -->
  <div id="seat-modal">
    <div class="seat-content">
      <h3 style="margin-top:0; font-size: 1.5rem; color: #0f172a;">Virtual Seating Deck</h3>
      <p style="color:#64748b; font-size:0.95em; margin-bottom: 20px;">Please click on your
preferred seats to begin the reservation.</p>

      <!-- GUIDANCE: Color coding legend -->
      <div class="legend">
        <div class="legend-item"><div class="box" style="background:#fff;"></div>
Vacant</div>
        <div class="legend-item"><div class="box" style="background:#ef4444; border-color:
#ef4444;"></div> Reserved</div>
        <div class="legend-item"><div class="box" style="background:#22c55e; border-color:
#22c55e;"></div> Your Selection</div>
```

```
            </div>

            <!-- THE BUS: This is where JS will draw the seats -->
            <div id="bus-layout" class="bus-layout">
                <!-- Dynamically Generated -->
            </div>

            <!-- PASSENGER IDENTITY: This appears as soon as 1 seat is clicked -->
            <div id="passenger-details-section" class="passenger-details-section">
                <h3 style="font-size: 1.25rem; margin-bottom: 25px;">Traveller Identification</h3>
                <div id="passenger-info-container">
                    <!-- Cards appear here -->
                </div>
            </div>

            <!-- MODAL CONTROL PANEL: Action buttons at the bottom -->
            <div style="margin-top: 40px; display: flex; gap: 15px; justify-content: flex-end;
position: sticky; bottom: -40px; background: #ffffff; padding: 25px 0; border-top: 1px solid
#f1f5f9;">
                <button class="button regular-button" onclick="closeSeatMap()"
style="background:#f1f5f9; color: #475569;">Cancel Action</button>
                <button id="confirm-booking-btn" class="button regular-button pink-background"
disabled onclick="submitBooking()">Finalize Booking</button>
            </div>
        </div>
    </div>

    <div style="height: 100px;"></div>
```

```
    <!-- Link the central script for the footer -->
    <script src="js/footer.js"></script>



    <!--
==========================================================================
        LOGIC: ENGINE JAVASCRIPT
        ==========================================================================
-->
    <script>
        // CLIENT STATE
        let currentRouteId = null;  // The Trip we are currently looking at
        let selectedSeats = [];     // List of seats currently highlighted green

        /**
         * ACTION: OPEN SEAT ENGINE
         * Loads fresh data for the specific trip and resets the modal.
         */
        function openSeatMap(route_id, max_passengers) {
            currentRouteId = route_id;
            selectedSeats = []; // Wipe selections from previous modal open

            // UI RESET
            updateBookingButton();
            document.body.style.overflow = 'hidden'; // Lock background scroll

            /**
```

```
     * FETCH OCCUPANCY:
     * We call a background PHP script (op_get_occupied_seats.php)
     * to find out which seats are already taken in the database.
     */
    fetch(`op_get_occupied_seats.php?route_id=${route_id}`)
        .then(res => res.json())
        .then(data => {
            const occupiedList = data.occupied || [];
            // TRIGGER DRAW: Build the visual grid
            generateLayout(max_passengers, occupiedList);

            // SHOW: Reveal the modal
            document.getElementById('seat-modal').style.display = 'block';
            document.getElementById('passenger-details-section').style.display = 'none';
            document.getElementById('passenger-info-container').innerHTML = '';
        });
}

/**
 * LOGIC: GRID GENERATOR
 * Creates a clickable squares for every seat in the bus fleet.
 */
function generateLayout(total, occupied) {
    const container = document.getElementById('bus-layout');
    container.innerHTML = ''; // Clean slate

    for (let i = 1; i <= total; i++) {
```

```
        const seatNo = `S${i}`; // Standardized ID (e.g., S24)
        const seatNode = document.createElement('div');

        // If the seat is in our 'occupied' list from the server
        if (occupied.includes(seatNo)) {
            seatNode.className = 'seat occupied';
            seatNode.innerText = seatNo;
        } else {
            // Seat is available
            seatNode.className = 'seat available';
            seatNode.innerText = seatNo;
            // Connect the interactive logic
            seatNode.onclick = () => toggleSeatSelection(seatNode, seatNo);
        }
        container.appendChild(seatNode);
    }
}

/**
 * ACTION: TOGGLE SELECTION
 * Logic to add or remove a seat from the user's "shopping cart".
 */
function toggleSeatSelection(element, seatNo) {
    if (selectedSeats.includes(seatNo)) {
        // Remove if already there
        selectedSeats = selectedSeats.filter(s => s !== seatNo);
        element.classList.replace('selected', 'available');
```

```
    } else {
        // Add to selection
        selectedSeats.push(seatNo);
        element.classList.replace('available', 'selected');
    }

    // SIDE EFFECTS:
    updatePassengerDataForms(); // Update the Name/ID inputs below
    updateBookingButton();      // Enable/Disable the confirm button
}

/**
 * UI: DYNAMIC PASSENGER FORMS
 * Generates one data card (Name, Age, ID) for every green seat.
 * Crucially: It saves what the user already typed before rebuilding.
 */
function updatePassengerDataForms() {
    const container = document.getElementById('passenger-info-container');
    const section = document.getElementById('passenger-details-section');

    // Clean up if nothing is selected
    if (selectedSeats.length === 0) {
        section.style.display = 'none';
        return;
    }

    section.style.display = 'block';
```

```
// 1. MEMORY: Save existing text input values before we wipe the container
const draftData = {};
container.querySelectorAll('.passenger-info-card').forEach(card => {
    const s = card.dataset.seat;
    draftData[s] = {
        name: card.querySelector('.p-name').value,
        age: card.querySelector('.p-age').value,
        id: card.querySelector('.p-id').value
    };
});

// 2. CLEAR: Wipe current inputs
container.innerHTML = '';

// 3. REBUILD: Create a fresh form for every selected seat
// We sort selections (S1, S2...) for a clean layout
selectedSeats.sort((a,b) => parseInt(a.substring(1)) -
parseInt(b.substring(1))).forEach(seatKey => {
    const card = document.createElement('div');
    card.className = 'passenger-info-card';
    card.dataset.seat = seatKey;

    // Restore values if we saved them in Step 1
    const v = draftData[seatKey] || { name: '', age: '', id: '' };

    card.innerHTML = `
```

```
                    <h4>Reservation: Seat ${seatKey}</h4>
                    <div class="info-grid">
                        <div class="info-group">
                            <label>Traveller Name</label>
                            <input type="text" class="p-name" value="${v.name}" placeholder="Full Legal Name" required>
                        </div>
                        <div class="info-group">
                            <label>Age</label>
                            <input type="number" class="p-age" value="${v.age}" placeholder="e.g. 25" required>
                        </div>
                        <div class="info-group">
                            <label>ID / Identity Number</label>
                            <input type="text" class="p-id" value="${v.id}" placeholder="ID or Passport" required>
                        </div>
                    </div>
                `;
                container.appendChild(card);
            });
        }

        // CONTROL: Only allow "Confirm" if seats are picked
        function updateBookingButton() {
            document.getElementById('confirm-booking-btn').disabled = (selectedSeats.length === 0);
        }
```

```
// ACTION: Close the overlay and restore system scroll
function closeSeatMap() {
    document.getElementById('seat-modal').style.display = 'none';
    document.body.style.overflow = 'auto';
}

/**
 * THE BIG ACTION: SUBMIT BOOKING
 * Collects all seat/passenger data and sends it to 'process_booking.php'
 * using a JSON POST request.
 */
function submitBooking() {
    const system_user_id = document.getElementById('target_user_id').value;
    const payloadArray = [];

    let allFieldsValid = true;

    // Loop through all generated forms to validate and capture
    document.querySelectorAll('.passenger-info-card').forEach(card => {
        const p_name = card.querySelector('.p-name').value.trim();
        const p_age  = card.querySelector('.p-age').value.trim();
        const p_id   = card.querySelector('.p-id').value.trim();
        if (!p_name || !p_age || !p_id) allFieldsValid = false;

        payloadArray.push({
            seat_number: card.dataset.seat,
```

```
        passenger_name: p_name,
        passenger_age: p_age,
        passenger_id_number: p_id
    });
});

// Check for empty fields
if (!allFieldsValid) {
    alert('Incomplete Data: Please provide details for all selected seats.');
    return;
}
// User Confirmation
if (!confirm(`Commit reservation for ${selectedSeats.length} passenger(s)?`)) return;
/**
 * EXECUTE AJAX:
 * This allows us to book the ticket WITHOUT the page refreshing.
 */
fetch('process_booking.php', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
        route_id: currentRouteId,
        target_user_id: system_user_id,
        bookings: payloadArray
    })
})
.then(res => res.json())
```

```
            .then(result => {
                if (result.success) {
                    // SUCCESS: Route the user to their tickets
                    alert(result.message);
                    window.location.href = result.redirect || 'view_user_history.php';
                } else {
                    // CONFLICT: e.g. Someone else just took the same seat
                    alert('Reservation Failed: ' + result.message);
                }
            })
            .catch(error => {
                console.error('Network/Server Crash:', error);
                alert('System Malfunction: Could not communicate with booking server.');
            });
        }
    </script>
</body>
</html>
```